

Standard weryfikacji bezpieczeństwa aplikacji

Wersja 5.0.0



May 2025

Contents

Strona tytułowa	7
O standardzie	7
Prawa autorskie i licencja	7
Liderzy projektu	7
Grupa robocza	7
Pozostali główni współtwórcy	7
Pozostali współtwórcy i recenzenci	8
Przedmowa	8
Wprowadzenie	8
Zasady przyświecające wersji 5.0	8
Spojrzenie w przyszłość	9
Czym jest ASVS?	9
Zakres ASVS	9
Aplikacja	10
Bezpieczeństwo	10
Weryfikacja	10
Standard	11
Wymaganie	11
Udokumentowane decyzje dotyczące bezpieczeństwa	11
Poziomy weryfikacji bezpieczeństwa aplikacji	12
Ocena poziomów	13
Poziom 1	13
Poziom 2	14
Poziom 3	14
Który poziom osiągnąć	14
Jak korzystać z ASVS	14
Struktura ASVS	14
Strategia wydań	15
Elastyczność ASVS	15
Jak przywoływać wymagania ASVS	15
Forkowanie ASVS	16
Przypadki użycia ASVS	16
Jako szczegółowe wytyczne architektury bezpieczeństwa	16
Jako specjalistyczny przewodnik bezpiecznego kodowania	17
Jako przewodnik dla zautomatyzowanych testów jednostkowych i integracyjnych	17
Do szkoleń z bezpiecznego wytwarzania oprogramowania	17
Jako ramy wspierające zakup bezpiecznego oprogramowania	17
Stosowanie ASVS w praktyce	18

Ocena i certyfikacja	18
Stanowisko OWASP wobec certyfikacji ASVS i znaków zaufania	18
Jak weryfikować zgodność z ASVS	18
Raportowanie weryfikacji	18
Zakres weryfikacji	19
Mechanizmy weryfikacji	19
Zmiany względem wersji 4.x	20
Wprowadzenie	20
Filozofia wymagań	21
Zakres i ukierunkowanie	21
Nacisk na cele bezpieczeństwa zamiast mechanizmów	21
Udokumentowane decyzje dotyczące bezpieczeństwa	21
Zmiany strukturalne i nowe rozdziały	21
Usunięcie bezpośrednich mapowań do innych standardów	22
Ograniczenie powiązania z wytycznymi NIST Digital Identity Guidelines	22
Odejście od Common Weakness Enumeration (CWE)	23
Nowe podejście do definicji poziomów	23
Łatwiejszy poziom wejściowy	23
Złudzenie testowalności	23
Nie tylko kwestia ryzyka	24
V1 Kodowanie i sanityzacja	24
Cel kontrolny	24
V1.1 Architektura kodowania i sanityzacji	24
V1.2 Zapobieganie wstrzyknięciom	25
V1.3 Sanityzacja	26
V1.4 Pamięć, ciągi znaków i kod niezarządzany	28
V1.5 Bezpieczna deserializacja	28
Źródła	29
V2 Walidacja i logika biznesowa	30
Cel kontrolny	30
V2.1 Dokumentacja walidacji i logiki biznesowej	30
V2.2 Walidacja danych wejściowych	31
V2.3 Bezpieczeństwo logiki biznesowej	32
V2.4 Ochrona przed automatyzacją	33
Źródła	33
V3 Bezpieczeństwo frontendu webowego	33
Cel kontrolny	33
V3.1 Dokumentacja bezpieczeństwa frontendu webowego	33

V3.2 Niezamierzona interpretacja treści	34
V3.3 Konfiguracja ciasteczek	34
V3.4 Nagłówki mechanizmów bezpieczeństwa przeglądarki	35
V3.5 Separacja źródeł w przeglądarce	37
V3.6 Integralność zasobów zewnętrznych	38
V3.7 Pozostałe kwestie bezpieczeństwa przeglądarki	39
Źródła	39
V4 API i usługi sieciowe	40
Cel kontrolny	40
V4.1 Ogólne bezpieczeństwo usług sieciowych	40
V4.2 Walidacja struktury komunikatów HTTP	41
V4.3 GraphQL	42
V4.4 WebSocket	42
Źródła	43
V5 Obsługa plików	43
Cel kontrolny	43
V5.1 Dokumentacja obsługi plików	43
V5.2 Przesyłanie plików i ich zawartość	44
V5.3 Przechowywanie plików	45
V5.4 Pobieranie plików	45
Źródła	46
V6 Uwierzytelnianie	46
Cel kontrolny	46
V6.1 Dokumentacja uwierzytelniania	47
V6.2 Bezpieczeństwo haseł	47
V6.3 Ogólne bezpieczeństwo uwierzytelniania	48
V6.4 Cykl życia i odzyskiwanie czynników uwierzytelniania	50
V6.5 Ogólne wymagania uwierzytelniania wieloskładnikowego	51
V6.6 Pozapasmowe mechanizmy uwierzytelniania	52
V6.7 Kryptograficzny mechanizm uwierzytelniania	53
V6.8 Uwierzytelnianie z dostawcą tożsamości	54
Źródła	55
V7 Zarządzanie sesją	56
Cel kontrolny	56
V7.1 Dokumentacja zarządzania sesją	56
V7.2 Podstawowe bezpieczeństwo zarządzania sesją	57
V7.3 Wygasanie sesji	58
V7.4 Kończenie sesji	58

V7.5 Obrona przed nadużyciem sesji	59
V7.6 Ponowne uwierzytelnianie federacyjne	59
Źródła	60
V8 Autoryzacja	60
Cel kontrolny	60
V8.1 Dokumentacja autoryzacji	60
V8.2 Ogólny projekt autoryzacji	61
V8.3 Autoryzacja na poziomie operacji	62
V8.4 Pozostałe kwestie autoryzacji	62
Źródła	63
V9 Tokeny samowystarczalne	63
Cel kontrolny	63
V9.1 Źródło i integralność tokena	63
V9.2 Zawartość tokena	64
Źródła	65
V10 OAuth i OIDC	65
Cel kontrolny	65
V10.1 Ogólne bezpieczeństwo OAuth i OIDC	67
V10.2 Klient OAuth	68
V10.3 Serwer zasobów OAuth	68
V10.4 Serwer autoryzacji OAuth	69
V10.5 Klient OIDC	72
V10.6 Dostawca OpenID	72
V10.7 Zarządzanie zgodami	73
Źródła	73
V11 Kryptografia	74
Cel kontrolny	74
V11.1 Inwentarz kryptograficzny i dokumentacja	75
V11.2 Bezpieczna implementacja kryptografii	76
V11.3 Algorytmy szyfrowania	77
V11.4 Haszowanie i funkcje oparte na skrótach	77
V11.5 Wartości losowe	78
V11.6 Kryptografia klucza publicznego	79
V11.7 Kryptografia danych w użyciu	79
Źródła	80
V12 Bezpieczna komunikacja	80
Cel kontrolny	80
V12.1 Ogólne wytyczne bezpieczeństwa TLS	80

V12.2 Komunikacja HTTPS z usługami skierowanymi na zewnątrz	81
V12.3 Ogólne bezpieczeństwo komunikacji między usługami	82
Źródła	82
V13 Konfiguracja	83
Cel kontrolny	83
V13.1 Dokumentacja konfiguracji	83
V13.2 Konfiguracja komunikacji backendowej	84
V13.3 Zarządzanie sekretami	85
V13.4 Niezamierzony wyciek informacji	85
Źródła	86
V14 Ochrona danych	86
Cel kontrolny	86
V14.1 Dokumentacja ochrony danych	87
V14.2 Ogólna ochrona danych	87
V14.3 Ochrona danych po stronie klienta	89
Źródła	89
V15 Bezpieczne kodowanie i architektura	89
Cel kontrolny	89
V15.1 Dokumentacja bezpiecznego kodowania i architektury	90
V15.2 Architektura bezpieczeństwa i zależności	91
V15.3 Kodowanie defensywne	92
V15.4 Bezpieczna współbieżność	93
Źródła	94
V16 Logowanie zdarzeń bezpieczeństwa i obsługa błędów	94
Cel kontrolny	94
V16.1 Dokumentacja logowania zdarzeń bezpieczeństwa	95
V16.2 Ogólne logowanie	95
V16.3 Zdarzenia bezpieczeństwa	96
V16.4 Ochrona logów	97
V16.5 Obsługa błędów	97
Źródła	98
V17 WebRTC	98
Cel kontrolny	98
V17.1 Serwer TURN	99
V17.2 Media	100
V17.3 Sygnalizacja	101
Źródła	102

Załącznik A: Glosariusz	102
Załącznik B: Źródła	109
Główne projekty OWASP	109
Projekt OWASP Cheat Sheet Series	109
Projekty związane z bezpieczeństwem mobilnym	109
Projekty OWASP związane z Internetem rzeczy	109
Projekty OWASP dotyczące serverless	110
Pozostałe	110
Załącznik C: Standardy kryptograficzne	110
Inwentarz kryptograficzny i dokumentacja	111
Równoważne siły parametrów kryptograficznych	111
Wartości losowe	111
Algorytmy szyfrów	112
Tryby szyfrowania AES	113
Opakowywanie kluczy	114
Szyfrowanie uwierzytelnione	115
Funkcje skrótu	115
Funkcje skrótu do ogólnych przypadków użycia	116
Funkcje skrótu do przechowywania haseł	117
Funkcje wyprowadzania klucza (KDF)	118
Ogólne funkcje wyprowadzania klucza	118
Funkcje wyprowadzania klucza oparte na hasłach	118
Mechanizmy wymiany kluczy	118
Schematy KEX	119
Grupy Diffiego-Hellmana	119
Kody uwierzytelniania wiadomości (MAC)	120
Podpisy cyfrowe	121
Postkwantowe standardy szyfrowania	121
Załącznik D: Zalecenia	121
Wprowadzenie	121
Zalecane mechanizmy w zakresie standardu	122
Zasady bezpieczeństwa oprogramowania	122
Procesy bezpieczeństwa oprogramowania	123
Załącznik E - Współtwórcy	124

Strona tytułowa

O standardzie

Application Security Verification Standard (Standard Weryfikacji Bezpieczeństwa Aplikacji) to lista wymagań bezpieczeństwa aplikacji, z której architekci, programiści, testerzy, specjaliści ds. bezpieczeństwa, dostawcy narzędzi oraz odbiorcy mogą korzystać przy definiowaniu, tworzeniu, testowaniu i weryfikowaniu bezpiecznych aplikacji.

Prawa autorskie i licencja

Wersja 5.0.0, maj 2025



Figure 1: licencja

Copyright © 2008-2025 The OWASP Foundation.

Niniejszy dokument został udostępniony na licencji Creative Commons Uznanie autorstwa –Na tych samych warunkach 4.0 Międzynarodowe.

W przypadku ponownego wykorzystania lub rozpowszechniania należy wyraźnie poinformować innych o warunkach licencji tego dzieła.

Liderzy projektu

Elar Lang	Josh C Grossman
Jim Manico	Daniel Cuthbert

Grupa robocza

Tobias Ahnoff	Ralph Andalis	Ryan Armstrong	Gabriel Corona
Meghan Jacquot	Shanni Prutchi	Iman Sharafaldin	Eden Yardeni

Pozostali główni współtwórcy

Sjoerd Langkemper	Isaac Lewis
Mark Carney	Sandro Gauci

Pozostali współtwórcy i recenzenci

Lista pozostałych współtwórców znajduje się w Załączniku E.

Jeśli w liście zasług wersji 5.x brakuje czyjegoś nazwiska, prosimy o zgłoszenie tego w serwisie GitHub, aby osoba ta mogła zostać uwzględniona w przyszłych aktualizacjach wersji 5.x.

Application Security Verification Standard opiera się na pracy osób zaangażowanych w rozwój ASVS od wersji 1.0 (2008) do 4.0 (2019). Znaczna część struktury oraz wiele wymagań weryfikacyjnych obecnych w ASVS do dziś zostało pierwotnie opracowanych przez Andrew van der Stocka, Mike’a Boberskiego, Jeffa Williama i Dave’a Wichersa, przy udziale wielu innych współtwórców. Dziękujemy wszystkim, którzy wnieśli swój wkład w przeszłości. Pełna lista wcześniejszych współtwórców znajduje się w poprzednich wersjach dokumentu.

Przedmowa

Witamy w Application Security Verification Standard (ASVS) w wersji 5.0.

Wprowadzenie

ASVS, zapoczątkowany w 2008 roku jako efekt globalnej współpracy społeczności, definiuje kompleksowy zestaw wymagań bezpieczeństwa dotyczących projektowania, tworzenia i testowania nowoczesnych aplikacji internetowych oraz usług sieciowych.

Po wydaniu ASVS 4.0 w 2019 roku oraz jego drobnej aktualizacji (v4.0.3) w 2021 roku, wersja 5.0 stanowi istotny kamień milowy – została zmodernizowana, aby odzwierciedlać najnowsze osiągnięcia w dziedzinie bezpieczeństwa oprogramowania.

ASVS 5.0 jest wynikiem szerokiego zaangażowania liderów projektu, członków grupy roboczej oraz całej społeczności OWASP w aktualizację i udoskonalenie tego ważnego standardu.

Zasady przyświecające wersji 5.0

Ta gruntowna rewizja została opracowana z myślą o kilku kluczowych zasadach:

- **Doprecyzowany zakres i ukierunkowanie:** Ta wersja standardu została zaprojektowana tak, aby ściślej odpowiadać filarom zawartym w jego nazwie: Aplikacja, Bezpieczeństwo, Weryfikacja i Standard. Wymagania zostały przeredagowane w taki sposób, aby kłaść nacisk na zapobieganie

lukom bezpieczeństwa, a nie narzucać konkretne rozwiązania techniczne. Treść wymagań ma być zrozumiała sama w sobie i wyjaśniać, dlaczego dane wymaganie istnieje.

- Wsparcie dla udokumentowanych decyzji dotyczących bezpieczeństwa: ASVS 5.0 wprowadza wymagania dotyczące dokumentowania kluczowych decyzji z zakresu bezpieczeństwa. Zwiększa to identyfikowalność i wspiera wdrożenia uwzględniające kontekst, pozwalając organizacjom dostosować swój poziom bezpieczeństwa do własnych potrzeb i ryzyk.
- Zaktualizowane poziomy: Choć ASVS zachowuje swój trypoziomowy model, definicje poziomów ewoluowały, aby ułatwić przyjęcie standardu. Poziom 1 został zaprojektowany jako pierwszy krok we wdrażaniu ASVS, zapewniający pierwszą warstwę obrony. Poziom 2 stanowi kompleksowe ujęcie standardowych praktyk bezpieczeństwa, a poziom 3 obejmuje zaawansowane wymagania o wysokim poziomie pewności.
- Przebudowana i rozszerzona zawartość: ASVS 5.0 obejmuje około 350 wymagań w 17 rozdziałach. Rozdziały zostały uporządkowane na nowo z myślą o przejrzystości i użyteczności. Udostępniono dwukierunkowe mapowanie między wersjami 4.0 i 5.0, aby ułatwić migrację.

Spojrzenie w przyszłość

Tak jak zabezpieczanie aplikacji nigdy nie jest w pełni zakończone, tak samo nie jest zakończony rozwój ASVS. Choć wersja 5.0 to wydanie przełomowe, prace trwają nadal. To wydanie pozwala szerszej społeczności korzystać z nagromadzonych ulepszeń i uzupełnień, a jednocześnie kładzie fundamenty pod przyszłe usprawnienia. Mogą one obejmować tworzone przez społeczność wytyczne dotyczące wdrażania i weryfikacji, budowane na bazie podstawowego zestawu wymagań.

ASVS 5.0 został zaprojektowany jako solidny fundament bezpiecznego wytwarzania oprogramowania. Zapraszamy społeczność do przyjmowania tego standardu, współtworzenia go i rozwijania, aby wspólnie podnosić poziom bezpieczeństwa aplikacji.

Czym jest ASVS?

Application Security Verification Standard (ASVS) definiuje wymagania bezpieczeństwa dla aplikacji internetowych i usług sieciowych. Stanowi cenne źródło wiedzy dla każdego, kto chce projektować, tworzyć i utrzymywać bezpieczne aplikacje lub oceniać ich bezpieczeństwo.

Niniejszy rozdział przedstawia kluczowe aspekty korzystania z ASVS, w tym jego zakres, strukturę poziomów opartych na priorytetach oraz główne przypadki użycia standardu.

Zakres ASVS

Zakres ASVS wyznacza jego nazwa: Aplikacja, Bezpieczeństwo, Weryfikacja i Standard. Określa ona, które wymagania są uwzględnione, a które wykluczone —a nadrzędnym celem jest wskazanie za-

sad bezpieczeństwa, które muszą zostać spełnione. Zakres obejmuje również wymagania dotyczące dokumentacji, stanowiące fundament dla wymagań wdrożeniowych.

Dla atakujących pojęcie zakresu nie istnieje. Dlatego wymagania ASVS należy rozpatrywać łącznie z wytycznymi dotyczącymi innych aspektów cyklu życia aplikacji, w tym procesów CI/CD, hostingu oraz działań operacyjnych.

Aplikacja

ASVS definiuje „aplikację” jako tworzony produkt programowy, w który muszą zostać wbudowane mechanizmy bezpieczeństwa. ASVS nie narzuca działań w ramach cyklu wytwarzania oprogramowania ani nie dyktuje, jak aplikacja ma być budowana w potoku CI/CD; zamiast tego określa efekty w zakresie bezpieczeństwa, które muszą zostać osiągnięte w samym produkcie.

Komponenty, które obsługują, modyfikują lub walidują ruch HTTP —takie jak zapory aplikacji internetowych (WAF), moduły równoważenia obciążenia czy serwery proxy —mogą być traktowane jako część aplikacji w tych konkretnych zastosowaniach, ponieważ niektóre mechanizmy bezpieczeństwa zależą bezpośrednio od nich lub mogą być przez nie realizowane. Komponenty te należy uwzględnić przy wymaganiach dotyczących odpowiedzi buforowanych w pamięci podręcznej, ograniczania częstotliwości żądań lub ograniczania połączeń przychodzących i wychodzących na podstawie źródła i celu.

Z drugiej strony ASVS zasadniczo wyklucza wymagania, które nie odnoszą się bezpośrednio do aplikacji lub których konfiguracja leży poza jej odpowiedzialnością. Przykładowo kwestie DNS są zazwyczaj zarządzane przez odrębny zespół lub funkcję w organizacji.

Podobnie —choć aplikacja odpowiada za sposób, w jaki przetwarza dane wejściowe i generuje dane wyjściowe —jeśli zewnętrzny proces wchodzi w interakcję z aplikacją lub jej danymi, pozostaje on poza zakresem ASVS. Na przykład tworzenie kopii zapasowych aplikacji lub jej danych jest zwykle zadaniem zewnętrznego procesu i nie jest kontrolowane przez aplikację ani jej twórców.

Bezpieczeństwo

Każde wymaganie musi mieć wykazywalny wpływ na bezpieczeństwo. Brak danego wymagania musi skutkować mniej bezpieczną aplikacją, a jego wdrożenie musi zmniejszać prawdopodobieństwo lub skutki ryzyka bezpieczeństwa.

Wszystkie pozostałe kwestie, takie jak aspekty funkcjonalne, styl kodu czy wymagania wynikające z polityk, pozostają poza zakresem.

Weryfikacja

Wymaganie musi być weryfikowalne, a weryfikacja musi kończyć się decyzją „spełnione” albo „niespełnione”.

Standard

ASVS został zaprojektowany jako zbiór wymagań bezpieczeństwa, których wdrożenie oznacza zgodność ze standardem. Oznacza to, że wymagania ograniczają się do zdefiniowania celu bezpieczeństwa, który należy osiągnąć. Pozostałe powiązane informacje mogą być budowane na bazie ASVS lub powiązane z nim poprzez mapowania.

W szczególności —OWASP prowadzi wiele projektów, a ASVS celowo unika pokrywania się z treścią innych z nich. Przykładowo programista może zapytać: „jak wdrożyć dane wymaganie w mojej konkretnej technologii lub środowisku”—na to odpowiada projekt Cheat Sheet Series. Weryfikator może zapytać: „jak przetestować to wymaganie w tym środowisku”—na to odpowiada projekt Web Security Testing Guide.

Choć ASVS nie jest przeznaczony wyłącznie dla ekspertów ds. bezpieczeństwa, zakłada, że czytelnik dysponuje wiedzą techniczną pozwalającą zrozumieć treść lub umiejętnością samodzielnego zgłębienia poszczególnych zagadnień.

Wymaganie

Słowo „wymaganie” jest używane w ASVS w ścisłym znaczeniu —opisuje to, co musi zostać osiągnięte, aby je spełnić. ASVS zawiera wyłącznie wymagania (must) i nie zawiera zaleceń (should) jako głównego warunku.

Innymi słowy, zalecenia —niezależnie od tego, czy stanowią tylko jedną z wielu możliwych opcji rozwiązania problemu, czy dotyczą stylu kodu —nie spełniają definicji wymagania.

Wymagania ASVS mają odnosić się do konkretnych zasad bezpieczeństwa, nie będąc przy tym nadmiernie związane z konkretną implementacją czy technologią, a jednocześnie samodzielnie wyjaśniać, dlaczego istnieją. Oznacza to również, że wymagania nie są budowane wokół określonej metody weryfikacji ani implementacji.

Udokumentowane decyzje dotyczące bezpieczeństwa

W bezpieczeństwie oprogramowania wczesne zaplanowanie projektu zabezpieczeń oraz mechanizmów, które zostaną użyte, prowadzi do spójniejszej i bardziej niezawodnej implementacji w gotowym produkcie lub funkcjonalności.

Ponadto w przypadku niektórych wymagań implementacja będzie złożona i silnie zależna od potrzeb konkretnej aplikacji. Typowe przykłady to uprawnienia, walidacja danych wejściowych oraz mechanizmy ochronne wokół różnych poziomów danych wrażliwych.

Aby to uwzględnić —zamiast ogólnikowych stwierdzeń w rodzaju „wszystkie dane muszą być szyfrowane” lub prób objęcia jednym wymaganiem każdego możliwego przypadku użycia —wprowadzono wymagania dotyczące dokumentacji, które nakazują udokumentowanie podejścia twórcy aplikacji do tego typu mechanizmów oraz ich konfiguracji. Dokumentację można następnie

ocenić pod kątem adekwatności, a rzeczywistą implementację porównać z nią, aby sprawdzić, czy odpowiada oczekiwaniom.

Wymagania te służą udokumentowaniu decyzji, które organizacja tworząca aplikację podjęła w kwestii sposobu wdrożenia określonych wymagań bezpieczeństwa.

Wymagania dotyczące dokumentacji znajdują się zawsze w pierwszej sekcji rozdziału (choć nie każdy rozdział je zawiera) i zawsze mają powiązane wymaganie wdrożeniowe, zgodnie z którym udokumentowane decyzje powinny zostać faktycznie wprowadzone w życie. Chodzi o to, że weryfikacja istnienia dokumentacji oraz weryfikacja rzeczywistej implementacji to dwie odrębne czynności.

Za włączeniem tych wymagań stoją dwa kluczowe powody. Pierwszy: wymaganie bezpieczeństwa często wiąże się z egzekwowaniem reguł — np. jakie typy plików można przysyłać, jakie mechanizmy kontroli biznesowej powinny być egzekwowane, jakie znaki są dozwolone w danym polu. Reguły te będą różne dla każdej aplikacji, dlatego ASVS nie może ich ogólnie zdefiniować — nie pomoże tu także ściągą ani bardziej szczegółowa odpowiedź. Podobnie, bez udokumentowania tych decyzji nie będzie możliwa weryfikacja wymagań, które je wdrażają.

Drugi powód: w przypadku niektórych wymagań istotne jest pozostawienie zespołowi tworzącemu aplikację elastyczności w podejściu do konkretnych wyzwań bezpieczeństwa. Przykładowo we wcześniejszych wersjach ASVS reguły wygasania sesji były bardzo restrykcyjnie zdefiniowane. W praktyce wiele aplikacji — zwłaszcza konsumenckich — stosuje znacznie łagodniejsze reguły i woli wdrożyć w zamian inne mechanizmy ograniczające ryzyko. Wymagania dotyczące dokumentacji jawnie dopuszczają taką elastyczność.

Oczywiście nie oczekuje się, że decyzje te będą podejmowane i dokumentowane przez poszczególnych programistów — to organizacja jako całość powinna je podejmować i zadbać o ich zakomunikowanie programistom, którzy następnie będą ich przestrzegać.

Dostarczanie programistom specyfikacji i projektów nowych funkcji to standardowy element wytwarzania oprogramowania. Podobnie oczekuje się, że programiści będą korzystać ze wspólnych komponentów i mechanizmów interfejsu użytkownika, zamiast każdorazowo podejmować własne decyzje. Rozszerzenie tej praktyki na bezpieczeństwo nie powinno więc dziwić ani budzić kontrowersji.

Istnieje również elastyczność co do sposobu realizacji. Decyzje dotyczące bezpieczeństwa mogą zostać udokumentowane w postaci dokumentu, do którego programiści mają się odwoływać. Alternatywnie mogą zostać udokumentowane i zaimplementowane we wspólnej bibliotece kodu, z której wszyscy programiści mają obowiązek korzystać. W obu przypadkach osiągnąony jest pożądany rezultat.

Poziomy weryfikacji bezpieczeństwa aplikacji

ASVS definiuje trzy poziomy weryfikacji bezpieczeństwa, z których każdy kolejny zwiększa głębokość i złożoność. Ogólnym założeniem jest, aby organizacje zaczynały od pierwszego poziomu, aby zająć się najbardziej krytycznymi kwestiami bezpieczeństwa, a następnie przechodziły na wyższe poziomy

stosownie do potrzeb organizacji i aplikacji. W dokumencie oraz w treści wymagań poziomy mogą być oznaczane jako L1, L2 i L3.

Każdy poziom ASVS wskazuje wymagania bezpieczeństwa, których spełnienie jest konieczne do jego osiągnięcia, przy czym wymagania pozostałych, wyższych poziomów mają charakter zaleceń.

Aby uniknąć duplikowania wymagań lub wymagań, które na wyższych poziomach tracą aktualność, niektóre wymagania obowiązują na określonym poziomie, lecz na wyższych poziomach mają bardziej rygorystyczne warunki.

Ocena poziomów

Poziomy zostały zdefiniowane w drodze opartej na priorytetach oceny każdego wymagania, bazującej na doświadczeniu we wdrażaniu i testowaniu wymagań bezpieczeństwa. Główny nacisk położono na porównanie redukcji ryzyka z nakładem pracy potrzebnym do wdrożenia wymagania. Kolejnym kluczowym czynnikiem było utrzymanie niskiego progu wejścia.

Redukcja ryzyka uwzględnia stopień, w jakim wymaganie obniża poziom ryzyka bezpieczeństwa w aplikacji, biorąc pod uwagę klasyczne czynniki wpływu: poufność, integralność i dostępność, a także to, czy dane wymaganie stanowi podstawową warstwę obrony, czy raczej element obrony w głąb.

Rygorystyczne dyskusje zarówno nad kryteriami, jak i nad decyzjami o przypisaniu poziomów doprowadziły do podziału, który powinien sprawdzać się w zdecydowanej większości przypadków — przy założeniu, że nie będzie on w 100% dopasowany do każdej sytuacji. Oznacza to, że w pewnych przypadkach organizacje mogą zdecydować się na wcześniejsze nadanie priorytetu wymaganiom z wyższego poziomu, kierując się własną, specyficzną oceną ryzyka.

Rodzaje wymagań na poszczególnych poziomach można scharakteryzować następująco.

Poziom 1

Ten poziom zawiera minimalny zestaw wymagań, które należy rozważyć przy zabezpieczaniu aplikacji, i stanowi krytyczny punkt wyjścia. Obejmuje około 20% wymagań ASVS. Celem tego poziomu jest ograniczenie liczby wymagań do minimum, aby obniżyć próg wejścia.

Są to na ogół wymagania krytyczne lub podstawowe, stanowiące pierwszą warstwę obrony przed powszechnymi atakami, które do wykorzystania nie wymagają innych podatności ani warunków wstępnych.

Oprócz wymagań pierwszej warstwy obrony znajdują się tu wymagania, których znaczenie na wyższych poziomach maleje — na przykład te dotyczące haseł. Są one ważniejsze na poziomie 1, ponieważ od wyższych poziomów zaczynają obowiązywać wymagania dotyczące uwierzytelniania wieloskładnikowego.

Poziom 1 niekoniecznie da się zweryfikować testem penetracyjnym prowadzonym przez zewnętrznego testera bez dostępu do dokumentacji lub kodu (tzw. testowanie „czarnoskrzynkowe”), choć mniejsza liczba wymagań powinna ułatwić weryfikację.

Poziom 2

Do tego poziomu bezpieczeństwa powinna dążyć większość aplikacji. Około 50% wymagań ASVS należy do poziomu L2, co oznacza, że aby osiągnąć zgodność z L2, aplikacja musi wdrożyć około 70% wymagań ASVS (wszystkie wymagania L1 i L2).

Wymagania te dotyczą na ogół rzadszych ataków lub bardziej złożonych zabezpieczeń przed atakami powszechnymi. Mogą nadal stanowić pierwszą warstwę obrony albo wymagać spełnienia określonych warunków wstępnych, aby atak mógł się powieść.

Poziom 3

Ten poziom powinien być celem aplikacji, które chcą wykazać najwyższy poziom bezpieczeństwa. Obejmuje ostatnie ~30% wymagań niezbędnych do pełnej zgodności.

Wymagania w tej grupie to na ogół mechanizmy obrony w głąb lub inne przydatne, lecz trudne we wdrożeniu zabezpieczenia.

Który poziom osiągnąć

Poziomy oparte na priorytetach mają odzwierciedlać dojrzałość organizacji i aplikacji w obszarze bezpieczeństwa aplikacji. ASVS nie wskazuje odgórnie, na jakim poziomie powinna znajdować się dana aplikacja –to organizacja powinna przeanalizować swoje ryzyka i zdecydować, do jakiego poziomu jej zdaniem powinna dążyć, w zależności od wrażliwości aplikacji oraz, rzecz jasna, oczekiwań jej użytkowników.

Przykładowo start-up na wczesnym etapie rozwoju, gromadzący jedynie ograniczoną ilość danych wrażliwych, może zdecydować się skupić na poziomie 1 jako początkowym celu bezpieczeństwa, natomiast bankowi trudno będzie uzasadnić przed klientami cokolwiek poniżej poziomu 3 dla aplikacji bankowości internetowej.

Jak korzystać z ASVS

Struktura ASVS

ASVS składa się łącznie z około 350 wymagań podzielonych na 17 rozdziałów, z których każdy dzieli się dalej na sekcje.

Celem podziału na rozdziały i sekcje jest ułatwienie wyboru lub odfiltrowania rozdziałów i sekcji w zależności od tego, co jest istotne dla danej aplikacji. Przykładowo dla API działającego w komunikacji maszyna-maszyna wymagania rozdziału V3 dotyczące frontendów webowych nie będą miały zastosowania. Jeśli aplikacja nie korzysta z OAuth ani WebRTC, te rozdziały również można pominąć.

Strategia wydań

Wydania ASVS są numerowane według wzorca „Major.Minor.Patch” (główne.poboczne.poprawka), a numery informują, co zmieniło się w danym wydaniu. W wydaniu głównym zmienia się pierwsza liczba, w wydaniu pobocznym – druga, a w poprawce – trzecia.

- Wydanie główne – pełna reorganizacja; zmianie mogło ulec niemal wszystko, łącznie z numeracją wymagań. Konieczna będzie ponowna ocena zgodności (na przykład 4.0.3 -> 5.0.0).
- Wydanie poboczne – wymagania mogą zostać dodane lub usunięte, ale ogólna numeracja pozostaje bez zmian. Ponowna ocena zgodności będzie konieczna, lecz powinna być łatwiejsza (na przykład 5.0.0 -> 5.1.0).
- Poprawka – wymagania mogą zostać usunięte (na przykład jako duplikaty lub nieaktualne) lub złagodzone, ale aplikacja zgodna z poprzednim wydaniem będzie zgodna również z poprawką (na przykład 5.0.0 -> 5.0.1).

Powyższe odnosi się wyłącznie do wymagań ASVS. Zmiany w tekście towarzyszącym i pozostałych treściach, takich jak załączniki, nie będą traktowane jako zmiany łamiące zgodność wsteczną.

Elastyczność ASVS

Kilka z opisanych wyżej mechanizmów – takich jak wymagania dotyczące dokumentacji czy system poziomów – pozwala korzystać z ASVS w sposób bardziej elastyczny i dopasowany do organizacji.

Dodatkowo zdecydowanie zachęca się organizacje do tworzenia własnych forków standardu, dostosowanych do organizacji lub domeny, które dostosowują wymagania na podstawie specyficznych cech i poziomów ryzyka ich aplikacji. Należy jednak zachować identyfikowalność, tak aby spełnienie wymagania 4.1.1 oznaczało to samo we wszystkich wersjach.

Najlepiej, aby każda organizacja stworzyła własną, dopasowaną wersję ASVS, pomijając nieistotne sekcje (np. GraphQL, WebSockets, SOAP – jeśli nie są używane). Własna wersja ASVS lub jej uzupełnienie to również dobre miejsce, aby zamieścić wytyczne wdrożeniowe specyficzne dla organizacji, wskazujące biblioteki lub zasoby, z których należy korzystać przy spełnianiu wymagań.

Jak przywoływać wymagania ASVS

Każde wymaganie ma identyfikator w formacie <rozdział>.<sekcja>.<wymaganie>, gdzie każdy element jest liczbą. Na przykład: 1.11.3.

- Wartość <rozdział> odpowiada rozdziałowi, z którego pochodzi wymaganie; na przykład wszystkie wymagania 1.#.# pochodzą z rozdziału „Kodowanie i sanityzacja”.
- Wartość <sekcja> odpowiada sekcji w obrębie tego rozdziału, w której znajduje się wymaganie; na przykład wszystkie wymagania 1.2.# znajdują się w sekcji „Zapobieganie wstrzyknięciom” rozdziału „Kodowanie i sanityzacja”.

- Wartość `<wymaganie>` identyfikuje konkretne wymaganie w obrębie rozdziału i sekcji, na przykład 1.2.5, które w wersji 5.0.0 niniejszego standardu brzmi:

Zweryfikuj, że aplikacja chroni przed wstrzyknięciem poleceń systemu operacyjnego oraz że wywołania systemowe używają sparametryzowanych zapytań systemowych lub stosują kontekstowe kodowanie danych wyjściowych wiersza poleceń.

Ponieważ identyfikatory mogą się zmieniać między wersjami standardu, w innych dokumentach, raportach lub narzędziach zaleca się stosowanie formatu: `v<wersja>-<rozdział>.<sekcja>.<wymaganie>`, gdzie „wersja” to znacznik wersji ASVS. Na przykład `v5.0.0-1.2.5` oznaczałoby konkretne piąte wymaganie w sekcji „Zapobieganie wstrzyknięciom” rozdziału „Kodowanie i sanityzacja” z wersji 5.0.0. (Można to podsumować jako `v<wersja>-<identyfikator_wymagania>`.)

Uwaga: litera `v` poprzedzająca numer wersji w tym formacie powinna być zawsze mała.

Jeśli identyfikatory są używane bez elementu `v<wersja>`, należy przyjąć, że odnoszą się do najnowszej wersji Application Security Verification Standard. W miarę rozwoju i zmian standardu staje się to problematyczne — dlatego autorzy tekstów i programiści powinni uwzględniać element wersji.

Listy wymagań ASVS są udostępniane w formatach CSV, JSON i innych, które mogą być przydatne do celów referencyjnych lub programistycznych.

Forkowanie ASVS

Organizacje mogą czerpać korzyści z przyjęcia ASVS, wybierając jeden z trzech poziomów lub tworząc fork dostosowany do własnej domeny, który koryguje wymagania odpowiednio do poziomu ryzyka aplikacji. Tego typu forki są mile widziane, pod warunkiem zachowania identyfikowalności — tak aby spełnienie wymagania 4.1.1 oznaczało to samo we wszystkich wersjach.

Najlepiej, aby każda organizacja stworzyła własną, dopasowaną wersję ASVS, pomijając nieistotne sekcje (np. GraphQL, WebSockets, SOAP — jeśli nie są używane). Punktem wyjścia forka powinien być poziom 1 ASVS, z przejściem na poziomy 2 lub 3 stosownie do ryzyka aplikacji.

Przypadki użycia ASVS

ASVS może służyć do oceny bezpieczeństwa aplikacji — temat ten został szerzej omówiony w kolejnym rozdziale. Zidentyfikowano jednak także szereg innych potencjalnych zastosowań ASVS (lub jego forka).

Jako szczegółowe wytyczne architektury bezpieczeństwa

Jednym z częstszych zastosowań Application Security Verification Standard jest wykorzystanie go jako źródła wiedzy przez architektów bezpieczeństwa. Zasobów opisujących, jak budować bez-

pieczną architekturę aplikacji, jest niewiele —zwłaszcza w odniesieniu do nowoczesnych aplikacji. ASVS może wypełnić te luki, pozwalając architektom bezpieczeństwa dobrać lepsze mechanizmy dla typowych problemów, takich jak wzorce ochrony danych czy strategie walidacji danych wejściowych. Szczególnie przydatne będą tu wymagania dotyczące architektury i dokumentacji.

Jako specjalistyczny przewodnik bezpiecznego kodowania

ASVS może posłużyć jako podstawa do przygotowania przewodnika bezpiecznego kodowania na etapie wytwarzania aplikacji, pomagając programistom pamiętać o bezpieczeństwie podczas tworzenia oprogramowania. Choć ASVS może stanowić bazę, organizacje powinny przygotować własne, konkretne wytyczne —jasne i ujednolicone, najlepiej opracowane na podstawie wskazówek inżynierów lub architektów bezpieczeństwa. W rozwinięciu tej praktyki zachęca się organizacje, aby tam, gdzie to możliwe, przygotowywały zatwierdzone mechanizmy bezpieczeństwa i biblioteki, do których wytyczne będą się odwoływać i z których będą korzystać programiści.

Jako przewodnik dla zautomatyzowanych testów jednostkowych i integracyjnych

ASVS został zaprojektowany tak, aby był wysoce testowalny. Niektóre weryfikacje będą miały charakter techniczny, podczas gdy inne wymagania (np. architektoniczne i dotyczące dokumentacji) mogą wymagać przeglądu dokumentacji lub architektury. Budując testy jednostkowe i integracyjne, które testują i fuzzują konkretne, istotne przypadki nadużyć powiązane z wymaganiami weryfikowalnymi środkami technicznymi, łatwiej będzie sprawdzać przy każdym buildzie, czy te mechanizmy działają poprawnie. Przykładowo do zestawu testów kontrolera logowania można dodać testy parametru nazwy użytkownika pod kątem typowych domyślnych nazw użytkowników, enumeracji kont, ataków siłowych, wstrzyknięć LDAP i SQL oraz XSS. Analogicznie test parametru hasła powinien obejmować typowe hasła, długość hasła, wstrzyknięcie bajtu null, usunięcie parametru, XSS i inne.

Do szkoleń z bezpiecznego wytwarzania oprogramowania

ASVS może również służyć do zdefiniowania cech bezpiecznego oprogramowania. Wiele kursów „bezpiecznego kodowania”to w istocie kursy etycznego hakowania z lekką domieszką wskazówek programistycznych. Niekoniecznie pomaga to programistom pisać bezpieczniejszy kod. Zamiast tego kursy bezpiecznego wytwarzania oprogramowania mogą opierać się na ASVS, z silnym naciskiem na pozytywne mechanizmy w nim opisane —zamiast na listę dziesięciu negatywnych rzeczy, których robić nie należy. Struktura ASVS zapewnia też logiczny układ do omawiania kolejnych zagadnień przy zabezpieczaniu aplikacji.

Jako ramy wspierające zakup bezpiecznego oprogramowania

ASVS doskonale sprawdza się jako ramy wspierające zakup bezpiecznego oprogramowania lub usług programistycznych na zamówienie. Kupujący może po prostu postawić wymóg, aby nabywane opro-

gramowanie zostało wytworzone zgodnie z poziomem X ASVS, i zażądać od sprzedawcy wykazania, że oprogramowanie ten poziom spełnia.

Stosowanie ASVS w praktyce

Różne zagrożenia mają różne motywacje. Niektóre branże dysponują unikalnymi zasobami informacyjnymi i technologicznymi oraz podlegają specyficznym dla danej domeny wymogom zgodności regulacyjnej.

Zdecydowanie zachęca się organizacje, aby dogłębnie przeanalizowały swoją unikalną charakterystykę ryzyka wynikającą z natury prowadzonej działalności i na podstawie tego ryzyka oraz wymagań biznesowych określiły odpowiedni poziom ASVS.

Ocena i certyfikacja

Stanowisko OWASP wobec certyfikacji ASVS i znaków zaufania

OWASP, jako neutralna wobec dostawców organizacja non-profit, nie certyfikuje żadnych dostawców, weryfikatorów ani oprogramowania. Żadne poświadczenie, znak zaufania ani certyfikat deklarujący zgodność z ASVS nie jest oficjalnie zatwierdzony przez OWASP, dlatego organizacje powinny zachować ostrożność wobec deklaracji certyfikacji ASVS składanych przez strony trzecie.

Organizacje mogą oferować usługi poświadczające, o ile nie twierdzą, że posiadają oficjalną certyfikację OWASP.

Jak weryfikować zgodność z ASVS

ASVS celowo nie narzuca dokładnego sposobu weryfikacji zgodności na poziomie przewodnika testowania. Warto jednak podkreślić kilka kluczowych kwestii.

Raportowanie weryfikacji

Tradycyjne raporty z testów penetracyjnych zgłaszają problemy „przez wyjątek”, wymieniając wyłącznie niespełnione wymagania. Natomiast raport certyfikacyjny ASVS powinien zawierać zakres, podsumowanie wszystkich sprawdzonych wymagań, wymagania, przy których odnotowano wyjątki, oraz wskazówki dotyczące rozwiązania problemów. Niektóre wymagania mogą nie mieć zastosowania (np. zarządzanie sesją w bezstanowych API) — musi to zostać odnotowane w raporcie.

Zakres weryfikacji

Organizacja tworząca aplikację z reguły nie wdroży wszystkich wymagań, ponieważ część z nich może być nieistotna lub mniej znacząca ze względu na funkcjonalność aplikacji. Weryfikator powinien jasno określić zakres weryfikacji, w tym poziom, który organizacja stara się osiągnąć, oraz wymagania, które zostały uwzględnione. Zakres powinien być opisany z perspektywy tego, co uwzględniono, a nie tego, co pominięto. Weryfikator powinien również przedstawić opinię o zasadności wykluczenia wymagań, które nie zostały wdrożone.

Powinno to pozwolić odbiorcy raportu z weryfikacji zrozumieć jej kontekst i podjąć świadomą decyzję o poziomie zaufania, jakim może obdarzyć aplikację.

Organizacje certyfikujące mogą wybierać własne metody testowania, ale powinny ujawnić je w raporcie, a testy powinny być w miarę możliwości powtarzalne. Do weryfikacji poszczególnych aspektów, takich jak walidacja danych wejściowych, mogą być stosowane różne metody – np. ręczne testy penetracyjne lub analiza kodu źródłowego – w zależności od aplikacji i wymagań.

Mechanizmy weryfikacji

Weryfikacja poszczególnych wymagań ASVS może wymagać zastosowania szeregu różnych technik. Poza testami penetracyjnymi (z użyciem prawidłowych danych uwierzytelniających, aby uzyskać pełne pokrycie aplikacji) weryfikacja wymagań ASVS może wymagać dostępu do dokumentacji, kodu źródłowego, konfiguracji oraz osób zaangażowanych w proces wytwórczy –zwłaszcza przy weryfikacji wymagań L2 i L3. Standardową praktyką jest dostarczanie solidnych dowodów ustaleń wraz ze szczegółową dokumentacją, która może obejmować dokumenty robocze, zrzuty ekranu, skrypty i logi z testów. Samo uruchomienie zautomatyzowanego narzędzia bez gruntownych testów jest niewystarczające do certyfikacji, ponieważ każde wymaganie musi zostać w sposób weryfikowalny przetestowane.

Wykorzystanie automatyzacji do weryfikacji wymagań ASVS to temat budzący nieustanne zainteresowanie. Warto zatem doprecyzować kilka kwestii związanych z testowaniem automatycznym i czarnoskrzynkowym.

Rola zautomatyzowanych narzędzi do testowania bezpieczeństwa

Zautomatyzowane narzędzia do testowania bezpieczeństwa, takie jak narzędzia dynamicznej i statycznej analizy bezpieczeństwa aplikacji (DAST i SAST), poprawnie wdrożone w potoku budowania, mogą być w stanie zidentyfikować niektóre problemy bezpieczeństwa, które nigdy nie powinny wystąpić. Jednak bez starannej konfiguracji i dostrojenia nie zapewnią wymaganego pokrycia, a poziom szumu uniemożliwi zidentyfikowanie i wyeliminowanie rzeczywistych problemów bezpieczeństwa.

Choć narzędzia te mogą pokryć niektóre z bardziej podstawowych i prostych wymagań technicznych, na przykład dotyczących kodowania danych wyjściowych czy sanitizacji, należy koniecznie zaznaczyć, że nie będą one w stanie w pełni zweryfikować wielu bardziej złożonych wymagań ASVS ani

tych dotyczących logiki biznesowej i kontroli dostępu.

W przypadku mniej oczywistych wymagań automatyzacja nadal może być wykorzystana, ale konieczne będzie napisanie weryfikacji specyficznych dla danej aplikacji. Mogą one przypominać testy jednostkowe i integracyjne, z których organizacja być może już korzysta. Możliwe zatem, że istniejącą infrastrukturę automatyzacji testów da się wykorzystać do napisania testów specyficznych dla ASVS. Choć będzie to wymagało krótkoterminowej inwestycji, długoterminowe korzyści z możliwości ciągłej weryfikacji tych wymagań ASVS będą znaczące.

Podsumowując: testowalne za pomocą automatyzacji != uruchomienie gotowego narzędzia z półki.

Rola testów penetracyjnych

Choć poziom L1 w wersji 4.0 był zoptymalizowany pod kątem testowania „czarnoskrzynkowego” (bez dokumentacji i bez kodu źródłowego), już wtedy standard jasno wskazywał, że nie jest to skuteczne działanie poświadczające i powinno być aktywnie odradzane.

Testowanie bez dostępu do niezbędnych dodatkowych informacji jest niewydajnym i nieskutecznym mechanizmem weryfikacji bezpieczeństwa, ponieważ pomija możliwość przeglądu kodu źródłowego, identyfikacji zagrożeń i brakujących mechanizmów oraz przeprowadzenia znacznie dokładniejszego testu w krótszym czasie.

Zdecydowanie zachęca się do przeprowadzania testów penetracyjnych opartych na dokumentacji lub kodzie źródłowym (hybrydowych), z pełnym dostępem do twórców aplikacji i jej dokumentacji, zamiast tradycyjnych testów penetracyjnych. Będzie to z pewnością konieczne do zweryfikowania wielu wymagań ASVS.

Zmiany względem wersji 4.x

Wprowadzenie

Użytkownikom zaznajomionym z wersją 4.x standardu może pomóc przegląd kluczowych zmian wprowadzonych w wersji 5.0, obejmujących treść, zakres oraz filozofię leżącą u podstaw standardu.

Z 286 wymagań wersji 4.0.3 jedynie 11 pozostało bez zmian, a 15 przeszło drobne korekty gramatyczne niezmieniające ich znaczenia. Łącznie 109 wymagań (38%) nie funkcjonuje już w wersji 5.0 jako odrębne wymagania — 50 zostało po prostu usuniętych, 28 usunięto jako duplikaty, a 31 scalono z innymi wymaganiami. Pozostałe zostały w jakiś sposób zrewidowane. Nawet wymagania, których treść nie uległa istotnej zmianie, mają inne identyfikatory ze względu na zmianę kolejności lub restrykturyzację.

Aby ułatwić przejście na wersję 5.0, udostępniono dokumenty mapujące, które pomagają prześledzić, jak wymagania wersji 4.x odpowiadają wymaganiom wersji 5.0. Mapowania te nie są powiązane z wersjonowaniem wydań i mogą być w razie potrzeby aktualizowane lub doprecyzowywane.

Filozofia wymagań

Zakres i ukierunkowanie

Wersja 4.x zawierała wymagania, które nie mieściły się w zamierzonym zakresie standardu — zostały one usunięte. Wykluczono również wymagania, które nie spełniały kryteriów zakresu wersji 5.0 lub nie były weryfikowalne.

Nacisk na cele bezpieczeństwa zamiast mechanizmów

W wersji 4.x wiele wymagań koncentrowało się na konkretnych mechanizmach, a nie na leżących u ich podstaw celach bezpieczeństwa. W wersji 5.0 wymagania skupiają się na celach bezpieczeństwa, przywołując konkretne mechanizmy tylko wtedy, gdy stanowią jedyne praktyczne rozwiązanie, albo podając je jako przykłady lub wskazówki uzupełniające.

Podejście to uwzględnia fakt, że dany cel bezpieczeństwa można osiągnąć wieloma metodami, i pozwala uniknąć zbędnej nakazowości, która mogłaby ograniczać elastyczność organizacji.

Dodatkowo wymagania dotyczące tego samego problemu bezpieczeństwa zostały tam, gdzie było to zasadne, skonsolidowane.

Udokumentowane decyzje dotyczące bezpieczeństwa

Choć koncepcja udokumentowanych decyzji dotyczących bezpieczeństwa może wydawać się w wersji 5.0 nowa, stanowi ona ewolucję wcześniejszych wymagań wersji 4.0 związanych ze stosowaniem polityk i modelowaniem zagrożeń. Wcześniej niektóre wymagania w sposób dorozumiany wymagały analizy niezbędnej do wdrożenia mechanizmów bezpieczeństwa, na przykład określenia dozwolonych połączeń sieciowych.

Aby zapewnić dostępność informacji niezbędnych do implementacji i weryfikacji, oczekiwania te są teraz jawnie zdefiniowane jako wymagania dotyczące dokumentacji — dzięki czemu są jasne, wykonalne i weryfikowalne.

Zmiany strukturalne i nowe rozdziały

Kilka rozdziałów wersji 5.0 wprowadza zupełnie nową treść:

- OAuth i OIDC —ze względu na powszechne przyjęcie tych protokołów do delegowania dostępu i jednokrotnego logowania dodano dedykowane wymagania obejmujące różnorodne scenariusze, z jakimi mogą zetknąć się programiści. Obszar ten może z czasem przekształcić się w samodzielny standard, podobnie jak potraktowano wymagania dotyczące urządzeń mobilnych i IoT w poprzednich wersjach.
- WebRTC —wraz ze wzrostem popularności tej technologii jej specyficzne kwestie i wyzwania bezpieczeństwa są teraz omawiane w dedykowanej sekcji.

Dołożono również starań, aby rozdziały i sekcje były zorganizowane wokół spójnych zestawów powiązanych wymagań.

Ta restrukturyzacja doprowadziła do powstania dodatkowych rozdziałów:

- Tokeny samowystarczalne —wcześniej zgrupowane w ramach zarządzania sesją, obecnie są uznawane za odrębny mechanizm i fundament komunikacji bezstanowej (np. w OAuth i OIDC). Ze względu na ich specyficzne implikacje dla bezpieczeństwa poświęcono im dedykowany rozdział, wprowadzając w wersji 5.x kilka nowych wymagań.
- Bezpieczeństwo frontendu webowego —wraz z rosnącą złożonością aplikacji przeglądarkowych i upowszechnieniem architektur opartych wyłącznie na API wymagania dotyczące bezpieczeństwa frontendu zostały wydzielone do osobnego rozdziału.
- Bezpieczne kodowanie i architektura —zgrupowano tu nowe wymagania dotyczące ogólnych praktyk bezpieczeństwa, które nie pasowały do istniejących rozdziałów.

Pozostałe zmiany organizacyjne w wersji 5.0 wprowadzono w celu doprecyzowania intencji. Przykładowo wymagania dotyczące walidacji danych wejściowych przeniesiono do logiki biznesowej —co odzwierciedla ich rolę w egzekwowaniu reguł biznesowych —zamiast grupować je z sanitizacją i kodowaniem.

Dawny rozdział V1 Architektura został usunięty. Jego początkowa sekcja zawierała wymagania wykraczające poza zakres standardu, a kolejne sekcje rozdzielono pomiędzy właściwe rozdziały, usuwając duplikaty i doprecyzowując wymagania tam, gdzie było to konieczne.

Usunięcie bezpośrednich mapowań do innych standardów

Bezpośrednie mapowania do innych standardów zostały usunięte z głównej części standardu. Celem jest przygotowanie mapowania z projektem OWASP Common Requirement Enumeration (CRE), który z kolei powiąże ASVS z szeregiem projektów OWASP i standardów zewnętrznych.

Bezpośrednie mapowania do CWE i NIST nie są już utrzymywane, co wyjaśniono poniżej.

Ograniczenie powiązania z wytycznymi NIST Digital Identity Guidelines

Wytyczne NIST Digital Identity Guidelines (SP 800-63) od dawna służą jako punkt odniesienia dla mechanizmów uwierzytelniania i autoryzacji. W wersji 4.x niektóre rozdziały były ściśle dopasowane do struktury i terminologii NIST.

Choć wytyczne te pozostają ważnym punktem odniesienia, ściśle dopasowanie rodziło trudności, w tym stosowanie mniej rozpoznawalnej terminologii, powielanie podobnych wymagań oraz niekompletne mapowania. Wersja 5.0 odchodzi od tego podejścia na rzecz większej przejrzystości i adekwatności.

Odejście od Common Weakness Enumeration (CWE)

Common Weakness Enumeration (CWE) dostarcza użytecznej taksonomii słabości bezpieczeństwa oprogramowania. Jednak trudności takie jak istnienie CWE będących wyłącznie kategoriami, problemy z przypisaniem wymagania do pojedynczego CWE oraz obecność nieprecyzyjnych mapowań w wersji 4.x doprowadziły do decyzji o zaprzestaniu bezpośrednich mapowań CWE w wersji 5.0.

Nowe podejście do definicji poziomów

Wersja 4.x opisywała poziomy jako L1 („Minimalny”), L2 („Standardowy”) i L3 („Zaawansowany”), sugerując, że wszystkie aplikacje przetwarzające dane wrażliwe powinny spełniać co najmniej L2.

Wersja 5.0 rozwiązuje kilka problemów związanych z tym podejściem, które opisano w kolejnych akapitach.

Od strony praktycznej: podczas gdy wersja 4.x używała symboli zaznaczenia jako wskaźników poziomu, wersja 5.x stosuje prostą liczbę we wszystkich formatach standardu, w tym Markdown, PDF, DOCX, CSV, JSON i XML. Dla zachowania zgodności wstecznej generowane są również starsze warianty plików CSV, JSON i XML, które nadal używają symboli zaznaczenia.

Łatwiejszy poziom wejściowy

Opinie użytkowników wskazywały, że duża liczba wymagań poziomu 1 (~120) w połączeniu z określeniem go jako poziomu „minimalnego”, niewystarczającego dla większości aplikacji, zniechęcała do przyjęcia standardu. Wersja 5.0 obniża ten próg, definiując poziom 1 przede wszystkim wokół wymagań pierwszej warstwy obrony, co przekłada się na jaśniejsze i mniej liczne wymagania na tym poziomie. Ujmując to liczbowo: w wersji 4.0.3 istniało 128 wymagań L1 na łącznie 278 wymagań, co stanowiło 46%. W wersji 5.0.0 jest 70 wymagań L1 na łącznie 345 wymagań, co stanowi 20%.

Złudzenie testowalności

Kluczowym czynnikiem doboru mechanizmów do poziomu 1 w wersji 4.x była ich przydatność do oceny w drodze zewnętrznych testów penetracyjnych „czarnoskrzynkowych”. Podejście to nie było jednak w pełni zgodne z przeznaczeniem poziomu 1 jako minimalnego zestawu mechanizmów bezpieczeństwa. Część użytkowników uważała, że poziom 1 nie wystarcza do zabezpieczenia aplikacji, inni z kolei uznawali go za zbyt trudny do przetestowania.

Opieranie się na testowalności jako kryterium jest zarówno względne, jak i miejscami mylące. To, że wymaganie jest testowalne, nie gwarantuje, że można je przetestować w sposób zautomatyzowany lub prosty. Co więcej, wymagania najłatwiejsze do przetestowania nie zawsze mają największy wpływ na bezpieczeństwo ani nie są najprostsze do wdrożenia.

W związku z tym w wersji 5.0 decyzje o poziomach podejmowano przede wszystkim na podstawie redukcji ryzyka, mając również na uwadze nakład pracy potrzebny do wdrożenia.

Nie tylko kwestia ryzyka

Stosowanie nakazowych, opartych na ryzyku poziomów, które narzucają określony poziom pewnym aplikacjom, okazało się nadmiernie sztywne. W praktyce priorytetyzacja i wdrażanie mechanizmów bezpieczeństwa zależą od wielu czynników, obejmujących zarówno redukcję ryzyka, jak i nakład pracy wymagany do implementacji.

Dlatego zachęca się organizacje, aby dążyły do poziomu, który w ich ocenie powinny osiągnąć, biorąc pod uwagę własną dojrzałość oraz przekaz, jaki chcą kierować do swoich użytkowników.

V1 Kodowanie i sanityzacja

Cel kontrolny

Niniejszy rozdział dotyczy najczęstszych słabości bezpieczeństwa aplikacji internetowych związanych z niebezpiecznym przetwarzaniem niezaufanych danych. Słabości te mogą prowadzić do różnych podatności technicznych, w których niezaufane dane są interpretowane zgodnie z regułami składni właściwego interpretera.

W nowoczesnych aplikacjach internetowych najlepiej zawsze korzystać z bezpieczniejszych API, takich jak zapytania parametryzowane, automatyczne escapowanie czy frameworki szablonów. W przeciwnym razie starannie wykonane kodowanie danych wyjściowych, escapowanie lub sanityzacja stają się krytyczne dla bezpieczeństwa aplikacji.

Walidacja danych wejściowych pełni rolę mechanizmu obrony w głąb, chroniącego przed nieoczekiwaną lub niebezpieczną treścią. Ponieważ jednak jej głównym celem jest zapewnienie, że przychodząca treść odpowiada oczekiwaniom funkcjonalnym i biznesowym, powiązane z nią wymagania znajdują się w rozdziale „Walidacja i logika biznesowa”.

V1.1 Architektura kodowania i sanityzacji

W poniższych sekcjach przedstawiono wymagania —specyficzne dla danej składni lub interpretera — dotyczące bezpiecznego przetwarzania niebezpiecznej treści w celu uniknięcia podatności. Wymagania w tej sekcji obejmują kolejność, w jakiej przetwarzanie powinno następować, oraz miejsce, w którym powinno się odbywać. Mają one również zapewnić, że przechowywane dane pozostają w swojej pierwotnej postaci i nie są zapisywane w formie zakodowanej ani escapowanej (np. kodowanie HTML), aby zapobiec problemom podwójnego kodowania.

#	Opis	Poziom
1.1.1	Zweryfikuj, że dane wejściowe są dekodowane lub pozbawiane escapowania do postaci kanonicznej tylko raz, że dekodowanie następuje wyłącznie wtedy, gdy oczekiwane są dane zakodowane w tej formie, oraz że odbywa się to przed dalszym przetwarzaniem danych wejściowych —na przykład nie jest wykonywane po walidacji lub sanityzacji danych wejściowych.	2
1.1.2	Zweryfikuj, że aplikacja wykonuje kodowanie i escapowanie danych wyjściowych jako ostatni krok przed ich użyciem przez interpreter, dla którego są przeznaczone, albo że wykonuje je sam interpreter.	2

V1.2 Zapobieganie wstrzyknięciom

Kodowanie lub escapowanie danych wyjściowych, wykonywane blisko potencjalnie niebezpiecznego kontekstu lub bezpośrednio przy nim, jest krytyczne dla bezpieczeństwa każdej aplikacji. Zazwyczaj kodowanie i escapowanie danych wyjściowych nie jest utrwalane —służy do uczynienia danych wyjściowych bezpiecznymi do natychmiastowego użycia we właściwym interpreterze. Próba wykonania tego zbyt wcześnie może skutkować zniekształceniem treści lub uczynić kodowanie bądź escapowanie nieskutecznym.

W wielu przypadkach biblioteki oprogramowania zawierają bezpieczne lub bezpieczniejsze funkcje, które wykonują to automatycznie —należy jednak upewnić się, że są one właściwe dla bieżącego kontekstu.

#	Opis	Poziom
1.2.1	Zweryfikuj, że kodowanie danych wyjściowych dla odpowiedzi HTTP, dokumentu HTML lub dokumentu XML jest właściwe dla wymaganego kontekstu —na przykład kodowanie odpowiednich znaków dla elementów HTML, atrybutów HTML, komentarzy HTML, CSS lub pól nagłówków HTTP —aby uniknąć zmiany struktury komunikatu lub dokumentu.	1
1.2.2	Zweryfikuj, że przy dynamicznym budowaniu adresów URL niezaufane dane są kodowane zgodnie z ich kontekstem (np. kodowanie URL lub base64url dla parametrów zapytania lub ścieżki). Upewnij się, że dozwolone są wyłącznie bezpieczne protokoły URL (np. zablokowane są javascript: lub data:).	1
1.2.3	Zweryfikuj, że przy dynamicznym budowaniu treści JavaScript (w tym JSON) stosowane jest kodowanie lub escapowanie danych wyjściowych, aby uniknąć zmiany struktury komunikatu lub dokumentu (aby zapobiec wstrzyknięciom JavaScript i JSON).	1

#	Opis	Poziom
1.2.4	Zweryfikuj, że wybieranie danych i zapytania do baz danych (np. SQL, HQL, NoSQL, Cypher) używają zapytań parametryzowanych, ORM-ów, frameworków encji lub są w inny sposób chronione przed wstrzyknięciem SQL i innymi atakami wstrzyknięcia do baz danych. Dotyczy to również pisania procedur składowanych.	1
1.2.5	Zweryfikuj, że aplikacja chroni przed wstrzyknięciem poleceń systemu operacyjnego oraz że wywołania systemowe używają sparametryzowanych zapytań systemowych lub stosują kontekstowe kodowanie danych wyjściowych wiersza poleceń.	1
1.2.6	Zweryfikuj, że aplikacja chroni przed podatnościami wstrzyknięcia LDAP lub że wdrożono dedykowane mechanizmy bezpieczeństwa zapobiegające wstrzyknięciu LDAP.	2
1.2.7	Zweryfikuj, że aplikacja jest chroniona przed atakami wstrzyknięcia XPath poprzez parametryzację zapytań lub użycie zapytań prekompilowanych.	2
1.2.8	Zweryfikuj, że procesory LaTeX są skonfigurowane bezpiecznie (np. bez użycia flagi “-shell-escape”) oraz że stosowana jest lista dozwolonych poleceń, aby zapobiec atakom wstrzyknięcia LaTeX.	2
1.2.9	Zweryfikuj, że aplikacja escapuje znaki specjalne w wyrażeniach regularnych (zazwyczaj za pomocą ukośnika wstecznego), aby zapobiec ich błędnej interpretacji jako metaznaków.	2
1.2.10	Zweryfikuj, że aplikacja jest chroniona przed wstrzyknięciem CSV i formuł. Aplikacja musi przestrzegać reguł escapowania zdefiniowanych w RFC 4180, sekcje 2.6 i 2.7, podczas eksportu treści CSV. Dodatkowo przy eksporcie do CSV lub innych formatów arkuszy kalkulacyjnych (takich jak XLS, XLSX czy ODF) znaki specjalne (w tym ‘=’, ‘+’, ‘-’, ‘@’, ‘\t’(tabulator) oraz ‘\0’(znak null)) muszą być escapowane pojedynczym cudzysłowem, jeśli występują jako pierwszy znak wartości pola.	3

Uwaga: użycie zapytań parametryzowanych lub escapowanie SQL nie zawsze wystarcza. Części zapytania, takie jak nazwy tabel i kolumn (w tym nazwy kolumn w „ORDER BY”), nie mogą być escapowane. Umieszczenie escapowanych danych pochodzących od użytkownika w tych miejscach skutkuje błędnymi zapytaniami lub wstrzyknięciem SQL.

V1.3 Sanityzacja

Idealną ochroną przed użyciem niezaufanej treści w niebezpiecznym kontekście jest zastosowanie kodowania lub escapowania właściwego dla kontekstu, które zachowuje semantykę niebezpiecznej

treści, czyniąc ją jednocześnie bezpieczną do użycia w danym kontekście –co omówiono szerzej w poprzedniej sekcji.

Tam, gdzie nie jest to możliwe, konieczna staje się sanityzacja, czyli usuwanie potencjalnie niebezpiecznych znaków lub treści. W niektórych przypadkach może to zmienić semantykę danych wejściowych, jednak ze względów bezpieczeństwa może nie być alternatywy.

#	Opis	Poziom
1.3.1	Zweryfikuj, że wszystkie niezaufane dane wejściowe HTML pochodzące z edytorów WYSIWYG lub podobnych są sanityzowane przy użyciu znanej i bezpiecznej biblioteki sanityzacji HTML lub funkcji frameworka.	1
1.3.2	Zweryfikuj, że aplikacja unika użycia <code>eval()</code> oraz innych funkcji dynamicznego wykonywania kodu, takich jak Spring Expression Language (SpEL). Tam, gdzie nie ma alternatywy, wszelkie dołączane dane wejściowe użytkownika muszą zostać zsanityzowane przed wykonaniem.	1
1.3.3	Zweryfikuj, że dane przekazywane do potencjalnie niebezpiecznego kontekstu są wcześniej sanityzowane w celu wymuszenia środków bezpieczeństwa – na przykład dopuszczania wyłącznie znaków bezpiecznych dla tego kontekstu oraz przycinania zbyt długich danych wejściowych.	2
1.3.4	Zweryfikuj, że dostarczana przez użytkownika skryptowalna treść Scalable Vector Graphics (SVG) jest walidowana lub sanityzowana tak, aby zawierała wyłącznie znaczniki i atrybuty bezpieczne dla aplikacji (np. rysujące grafikę), a nie zawierała skryptów ani elementu <code>foreignObject</code> .	2
1.3.5	Zweryfikuj, że aplikacja sanityzuje lub wyłącza dostarczaną przez użytkownika treść skryptowalną lub treść języków szablonów wyrażen, taką jak Markdown, arkusze stylów CSS lub XSL, BBCode i podobne.	2
1.3.6	Zweryfikuj, że aplikacja chroni przed atakami Server-side Request Forgery (SSRF), walidując niezaufane dane względem listy dozwolonych protokołów, domen, ścieżek i portów oraz sanityzując potencjalnie niebezpieczne znaki przed użyciem danych do wywołania innej usługi.	2
1.3.7	Zweryfikuj, że aplikacja chroni przed atakami wstrzyknięcia szablonu, nie pozwalając na budowanie szablonów na podstawie niezaufanych danych wejściowych. Tam, gdzie nie ma alternatywy, wszelkie niezaufane dane wejściowe dołączane dynamicznie podczas tworzenia szablonu muszą zostać zsanityzowane lub ściśle zwalidowane.	2
1.3.8	Zweryfikuj, że aplikacja odpowiednio sanityzuje niezaufane dane wejściowe przed użyciem w zapytaniach Java Naming and Directory Interface (JNDI) oraz że JNDI jest skonfigurowane bezpiecznie, aby zapobiec atakom wstrzyknięcia JNDI.	2

#	Opis	Poziom
1.3.9	Zweryfikuj, że aplikacja sanityzuje treść przed wysłaniem jej do memcache, aby zapobiec atakom wstrzyknięcia.	2
1.3.10	Zweryfikuj, że ciągi formatujące, które mogą zostać rozwiązane w nieoczekiwany lub złośliwy sposób, są sanityzowane przed przetworzeniem.	2
1.3.11	Zweryfikuj, że aplikacja sanityzuje dane wejściowe użytkownika przed przekazaniem ich do systemów pocztowych, aby chronić przed wstrzyknięciem SMTP lub IMAP.	2
1.3.12	Zweryfikuj, że wyrażenia regularne są wolne od elementów powodujących wykładniczy backtracking, oraz upewnij się, że niezaufane dane wejściowe są sanityzowane w celu ograniczenia ataków ReDoS lub Runaway Regex.	3

V1.4 Pamięć, ciągi znaków i kod niezarządzany

Poniższe wymagania dotyczą ryzyk związanych z niebezpiecznym użyciem pamięci, które na ogół mają zastosowanie, gdy aplikacja korzysta z języka systemowego lub kodu niezarządzanego.

W niektórych przypadkach można to osiągnąć, ustawiając flagi kompilatora, które włączają zabezpieczenia i ostrzeżenia przed przepełnieniem bufora —w tym randomizację stosu i zapobieganie wykonywaniu danych —oraz przerywają build w razie wykrycia niebezpiecznych operacji na wskaźnikach, pamięci, ciągach formatujących, liczbach całkowitych lub ciągach znaków.

#	Opis	Poziom
1.4.1	Zweryfikuj, że aplikacja używa bezpiecznych pamięciowo ciągów znaków, bezpieczniejszego kopiowania pamięci i arytmetyki wskaźników, aby wykrywać przepełnienia stosu, bufora lub sterty albo im zapobiegać.	2
1.4.2	Zweryfikuj, że stosowane są techniki walidacji znaku, zakresu i danych wejściowych, aby zapobiegać przepełnieniom liczb całkowitych.	2
1.4.3	Zweryfikuj, że dynamicznie alokowana pamięć i zasoby są zwalniane, a referencje lub wskaźniki do zwolnionej pamięci są usuwane lub ustawiane na null, aby zapobiec wiszącym wskaźnikom i podatnościom typu use-after-free.	2

V1.5 Bezpieczna deserializacja

Konwersja danych z postaci przechowywanej lub przesyłanej do rzeczywistych obiektów aplikacji (deserializacja) historycznie bywała przyczyną różnych podatności wstrzyknięcia kodu. Proces ten

należy przeprowadzać ostrożnie i bezpiecznie, aby uniknąć tego typu problemów.

W szczególności niektóre metody deserializacji zostały wskazane w dokumentacji języków programowania lub frameworków jako niebezpieczne i nie da się ich uczynić bezpiecznymi dla niezaufanych danych. Dla każdego stosowanego mechanizmu należy przeprowadzić staranną analizę due diligence.

#	Opis	Poziom
1.5.1	Zweryfikuj, że aplikacja konfiguruje parsery XML z użyciem restrykcyjnej konfiguracji oraz że niebezpieczne funkcje, takie jak rozwiązywanie encji zewnętrznych, są wyłączone, aby zapobiec atakom XML eXternal Entity (XXE).	1
1.5.2	Zweryfikuj, że deserializacja niezaufanych danych wymusza bezpieczną obsługę danych wejściowych — na przykład poprzez listę dozwolonych typów obiektów lub ograniczenie typów obiektów definiowanych przez klienta — aby zapobiec atakom deserializacji. Mechanizmy deserializacji jawnie określone jako niebezpieczne nie mogą być używane z niezaufanymi danymi wejściowymi.	2
1.5.3	Zweryfikuj, że różne parsery używane w aplikacji dla tego samego typu danych (np. parsery JSON, parsery XML, parsery URL) parsują dane w spójny sposób i używają tego samego mechanizmu kodowania znaków, aby uniknąć problemów takich jak podatności JSON Interoperability lub wykorzystanie odmiennego parsowania URI bądź plików w atakach Remote File Inclusion (RFI) lub Server-side Request Forgery (SSRF).	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP LDAP Injection Prevention Cheat Sheet
- OWASP Cross Site Scripting Prevention Cheat Sheet
- OWASP DOM Based Cross Site Scripting Prevention Cheat Sheet
- OWASP XML External Entity Prevention Cheat Sheet
- OWASP Web Security Testing Guide: Client-Side Testing
- OWASP Java Encoding Project
- DOMPurify —biblioteka sanitizacji HTML po stronie klienta
- RFC 4180 —Common Format and MIME Type for Comma-Separated Values (CSV) Files

Więcej informacji, w szczególności o problemach deserializacji i parsowania:

- OWASP Deserialization Cheat Sheet

- An Exploration of JSON Interoperability Vulnerabilities
- Orange Tsai —A New Era of SSRF: Exploiting URL Parser In Trending Programming Languages

V2 Walidacja i logika biznesowa

Cel kontrolny

Celem tego rozdziału jest zapewnienie, że weryfikowana aplikacja spełnia następujące cele wysokopoziomowe:

- Dane wejściowe otrzymywane przez aplikację odpowiadają oczekiwaniom biznesowym lub funkcjonalnym.
- Przepływ logiki biznesowej jest sekwencyjny, przetwarzany w kolejności i nie może zostać ominięty.
- Logika biznesowa zawiera limity i mechanizmy pozwalające wykrywać zautomatyzowane ataki i im zapobiegać —takie jak ciągłe drobne przelewy środków czy dodawanie miliona znajomych pojedynczo.
- Przepływy logiki biznesowej o wysokiej wartości uwzględniają przypadki nadużyć i złośliwych aktorów oraz posiadają zabezpieczenia przed atakami podszywania się (spoofing), manipulacji (tampering), ujawnienia informacji oraz eskalacji uprawnień.

V2.1 Dokumentacja walidacji i logiki biznesowej

Dokumentacja walidacji i logiki biznesowej powinna jasno definiować limity logiki biznesowej, reguły walidacji oraz kontekstową spójność powiązanych danych, tak aby było jasne, co należy zaimplementować w aplikacji.

#	Opis	Poziom
2.1.1	Zweryfikuj, że dokumentacja aplikacji definiuje reguły walidacji danych wejściowych określające, jak sprawdzać poprawność danych względem oczekiwanej struktury. Mogą to być powszechne formaty danych, takie jak numery kart płatniczych, adresy e-mail, numery telefonów, lub wewnętrzny format danych.	1
2.1.2	Zweryfikuj, że dokumentacja aplikacji definiuje sposób walidacji logicznej i kontekstowej spójności powiązanych danych —na przykład sprawdzenie, czy dzielnica i kod pocztowy do siebie pasują.	2
2.1.3	Zweryfikuj, że oczekiwania dotyczące limitów i walidacji logiki biznesowej są udokumentowane, zarówno dla poszczególnych użytkowników, jak i globalnie dla całej aplikacji.	2

V2.2 Walidacja danych wejściowych

Skuteczne mechanizmy walidacji danych wejściowych wymuszają oczekiwania biznesowe lub funkcjonalne wobec typu danych, które aplikacja spodziewa się otrzymać. Zapewnia to dobrą jakość danych i zmniejsza powierzchnię ataku. Nie eliminuje jednak ani nie zastępuje konieczności stosowania poprawnego kodowania, parametryzacji lub sanityzacji przy użyciu danych w innym komponencie lub przy prezentowaniu ich na wyjściu.

W tym kontekście „dane wejściowe” mogą pochodzić z bardzo różnych źródeł, w tym z pól formularzy HTML, żądań REST, parametrów URL, pól nagłówków HTTP, ciasteczek, plików na dysku, baz danych i zewnętrznych API.

Mechanizm logiki biznesowej może sprawdzać, czy dane wejściowe są liczbą mniejszą niż 100. Oczekiwanie funkcjonalne może sprawdzać, czy liczba znajduje się poniżej pewnego progu — jeśli liczba ta steruje tym, ile razy wykona się dana pętla, wysoka wartość mogłaby prowadzić do nadmiernego przetwarzania i potencjalnej odmowy usługi.

Choć walidacja schematem nie jest jawnie wymagana, może być najskuteczniejszym mechanizmem pełnego pokrycia walidacją API HTTP lub innych interfejsów używających JSON lub XML.

Należy zwrócić uwagę na następujące kwestie dotyczące walidacji schematem:

- „Opublikowana wersja” specyfikacji walidacji JSON Schema jest uznawana za gotową do użytku produkcyjnego, ale nie jest, ściśle rzecz biorąc, „stabilna”. Korzystając z walidacji JSON Schema, należy upewnić się, że nie ma rozbieżności ze wskazówkami zawartymi w poniższych wymaganiach.
- Używane biblioteki walidacji JSON Schema powinny być monitorowane i w razie potrzeby aktualizowane po sformalizowaniu standardu.
- Nie należy używać walidacji DTD, a przetwarzanie DTD we frameworkach powinno zostać wyłączone, aby uniknąć problemów z atakami XXE wymierzonymi w DTD.

#	Opis	Poziom
2.2.1	Zweryfikuj, że dane wejściowe są walidowane w celu wymuszenia oczekiwań biznesowych lub funkcjonalnych wobec tych danych. Walidacja powinna opierać się na podejściu pozytywnym — z użyciem listy dozwolonych wartości, wzorców i zakresów — albo na porównaniu danych wejściowych z oczekiwaną strukturą i limitami logicznymi według wcześniej zdefiniowanych reguł. Dla L1 można skupić się na danych wejściowych używanych do podejmowania konkretnych decyzji biznesowych lub dotyczących bezpieczeństwa. Dla L2 i wyżej powinno to obejmować wszystkie dane wejściowe.	1

#	Opis	Poziom
2.2.2	Zweryfikuj, że aplikacja została zaprojektowana tak, aby wymuszać walidację danych wejściowych na poziomie zaufanej warstwy usługowej. Walidacja po stronie klienta poprawia użyteczność i należy do niej zachęcać, ale nie wolno na niej polegać jako na mechanizmie bezpieczeństwa.	1
2.2.3	Zweryfikuj, że aplikacja zapewnia, iż kombinacje powiązanych danych są sensowne według wcześniej zdefiniowanych reguł.	2

V2.3 Bezpieczeństwo logiki biznesowej

Ta sekcja obejmuje kluczowe wymagania zapewniające, że aplikacja egzekwuje procesy logiki biznesowej we właściwy sposób i nie jest podatna na ataki wykorzystujące logikę i przepływ aplikacji.

#	Opis	Poziom
2.3.1	Zweryfikuj, że aplikacja przetwarza przepływy logiki biznesowej dla tego samego użytkownika wyłącznie w oczekiwanej, sekwencyjnej kolejności kroków, bez ich pomijania.	1
2.3.2	Zweryfikuj, że limity logiki biznesowej są zaimplementowane zgodnie z dokumentacją aplikacji, aby zapobiec wykorzystaniu błędów logiki biznesowej.	2
2.3.3	Zweryfikuj, że na poziomie logiki biznesowej stosowane są transakcje —tak aby operacja logiki biznesowej albo powiodła się w całości, albo została wycofana do poprzedniego poprawnego stanu.	2
2.3.4	Zweryfikuj, że na poziomie logiki biznesowej stosowane są mechanizmy blokad zapewniające, że zasoby o ograniczonej liczbie (takie jak miejsca w teatrze czy okna dostawy) nie mogą zostać zarezerwowane podwójnie poprzez manipulację logiką aplikacji.	2
2.3.5	Zweryfikuj, że przepływy logiki biznesowej o wysokiej wartości wymagają zatwierdzenia przez wielu użytkowników, aby zapobiec nieautoryzowanym lub przypadkowym działaniom. Może to obejmować między innymi duże przelewy pieniężne, zatwierdzanie umów, dostęp do informacji niejawnych lub nadpisywanie zabezpieczeń w procesach produkcyjnych.	3

V2.4 Ochrona przed automatyzacją

Ta sekcja obejmuje mechanizmy ochrony przed automatyzacją, zapewniające wymuszanie interakcji o charakterze ludzkim i zapobieganie nadmiernym żądaniom zautomatyzowanym.

#	Opis	Poziom
2.4.1	Zweryfikuj, że wdrożono mechanizmy ochrony przed automatyzacją, chroniące przed nadmiernymi wywołaniami funkcji aplikacji, które mogłyby prowadzić do eksfiltracji danych, tworzenia danych-śmieci, wyczerpania przydziałów, przekroczenia limitów częstotliwości, odmowy usługi lub nadużywania kosztownych zasobów.	2
2.4.2	Zweryfikuj, że przepływy logiki biznesowej wymagają realistycznego, ludzkiego tempa działania, uniemożliwiając nadmiernie szybkie przesyłanie transakcji.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing
- Ochronę przed automatyzacją można osiągnąć na wiele sposobów, w tym z wykorzystaniem projektu OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 Bezpieczeństwo frontendu webowego

Cel kontrolny

Ta kategoria koncentruje się na wymaganiach chroniących przed atakami przeprowadzanymi za pośrednictwem frontendu webowego. Wymagania te nie mają zastosowania do rozwiązań działających w komunikacji maszyna-maszyna.

V3.1 Dokumentacja bezpieczeństwa frontendu webowego

Ta sekcja wskazuje funkcje bezpieczeństwa przeglądarki, które powinny zostać określone w dokumentacji aplikacji.

#	Opis	Poziom
3.1.1	Zweryfikuj, że dokumentacja aplikacji określa oczekiwane funkcje bezpieczeństwa, które muszą wspierać przeglądarki korzystające z aplikacji (takie jak HTTPS, HTTP Strict Transport Security (HSTS), Content Security Policy (CSP) oraz inne istotne mechanizmy bezpieczeństwa HTTP). Dokumentacja musi również definiować, jak aplikacja ma się zachować, gdy część tych funkcji jest niedostępna (na przykład ostrzec użytkownika lub zablokować dostęp).	3

V3.2 Niezamierzona interpretacja treści

Wyrenderowanie treści lub funkcjonalności w niewłaściwym kontekście może skutkować wykonaniem lub wyświetleniem złośliwej treści.

#	Opis	Poziom
3.2.1	Zweryfikuj, że wdrożono mechanizmy bezpieczeństwa zapobiegające renderowaniu przez przeglądarki treści lub funkcjonalności z odpowiedzi HTTP w niewłaściwym kontekście (np. gdy API, plik przesłany przez użytkownika lub inny zasób jest żądany bezpośrednio). Możliwe mechanizmy obejmują: nieserwowanie treści, jeśli pola nagłówka żądania HTTP (takie jak Sec-Fetch-*) nie wskazują właściwego kontekstu, użycie dyrektywy sandbox pola nagłówka Content-Security-Policy lub użycie typu dyspozycji attachment w polu nagłówka Content-Disposition.	1
3.2.2	Zweryfikuj, że treść przeznaczona do wyświetlenia jako tekst —a nie do wyrenderowania jako HTML —jest obsługiwana za pomocą bezpiecznych funkcji renderowania (takich jak <code>createTextNode</code> lub <code>textContent</code>), aby zapobiec niezamierzonemu wykonaniu treści takiej jak HTML lub JavaScript.	1
3.2.3	Zweryfikuj, że aplikacja unika DOM clobbering przy korzystaniu z JavaScript po stronie klienta poprzez jawne deklaracje zmiennych, ścisłą kontrolę typów, unikanie przechowywania zmiennych globalnych w obiekcie <code>document</code> oraz izolację przestrzeni nazw.	3

V3.3 Konfiguracja ciasteczek

Ta sekcja określa wymagania dotyczące bezpiecznej konfiguracji wrażliwych ciasteczek, aby zapewnić wyższy poziom pewności, że zostały utworzone przez samą aplikację, oraz zapobiec wyciekowi ich zawartości lub jej niewłaściwej modyfikacji.

#	Opis	Poziom
3.3.1	Zweryfikuj, że ciasteczka mają ustawiony atrybut 'Secure', a jeśli nazwa ciasteczka nie używa przedrostka '__Host-', musi używać przedrostka '__Secure-'.	1
3.3.2	Zweryfikuj, że wartość atrybutu 'SameSite' każdego ciasteczka jest ustawiona stosownie do przeznaczenia ciasteczka, aby ograniczyć narażenie na ataki podmioty interfejsu użytkownika oraz przeglądarkowe ataki fałszowania żądań, powszechnie znane jako cross-site request forgery (CSRF).	2
3.3.3	Zweryfikuj, że nazwy ciasteczek mają przedrostek '__Host-', chyba że ciasteczka są jawnie zaprojektowane do współdzielenia z innymi hostami.	2
3.3.4	Zweryfikuj, że jeśli wartość ciasteczka nie ma być dostępna dla skryptów po stronie klienta (jak token sesji), ciasteczko musi mieć ustawiony atrybut 'HttpOnly', a ta sama wartość (np. token sesji) musi być przekazywana klientowi wyłącznie poprzez pole nagłówka 'Set-Cookie'.	2
3.3.5	Zweryfikuj, że gdy aplikacja zapisuje ciasteczko, łączna długość nazwy i wartości ciasteczka nie przekracza 4096 bajtów. Zbyt duże ciasteczka nie zostaną zapisane przez przeglądarkę, a więc nie będą wysyłane z żądaniami, co uniemożliwi użytkownikowi korzystanie z funkcjonalności aplikacji zależnej od tego ciasteczka.	3

V3.4 Nagłówki mechanizmów bezpieczeństwa przeglądarki

Ta sekcja opisuje, które nagłówki bezpieczeństwa powinny być ustawiane w odpowiedziach HTTP, aby włączyć funkcje i ograniczenia bezpieczeństwa przeglądarki podczas obsługi odpowiedzi z aplikacji.

#	Opis	Poziom
3.4.1	Zweryfikuj, że pole nagłówka Strict-Transport-Security jest dołączane do wszystkich odpowiedzi w celu wymuszenia polityki HTTP Strict Transport Security (HSTS). Maksymalny wiek (max-age) musi wynosić co najmniej 1 rok, a dla L2 i wyżej polityka musi obejmować również wszystkie subdomeny.	1
3.4.2	Zweryfikuj, że pole nagłówka Cross-Origin Resource Sharing (CORS) Access-Control-Allow-Origin ma wartość stałą, ustaloną przez aplikację — a jeśli używana jest wartość pola nagłówka żądania HTTP Origin, jest ona walidowana względem listy dozwolonych zaufanych źródeł (origins). Gdy konieczne jest użycie 'Access-Control-Allow-Origin: *', zweryfikuj, że odpowiedź nie zawiera żadnych informacji wrażliwych.	1

#	Opis	Poziom
3.4.3	Zweryfikuj, że odpowiedzi HTTP zawierają pole nagłówka odpowiedzi Content-Security-Policy definiujące dyrektywy zapewniające, że przeglądarka ładuje i wykonuje wyłącznie zaufaną treść lub zasoby, aby ograniczyć wykonanie złośliwego JavaScriptu. Jako minimum musi być stosowana polityka globalna zawierająca dyrektywy object-src 'none' i base-uri 'none' oraz definiująca listę dozwolonych albo używająca wartości nonce lub skrótów (hash). Dla aplikacji L3 musi być zdefiniowana polityka dla poszczególnych odpowiedzi z wartościami nonce lub skrótami.	2
3.4.4	Zweryfikuj, że wszystkie odpowiedzi HTTP zawierają pole nagłówka 'X-Content-Type-Options: nosniff'. Nakazuje ono przeglądarkom rezygnację ze sniffingu treści i zgadywania typu MIME dla danej odpowiedzi oraz wymaganie, aby wartość pola nagłówka Content-Type odpowiedzi odpowiadała docelowemu zasobowi. Na przykład odpowiedź na żądanie stylu jest akceptowana tylko wtedy, gdy Content-Type odpowiedzi to 'text/css'. Włącza to również funkcjonalność Cross-Origin Read Blocking (CORB) w przeglądarce.	2
3.4.5	Zweryfikuj, że aplikacja ustawia politykę referrer, aby zapobiec wyciekowi technicznie wrażliwych danych do usług stron trzecich poprzez pole nagłówka żądania HTTP 'Referer'. Można to zrobić za pomocą pola nagłówka odpowiedzi HTTP Referrer-Policy lub poprzez atrybuty elementów HTML. Dane wrażliwe mogą obejmować ścieżkę i parametry zapytania w URL, a dla wewnętrznych, niepublicznych aplikacji — również nazwę hosta.	2
3.4.6	Zweryfikuj, że aplikacja internetowa używa dyrektywy frame-ancestors pola nagłówka Content-Security-Policy w każdej odpowiedzi HTTP, aby zapewnić, że domyślnie nie może być osadzana, a osadzanie konkretnych zasobów jest dozwolone tylko wtedy, gdy to konieczne. Zwróć uwagę, że pole nagłówka X-Frame-Options, choć wspierane przez przeglądarki, jest przestarzałe i nie należy na nim polegać.	2
3.4.7	Zweryfikuj, że pole nagłówka Content-Security-Policy wskazuje lokalizację raportowania naruszeń.	3
3.4.8	Zweryfikuj, że wszystkie odpowiedzi HTTP inicjujące renderowanie dokumentu (takie jak odpowiedzi z Content-Type text/html) zawierają pole nagłówka Cross-Origin-Opener-Policy z dyrektywą same-origin lub — w razie potrzeby — same-origin-allow-popups. Zapobiega to atakom nadużywającym współdzielonego dostępu do obiektów Window, takim jak tabnabbing czy zliczanie ramek (frame counting).	3

V3.5 Separacja źródeł w przeglądarce

Przyjmując po stronie serwera żądanie dotyczące wrażliwej funkcjonalności, aplikacja musi upewnić się, że żądanie zostało zainicjowane przez samą aplikację lub zaufaną stronę i nie zostało sfalszowane przez atakującego.

Wrażliwa funkcjonalność w tym kontekście może obejmować przyjmowanie formularzy od użytkowników uwierzytelnionych i niewierzytelnionych (np. żądanie uwierzytelnienia), operacje zmieniające stan lub funkcjonalność wymagającą znacznych zasobów (np. eksport danych).

Kluczowe zabezpieczenia to polityki bezpieczeństwa przeglądarki, takie jak Same Origin Policy dla JavaScriptu oraz logika SameSite dla ciasteczek. Innym powszechnym zabezpieczeniem jest mechanizm preflight CORS. Mechanizm ten jest krytyczny dla endpointów zaprojektowanych do wywoływania z innego źródła, ale może też być użytecznym mechanizmem zapobiegania fałszowaniu żądań dla endpointów, które nie są przeznaczone do wywoływania z innego źródła.

#	Opis	Poziom
3.5.1	Zweryfikuj, że jeśli aplikacja nie opiera się na mechanizmie preflight CORS w celu zapobiegania niedozwolonym żądaniom cross-origin do wrażliwej funkcjonalności, żądania te są walidowane w celu potwierdzenia, że pochodzą z samej aplikacji. Można to zrobić poprzez stosowanie i walidację tokenów anti-forgery lub wymaganie dodatkowych pól nagłówka HTTP, które nie znajdują się na liście CORS-safelisted request-header fields. Ma to chronić przed przeglądarkowymi atakami fałszowania żądań, powszechnie znanymi jako cross-site request forgery (CSRF).	1
3.5.2	Zweryfikuj, że jeśli aplikacja opiera się na mechanizmie preflight CORS w celu zapobiegania niedozwolonemu użyciu wrażliwej funkcjonalności cross-origin, nie jest możliwe wywołanie tej funkcjonalności żądaniem, które nie wyzwala żądania preflight CORS. Może to wymagać sprawdzania wartości pól nagłówka żądania 'Origin' i 'Content-Type' lub użycia dodatkowego pola nagłówka spoza listy CORS-safelisted header-fields.	1
3.5.3	Zweryfikuj, że żądania HTTP do wrażliwej funkcjonalności używają odpowiednich metod HTTP, takich jak POST, PUT, PATCH lub DELETE, a nie metod zdefiniowanych w specyfikacji HTTP jako „bezpieczne”, takich jak HEAD, OPTIONS czy GET. Alternatywnie można zastosować ścisłą walidację pól nagłówka żądania Sec-Fetch-*, aby upewnić się, że żądanie nie pochodzi z niewłaściwego wywołania cross-origin, żądania nawigacji ani ładowania zasobu (np. źródła obrazu) tam, gdzie nie jest to oczekiwane.	1

#	Opis	Poziom
3.5.4	Zweryfikuj, że odrębne aplikacje są hostowane na różnych nazwach hostów, aby wykorzystać ograniczenia zapewniane przez politykę same-origin – w tym zasady interakcji dokumentów lub skryptów załadowanych przez jedno źródło z zasobami innego źródła oraz oparte na nazwie hosta ograniczenia dotyczące ciasteczek.	2
3.5.5	Zweryfikuj, że wiadomości odbierane przez interfejs postMessage są odrzucane, jeśli źródło wiadomości nie jest zaufane lub jej składnia jest nieprawidłowa.	2
3.5.6	Zweryfikuj, że funkcjonalność JSONP nie jest włączona nigdzie w aplikacji, aby uniknąć ataków Cross-Site Script Inclusion (XSSI).	3
3.5.7	Zweryfikuj, że dane wymagające autoryzacji nie są umieszczane w odpowiedziach zasobów skryptowych, takich jak pliki JavaScript, aby zapobiec atakom Cross-Site Script Inclusion (XSSI).	3
3.5.8	Zweryfikuj, że uwierzytelnione zasoby (takie jak obrazy, wideo, skrypty i inne dokumenty) mogą być ładowane lub osadzone w imieniu użytkownika tylko wtedy, gdy jest to zamierzone. Można to osiągnąć poprzez ścisłą walidację pól nagłówka żądania HTTP Sec-Fetch-*, aby upewnić się, że żądanie nie pochodzi z niewłaściwego wywołania cross-origin, lub poprzez ustawienie restrykcyjnego pola nagłówka odpowiedzi HTTP Cross-Origin-Resource-Policy, nakazującego przeglądarce zablokowanie zwróconej treści.	3

V3.6 Integralność zasobów zewnętrznych

Ta sekcja zawiera wytyczne dotyczące bezpiecznego hostowania treści w serwisach stron trzecich.

#	Opis	Poziom
3.6.1	Zweryfikuj, że zasoby po stronie klienta – takie jak biblioteki JavaScript, CSS czy fonty webowe – są hostowane zewnętrznie (np. w Content Delivery Network) tylko wtedy, gdy zasób jest statyczny i wersjonowany, a do walidacji jego integralności używana jest Subresource Integrity (SRI). Jeśli nie jest to możliwe, dla każdego zasobu powinna istnieć udokumentowana decyzja bezpieczeństwa to uzasadniająca.	3

V3.7 Pozostałe kwestie bezpieczeństwa przeglądarki

Ta sekcja obejmuje różne inne mechanizmy bezpieczeństwa oraz nowoczesne funkcje bezpieczeństwa przeglądarek wymagane dla bezpieczeństwa po stronie klienta.

#	Opis	Poziom
3.7.1	Zweryfikuj, że aplikacja korzysta wyłącznie z technologii po stronie klienta, które są nadal wspierane i uznawane za bezpieczne. Przykłady technologii niespełniających tego wymagania to wtyczki NSAPI, Flash, Shockwave, ActiveX, Silverlight, NACL oraz aplety Javy po stronie klienta.	2
3.7.2	Zweryfikuj, że aplikacja automatycznie przekierowuje użytkownika na inną nazwę hosta lub domenę (niekontrolowaną przez aplikację) tylko wtedy, gdy cel przekierowania znajduje się na liście dozwolonych.	2
3.7.3	Zweryfikuj, że aplikacja wyświetla powiadomienie, gdy użytkownik jest przekierowywany na adres URL poza kontrolą aplikacji, z możliwością anulowania nawigacji.	3
3.7.4	Zweryfikuj, że domena najwyższego poziomu aplikacji (np. site.tld) została dodana do publicznej listy preload dla HTTP Strict Transport Security (HSTS). Zapewnia to, że użycie TLS dla aplikacji jest wbudowane bezpośrednio w główne przeglądarki, a nie zależy wyłącznie od pola nagłówka odpowiedzi Strict-Transport-Security.	3
3.7.5	Zweryfikuj, że aplikacja zachowuje się zgodnie z dokumentacją (np. ostrzega użytkownika lub blokuje dostęp), jeśli przeglądarka użyta do korzystania z aplikacji nie wspiera oczekiwanych funkcji bezpieczeństwa.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- Set-Cookie __Host- prefix details
- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API i usługi sieciowe

Cel kontrolny

Szereg kwestii dotyczy w szczególności aplikacji udostępniających API do użytku przez przeglądarki internetowe lub innych konsumentów (zwykle z użyciem JSON, XML lub GraphQL). Niniejszy rozdział obejmuje odpowiednie konfiguracje i mechanizmy bezpieczeństwa, które należy zastosować.

Należy pamiętać, że kwestie uwierzytelniania, zarządzania sesją i walidacji danych wejściowych z pozostałych rozdziałów również dotyczą API – tego rozdziału nie można więc wyrywać z kontekstu ani testować w izolacji.

V4.1 Ogólne bezpieczeństwo usług sieciowych

Ta sekcja dotyczy ogólnych kwestii bezpieczeństwa usług sieciowych, a co za tym idzie – podstawowych praktyk higieny usług sieciowych.

#	Opis	Poziom
4.1.1	Zweryfikuj, że każda odpowiedź HTTP z treścią komunikatu zawiera pole nagłówek Content-Type odpowiadające rzeczywistej zawartości odpowiedzi, wraz z parametrem charset określającym bezpieczne kodowanie znaków (np. UTF-8, ISO-8859-1) zgodnie z IANA Media Types, np. dla "text/" i "/+xml" i "/xml".	1
4.1.2	Zweryfikuj, że wyłącznie endpointy skierowane do użytkownika (przeznaczone do ręcznego dostępu przez przeglądarkę) automatycznie przekierowują z HTTP na HTTPS, natomiast pozostałe usługi i endpointy nie stosują przezroczystych przekierowań. Ma to zapobiec sytuacji, w której klient błędnie wysyła nieszyfrowane żądania HTTP, lecz – ponieważ żądania są automatycznie przekierowywane na HTTPS – wyciek danych wrażliwych pozostaje niewykryty.	2
4.1.3	Zweryfikuj, że żadne pole nagłówka HTTP używane przez aplikację i ustawiane przez warstwę pośredniczącą – taką jak moduł równoważenia obciążenia, proxy webowe lub usługa backend-for-frontend – nie może zostać nadpisane przez użytkownika końcowego. Przykładowe nagłówki to X-Real-IP, X-Forwarded-* lub X-User-ID.	2
4.1.4	Zweryfikuj, że mogą być używane wyłącznie metody HTTP jawnie wspierane przez aplikację lub jej API (w tym OPTIONS podczas żądań preflight), a metody nieużywane są blokowane.	3

#	Opis	Poziom
4.1.5	Zweryfikuj, że dla żądań lub transakcji wysoce wrażliwych albo przechodzących przez wiele systemów stosowane są podpisy cyfrowe dla poszczególnych komunikatów, zapewniające dodatkową pewność ponad zabezpieczenia warstwy transportowej.	3

V4.2 Walidacja struktury komunikatów HTTP

Ta sekcja wyjaśnia, jak należy walidować strukturę i pola nagłówków komunikatu HTTP, aby zapobiegać atakom takim jak request smuggling, response splitting, wstrzyknięcie nagłówków oraz odmowa usługi poprzez nadmiernie długie komunikaty HTTP.

Wymagania te dotyczą ogólnego przetwarzania i generowania komunikatów HTTP, ale są szczególnie istotne przy konwersji komunikatów HTTP między różnymi wersjami protokołu.

#	Opis	Poziom
4.2.1	Zweryfikuj, że wszystkie komponenty aplikacji (w tym moduły równoważenia obciążenia, zapory i serwery aplikacyjne) wyznaczają granice przychodzących komunikatów HTTP przy użyciu mechanizmu właściwego dla danej wersji HTTP, aby zapobiec atakom HTTP request smuggling. W HTTP/1.x, jeśli obecne jest pole nagłówka Transfer-Encoding, nagłówek Content-Length musi zostać zignorowany zgodnie z RFC 2616. W HTTP/2 i HTTP/3, jeśli obecne jest pole nagłówka Content-Length, odbiorca musi upewnić się, że jest ono spójne z długością ramek DATA.	2
4.2.2	Zweryfikuj, że przy generowaniu komunikatów HTTP pole nagłówka Content-Length nie stoi w sprzeczności z długością treści wynikającą z ramkowania protokołu HTTP, aby zapobiec atakom request smuggling.	3
4.2.3	Zweryfikuj, że aplikacja nie wysyła ani nie akceptuje komunikatów HTTP/2 lub HTTP/3 z polami nagłówka specyficznymi dla połączenia, takimi jak Transfer-Encoding, aby zapobiec atakom response splitting i wstrzyknięcia nagłówków.	3
4.2.4	Zweryfikuj, że aplikacja akceptuje wyłącznie żądania HTTP/2 i HTTP/3, w których pola nagłówków i ich wartości nie zawierają sekwencji CR (\r), LF (\n) ani CRLF (\r\n), aby zapobiec atakom wstrzyknięcia nagłówków.	3

#	Opis	Poziom
4.2.5	Zweryfikuj, że jeśli aplikacja (backend lub frontend) buduje i wysyła żądania, stosuje walidację, sanityzację lub inne mechanizmy zapobiegające tworzeniu URI (np. dla wywołań API) lub pól nagłówka żądania HTTP (np. Authorization lub Cookie), które są zbyt długie, aby zostały zaakceptowane przez komponent odbierający. Mogłoby to spowodować odmowę usługi – na przykład wysłanie nadmiernie długiego żądania (np. długiego pola nagłówka cookie) skutkujące tym, że serwer zawsze odpowiada statusem błędu.	3

V4.3 GraphQL

GraphQL staje się coraz popularniejszym sposobem tworzenia bogatych w dane klientów, które nie są ściśle powiązane z różnorodnymi usługami backendowymi. Ta sekcja obejmuje kwestie bezpieczeństwa GraphQL.

#	Opis	Poziom
4.3.1	Zweryfikuj, że stosowana jest lista dozwolonych zapytań, ograniczanie głębokości, ograniczanie liczby lub analiza kosztu zapytań, aby zapobiec odmowie usługi (DoS) na poziomie GraphQL lub wyrażen warstwy danych w wyniku kosztownych, zagnieżdżonych zapytań.	2
4.3.2	Zweryfikuj, że zapytania introspekcyjne GraphQL są wyłączone w środowisku produkcyjnym, chyba że API GraphQL jest przeznaczone do użytku przez strony trzecie.	2

V4.4 WebSocket

WebSocket to protokół komunikacyjny zapewniający jednoczesny, dwukierunkowy kanał komunikacji przez pojedyncze połączenie TCP. Został ustandaryzowany przez IETF jako RFC 6455 w 2011 roku i jest odrębny od HTTP, mimo że został zaprojektowany do działania na portach HTTP 443 i 80.

Ta sekcja zawiera kluczowe wymagania bezpieczeństwa zapobiegające atakom związanym z bezpieczeństwem komunikacji i zarządzaniem sesją, które wykorzystują specyfikę tego kanału komunikacji w czasie rzeczywistym.

#	Opis	Poziom
4.4.1	Zweryfikuj, że dla wszystkich połączeń WebSocket używany jest WebSocket over TLS (WSS).	1
4.4.2	Zweryfikuj, że podczas początkowego uzgadniania HTTP WebSocket pole nagłówka Origin jest sprawdzane względem listy źródeł dozwolonych dla aplikacji.	2
4.4.3	Zweryfikuj, że jeśli nie można użyć standardowego zarządzania sesją aplikacji, stosowane są w tym celu dedykowane tokeny zgodne z odpowiednimi wymaganiami bezpieczeństwa zarządzania sesją.	2
4.4.4	Zweryfikuj, że dedykowane tokeny zarządzania sesją WebSocket są początkowo uzyskiwane lub walidowane poprzez wcześniej uwierzytelnioną sesję HTTPS podczas przechodzenia z istniejącej sesji HTTPS na kanał WebSocket.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP REST Security Cheat Sheet
- Materiały o autoryzacji w GraphQL z graphql.org oraz Apollo.
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 Obsługa plików

Cel kontrolny

Korzystanie z plików może stwarzać dla aplikacji różnorodne ryzyka, w tym odmowę usługi, nieautoryzowany dostęp i wyczerpanie przestrzeni dyskowej. Niniejszy rozdział zawiera wymagania odnoszące się do tych ryzyk.

V5.1 Dokumentacja obsługi plików

Ta sekcja zawiera wymaganie udokumentowania oczekiwanych cech plików przyjmowanych przez aplikację –jako niezbędnego warunku wstępnego dla opracowania i weryfikacji odpowiednich kontroli bezpieczeństwa.

#	Opis	Poziom
5.1.1	Zweryfikuj, że dokumentacja definiuje dozwolone typy plików, oczekiwane rozszerzenia plików oraz maksymalny rozmiar (w tym rozmiar po rozpakowaniu) dla każdej funkcji przesyłania plików. Dodatkowo upewnij się, że dokumentacja określa, w jaki sposób pliki są czynione bezpiecznymi do pobierania i przetwarzania przez użytkowników końcowych — na przykład jak aplikacja zachowuje się po wykryciu złośliwego pliku.	2

V5.2 Przesyłanie plików i ich zawartość

Funkcjonalność przesyłania plików to główne źródło niezaufanych plików. Ta sekcja określa wymagania zapewniające, że obecność, liczba lub zawartość tych plików nie może zaszkodzić aplikacji.

#	Opis	Poziom
5.2.1	Zweryfikuj, że aplikacja przyjmuje wyłącznie pliki o rozmiarze, który jest w stanie przetworzyć bez utraty wydajności lub ataku odmowy usługi.	1
5.2.2	Zweryfikuj, że gdy aplikacja przyjmuje plik — samodzielnie lub w archiwum takim jak plik zip — sprawdza, czy rozszerzenie pliku odpowiada oczekiwanemu rozszerzeniu, oraz waliduje, czy zawartość odpowiada typowi reprezentowanemu przez rozszerzenie. Obejmuje to między innymi sprawdzanie początkowych „magicznych bajtów”, ponowne przetwarzanie obrazów (image re-writing) oraz użycie wyspecjalizowanych bibliotek do walidacji zawartości plików. Dla L1 można skupić się wyłącznie na plikach używanych do podejmowania konkretnych decyzji biznesowych lub dotyczących bezpieczeństwa. Dla L2 i wyżej musi to dotyczyć wszystkich przyjmowanych plików.	1
5.2.3	Zweryfikuj, że aplikacja sprawdza pliki skompresowane (np. zip, gz, docx, odt) względem maksymalnego dozwolonego rozmiaru po dekompresji oraz maksymalnej liczby plików przed rozpakowaniem pliku.	2
5.2.4	Zweryfikuj, że egzekwowany jest przydział rozmiaru plików oraz maksymalna liczba plików na użytkownika, aby pojedynczy użytkownik nie mógł zapełnić przestrzeni dyskowej zbyt wieloma plikami lub plikami nadmiernie dużymi.	3
5.2.5	Zweryfikuj, że aplikacja nie pozwala na przesyłanie plików skompresowanych zawierających dowiązania symboliczne, chyba że jest to wyraźnie wymagane (w takim przypadku konieczne będzie wymuszenie listy dozwolonych plików, do których dowiązania mogą prowadzić).	3

#	Opis	Poziom
5.2.6	Zweryfikuj, że aplikacja odrzuca przesyłane obrazy o rozmiarze w pikselach większym niż maksymalny dozwolony, aby zapobiec atakom pixel flood.	3

V5.3 Przechowywanie plików

Ta sekcja zawiera wymagania zapobiegające niewłaściwemu wykonywaniu plików po przesłaniu, służące wykrywaniu niebezpiecznej zawartości oraz uniemożliwiające wykorzystanie niezaufanych danych do kontrolowania miejsca przechowywania plików.

#	Opis	Poziom
5.3.1	Zweryfikuj, że pliki przesłane lub wygenerowane na podstawie niezaufanych danych wejściowych i przechowywane w folderze publicznym nie są wykonywane jako kod programu po stronie serwera przy bezpośrednim dostępie żądaniem HTTP.	1
5.3.2	Zweryfikuj, że gdy aplikacja tworzy ścieżki plików dla operacji na plikach, używa danych generowanych wewnętrznie lub zaufanych zamiast nazw plików przesłanych przez użytkownika —a jeśli nazwy plików lub metadane plików od użytkownika muszą zostać użyte, stosowana jest ścisła walidacja i sanityzacja. Ma to chronić przed atakami path traversal, local i remote file inclusion (LFI, RFI) oraz server-side request forgery (SSRF).	1
5.3.3	Zweryfikuj, że przetwarzanie plików po stronie serwera, takie jak dekompresja, ignoruje informacje o ścieżce dostarczone przez użytkownika, aby zapobiec podatnościom takim jak zip slip.	3

V5.4 Pobieranie plików

Ta sekcja zawiera wymagania ograniczające ryzyka przy serwowaniu plików do pobrania, w tym ataki path traversal i wstrzyknięcia. Obejmuje również zapewnienie, że pliki nie zawierają niebezpiecznej zawartości.

#	Opis	Poziom
5.4.1	Zweryfikuj, że aplikacja waliduje lub ignoruje nazwy plików przesłane przez użytkownika —w tym w parametrze JSON, JSONP lub URL —oraz określa nazwę pliku w polu nagłówka Content-Disposition w odpowiedzi.	2

#	Opis	Poziom
5.4.2	Zweryfikuj, że serwowane nazwy plików (np. w polach nagłówek odpowiedzi HTTP lub załącznikach e-mail) są kodowane lub sanityzowane (np. zgodnie z RFC 6266) w celu zachowania struktury dokumentu i zapobieżenia atakom wstrzyknięcia.	2
5.4.3	Zweryfikuj, że pliki pozyskane z niezaufanych źródeł są skanowane przez skanery antywirusowe, aby zapobiec serwowaniu znanej złośliwej zawartości.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP File Upload Cheat Sheet
- Przykład wykorzystania dowiązań symbolicznych do odczytu dowolnych plików
- Wyjaśnienie „magicznych bajtów” w Wikipedii

V6 Uwierzytelnianie

Cel kontrolny

Uwierzytelnianie to proces ustalania lub potwierdzania autentyczności osoby lub urządzenia. Obejmuje weryfikację deklaracji składanych przez osobę lub dotyczących urządzenia, zapewnienie odporności na podszywanie się oraz zapobieganie odzyskaniu lub przechwyceniu haseł.

NIST SP 800-63 to nowoczesny, oparty na dowodach standard, wartościowy dla organizacji na całym świecie, lecz szczególnie istotny dla agencji rządowych USA i podmiotów z nimi współpracujących.

Choć wiele wymagań tego rozdziału opiera się na drugiej części tego standardu (znanej jako NIST SP 800-63B „Digital Identity Guidelines - Authentication and Lifecycle Management”), rozdział koncentruje się na powszechnych zagrożeniach i często wykorzystywanych lukach w uwierzytelnianiu. Nie próbuje wyczerpująco pokryć każdego punktu standardu. W przypadkach, gdy konieczna jest pełna zgodność z NIST SP 800-63, należy sięgnąć do NIST SP 800-63.

Dodatkowo terminologia NIST SP 800-63 może się miejscami różnić — w tym rozdziale często stosowana jest terminologia powszechniej rozumiana, aby poprawić przejrzystość.

Częstą cechą bardziej zaawansowanych aplikacji jest zdolność dostosowywania wymaganych etapów uwierzytelniania w zależności od różnych czynników ryzyka. Funkcja ta została omówiona w rozdziale „Autoryzacja”, ponieważ mechanizmy te muszą być brane pod uwagę również przy decyzjach autoryzacyjnych.

V6.1 Dokumentacja uwierzytelniania

Ta sekcja zawiera wymagania określające dokumentację uwierzytelniania, która powinna być utrzymywana dla aplikacji. Jest to kluczowe dla wdrożenia i oceny właściwej konfiguracji odpowiednich mechanizmów uwierzytelniania.

#	Opis	Poziom
6.1.1	Zweryfikuj, że dokumentacja aplikacji definiuje, w jaki sposób mechanizmy takie jak ograniczanie częstotliwości żądań, ochrona przed automatyzacją i odpowiedź adaptacyjna są wykorzystywane do obrony przed atakami takimi jak credential stuffing i łamanie haseł metodą siłową. Dokumentacja musi jasno określać, jak mechanizmy te są skonfigurowane i jak zapobiegają złośliwemu blokowaniu kont.	1
6.1.2	Zweryfikuj, że udokumentowana jest lista słów specyficznych dla kontekstu, których użycie w hasłach ma być blokowane. Lista może obejmować permutacje nazw organizacji, nazw produktów, identyfikatorów systemów, kryptonimów projektów, nazw działów lub ról i podobne.	2
6.1.3	Zweryfikuj, że jeśli aplikacja udostępnia wiele ścieżek uwierzytelniania, wszystkie są udokumentowane wraz z mechanizmami bezpieczeństwa i siłą uwierzytelniania, które muszą być spójnie egzekwowane we wszystkich tych ścieżkach.	2

V6.2 Bezpieczeństwo haseł

Hasła —nazywane w NIST SP 800-63 „zapamiętanymi sekretami”(Memorized Secrets) —obejmują hasła, frazy hasłowe, kody PIN, wzory odblokowania oraz wskazywanie właściwego kotka lub innego elementu obrazkowego. Są na ogół uznawane za czynnik „coś, co wiesz”i często używane jako mechanizm uwierzytelniania jednoskładnikowego.

W związku z tym ta sekcja zawiera wymagania zapewniające, że hasła są tworzone i obsługiwane w sposób bezpieczny. Większość wymagań ma poziom L1, ponieważ na tym poziomie są najistotniejsze. Od L2 wzwyż wymagane są mechanizmy uwierzytelniania wieloskładnikowego, w których hasła mogą być jednym ze składników.

Wymagania tej sekcji odnoszą się głównie do § 5.1.1.2 wytycznych NIST.

#	Opis	Poziom
6.2.1	Zweryfikuj, że hasła ustawiane przez użytkowników mają co najmniej 8 znaków długości, przy czym zdecydowanie zaleca się minimum 15 znaków.	1

#	Opis	Poziom
6.2.2	Zweryfikuj, że użytkownicy mogą zmienić swoje hasło.	1
6.2.3	Zweryfikuj, że funkcja zmiany hasła wymaga podania obecnego i nowego hasła użytkownika.	1
6.2.4	Zweryfikuj, że hasła przesyłane podczas rejestracji konta lub zmiany hasła są sprawdzane względem dostępnego zbioru co najmniej 3000 najpopularniejszych haseł spełniających politykę haseł aplikacji, np. minimalną długość.	1
6.2.5	Zweryfikuj, że można używać haseł o dowolnej kompozycji, bez reguł ograniczających dozwolone typy znaków. Nie może istnieć wymóg minimalnej liczby wielkich lub małych liter, cyfr ani znaków specjalnych.	1
6.2.6	Zweryfikuj, że pola wprowadzania hasła używają type=password w celu maskowania wpisywanej treści. Aplikacje mogą pozwalać użytkownikowi na tymczasowe wyświetlenie całego zamaskowanego hasła lub ostatnio wpisanego znaku.	1
6.2.7	Zweryfikuj, że dozwolone są funkcja „wklej”, przeglądarkowe pomocniki haseł oraz zewnętrzne menedżery haseł.	1
6.2.8	Zweryfikuj, że aplikacja weryfikuje hasło użytkownika dokładnie w postaci otrzymanej od użytkownika, bez żadnych modyfikacji, takich jak obcinanie czy zmiana wielkości liter.	1
6.2.9	Zweryfikuj, że dozwolone są hasła o długości co najmniej 64 znaków.	2
6.2.10	Zweryfikuj, że hasło użytkownika pozostaje ważne, dopóki nie zostanie wykryta jego kompromitacja lub użytkownik sam go nie zmieni. Aplikacja nie może wymagać okresowej rotacji poświadczeń.	2
6.2.11	Zweryfikuj, że udokumentowana lista słów specyficznych dla kontekstu jest używana, aby zapobiegać tworzeniu łatwych do odgadnięcia haseł.	2
6.2.12	Zweryfikuj, że hasła przesyłane podczas rejestracji konta lub zmiany hasła są sprawdzane względem zbioru haseł ujawnionych w wyciekach.	2

V6.3 Ogólne bezpieczeństwo uwierzytelniania

Ta sekcja zawiera ogólne wymagania dotyczące bezpieczeństwa mechanizmów uwierzytelniania oraz określa różne oczekiwania dla poszczególnych poziomów. Aplikacje L2 muszą wymuszać stosowanie uwierzytelniania wieloskładnikowego (MFA). Aplikacje L3 muszą używać uwierzytelniania sprzętowego, realizowanego w atestowanym, zaufanym środowisku wykonawczym (TEE). Może to obejmować passkeys powiązane z urządzeniem, mechanizmy uwierzytelniające o wysokim

poziomie pewności eIDAS (LoA High), mechanizmy o poziomie pewności NIST Authenticator Assurance Level 3 (AAL3) lub równoważne rozwiązania.

Choć jest to stosunkowo agresywne stanowisko w sprawie MFA, podniesienie poprzeczki w tym obszarze jest krytyczne dla ochrony użytkowników, a każda próba złagodzenia tych wymagań powinna iść w parze z jasnym planem ograniczania ryzyk związanych z uwierzytelnianiem, uwzględniającym wytyczne i badania NIST w tym zakresie.

Warto zauważyć, że w chwili wydania NIST SP 800-63 uznaje e-mail za niedopuszczalny mechanizm uwierzytelniania (kopia archiwalna).

Wymagania tej sekcji odnoszą się do różnych części wytycznych NIST, w tym: § 4.2.1, § 4.3.1, § 5.2.2 oraz § 6.1.2.

#	Opis	Poziom
6.3.1	Zweryfikuj, że mechanizmy zapobiegające atakom takim jak credential stuffing i łamanie haseł metodą siłową są wdrożone zgodnie z dokumentacją bezpieczeństwa aplikacji.	1
6.3.2	Zweryfikuj, że domyślne konta użytkowników (np. „root”, „admin” lub „sa”) nie występują w aplikacji lub są wyłączone.	1
6.3.3	Zweryfikuj, że do uzyskania dostępu do aplikacji musi być używany mechanizm uwierzytelniania wieloskładnikowego albo kombinacja mechanizmów jednoskładnikowych. Dla L3 jednym ze składników musi być sprzętowy mechanizm uwierzytelniania, zapewniający odporność na kompromitację i podszywanie się w atakach phishingowych oraz weryfikujący zamiar uwierzytelnienia poprzez wymaganie działania zainicjowanego przez użytkownika (takiego jak naciśnięcie przycisku na kluczu sprzętowym FIDO lub telefonie komórkowym). Złagodzenie któregośkolwiek z warunków tego wymagania wymaga w pełni udokumentowanego uzasadnienia oraz kompleksowego zestawu mechanizmów ograniczających ryzyko.	2
6.3.4	Zweryfikuj, że jeśli aplikacja udostępnia wiele ścieżek uwierzytelniania, nie istnieją ścieżki nieudokumentowane, a mechanizmy bezpieczeństwa i siła uwierzytelniania są egzekwowane spójnie.	2
6.3.5	Zweryfikuj, że użytkownicy są powiadamiani o podejrzanych próbach uwierzytelnienia (udanych lub nieudanych). Może to obejmować próby uwierzytelnienia z nietypowej lokalizacji lub klienta, uwierzytelnienie częściowo udane (tylko jeden z wielu składników), próbę uwierzytelnienia po długim okresie bezczynności lub udane uwierzytelnienie po kilku nieudanych próbach.	3

#	Opis	Poziom
6.3.6	Zweryfikuj, że e-mail nie jest używany jako mechanizm uwierzytelniania – ani jednoskładnikowego, ani wieloskładnikowego.	3
6.3.7	Zweryfikuj, że użytkownicy są powiadamiani po zmianach danych uwierzytelniania, takich jak resety poświadczeń lub modyfikacja nazwy użytkownika bądź adresu e-mail.	3
6.3.8	Zweryfikuj, że na podstawie nieudanych prób uwierzytelnienia nie można wywnioskować istnienia prawidłowych użytkowników – na przykład na podstawie komunikatów o błędach, kodów odpowiedzi HTTP lub różnych czasów odpowiedzi. Ochronę tę musi mieć również funkcjonalność rejestracji i odzyskiwania zapomnianego hasła.	3

V6.4 Cykl życia i odzyskiwanie czynników uwierzytelniania

Czynniki uwierzytelniania mogą obejmować hasła, tokeny programowe, tokeny sprzętowe oraz urządzenia biometryczne. Bezpieczna obsługa cyklu życia tych mechanizmów jest krytyczna dla bezpieczeństwa aplikacji – ta sekcja zawiera wymagania z tym związane.

Wymagania tej sekcji odnoszą się głównie do § 5.1.1.2 lub § 6.1.2.3 wytycznych NIST.

#	Opis	Poziom
6.4.1	Zweryfikuj, że generowane przez system hasła początkowe lub kody aktywacyjne są generowane w sposób bezpiecznie losowy, zgodne z obowiązującą polityką haseł oraz wygasają po krótkim czasie lub po pierwszym użyciu. Te początkowe sekrety nie mogą stać się hasłem długoterminowym.	1
6.4.2	Zweryfikuj, że nie występują podpowiedzi do haseł ani uwierzytelnianie oparte na wiedzy (tzw. „pytania bezpieczeństwa”).	1
6.4.3	Zweryfikuj, że wdrożony jest bezpieczny proces resetowania zapomnianego hasła, który nie omija żadnych włączonych mechanizmów uwierzytelniania wieloskładnikowego.	2
6.4.4	Zweryfikuj, że w razie utraty składnika uwierzytelniania wieloskładnikowego przeprowadzane jest potwierdzenie tożsamości na tym samym poziomie, co podczas rejestracji.	2

#	Opis	Poziom
6.4.5	Zweryfikuj, że instrukcje odnowienia dla wygasających mechanizmów uwierzytelniania są wysyłane z wyprzedzeniem wystarczającym na ich wykonanie przed wygaśnięciem starego mechanizmu, z konfiguracją automatycznych przypomnień w razie potrzeby.	3
6.4.6	Zweryfikuj, że użytkownicy administracyjni mogą zainicjować proces resetowania hasła użytkownika, ale nie mogą przy tym zmienić ani wybrać hasła użytkownika. Zapobiega to sytuacji, w której znalazłoby się hasło użytkownika.	3

V6.5 Ogólne wymagania uwierzytelniania wieloskładnikowego

Ta sekcja zawiera ogólne wytyczne istotne dla różnych metod uwierzytelniania wieloskładnikowego.

Mechanizmy te obejmują:

- Sekrety odszukiwane (lookup secrets)
- Hasła jednorazowe oparte na czasie (TOTP)
- Mechanizmy pozapasmowe (out-of-band)

Sekrety odszukiwane to wstępnie wygenerowane listy tajnych kodów, podobne do numerów autoryzacji transakcji (TAN), kodów odzyskiwania w mediach społecznościowych lub siatki zawierającej zestaw losowych wartości. Ten typ mechanizmu uwierzytelniania jest uznawany za „coś, co masz”, ponieważ kody są celowo niemożliwe do zapamiętania i muszą być gdzieś przechowywane.

Hasła jednorazowe oparte na czasie (TOTP) to tokeny fizyczne lub programowe wyświetlające stale zmieniające się, pseudolosowe wyzwanie jednorazowe. Ten typ mechanizmu uwierzytelniania jest uznawany za „coś, co masz”. Wieloskładnikowe TOTP są podobne do jednoskładnikowych, ale wymagają podania prawidłowego kodu PIN, odblokowania biometrycznego, włożenia USB lub sparowania NFC, albo dodatkowej wartości (jak w kalkulatorach podpisywania transakcji), aby utworzyć ostateczne hasło jednorazowe (OTP).

Szczegóły mechanizmów pozapasmowych zostaną przedstawione w kolejnej sekcji.

Wymagania tych sekcji odnoszą się głównie do § 5.1.2, § 5.1.3, § 5.1.4.2, § 5.1.5.2, § 5.2.1 oraz § 5.2.3 wytycznych NIST.

#	Opis	Poziom
6.5.1	Zweryfikuj, że sekrety odszukiwane, pozapasmowe żądania lub kody uwierzytelniania oraz hasła jednorazowe oparte na czasie (TOTP) mogą zostać skutecznie użyte tylko raz.	2
6.5.2	Zweryfikuj, że sekrety odszukiwane o entropii mniejszej niż 112 bitów (19 losowych znaków alfanumerycznych lub 34 losowe cyfry), przechowywane w backendzie aplikacji, są haszowane zatwierdzonym algorytmem haszowania do przechowywania haseł, wykorzystującym 32-bitową losową sól. Jeśli sekret ma entropię 112 bitów lub więcej, można użyć standardowej funkcji skrótu.	2
6.5.3	Zweryfikuj, że sekrety odszukiwane, pozapasmowe kody uwierzytelniania oraz ziarna haseł jednorazowych opartych na czasie są generowane przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych (CSPRNG), aby uniknąć przewidywalnych wartości.	2
6.5.4	Zweryfikuj, że sekrety odszukiwane i pozapasmowe kody uwierzytelniania mają entropię co najmniej 20 bitów (zwykle wystarczą 4 losowe znaki alfanumeryczne lub 6 losowych cyfr).	2
6.5.5	Zweryfikuj, że pozapasmowe żądania, kody lub tokeny uwierzytelniania, a także hasła jednorazowe oparte na czasie (TOTP), mają zdefiniowany czas życia. Żądania pozapasmowe muszą mieć maksymalny czas życia 10 minut, a TOTP – maksymalnie 30 sekund.	2
6.5.6	Zweryfikuj, że każdy czynnik uwierzytelniania (w tym urządzenia fizyczne) może zostać unieważniony w przypadku kradzieży lub innej utraty.	3
6.5.7	Zweryfikuj, że biometryczne mechanizmy uwierzytelniania są używane wyłącznie jako czynniki drugorzędne, łącznie z czynnikiem „coś, co masz” lub „coś, co wiesz”.	3
6.5.8	Zweryfikuj, że hasła jednorazowe oparte na czasie (TOTP) są sprawdzane na podstawie źródła czasu z zaufanej usługi, a nie czasu niezaufanego lub dostarczonego przez klienta.	3

V6.6 Pozapasmowe mechanizmy uwierzytelniania

Zwykle polega to na komunikacji serwera uwierzytelniania z urządzeniem fizycznym poprzez bezpieczny kanał dodatkowy – na przykład wysyłaniu powiadomień push na urządzenia mobilne. Ten typ mechanizmu uwierzytelniania jest uznawany za „coś, co masz”.

Niebezpieczne pozapasmowe mechanizmy uwierzytelniania, takie jak e-mail i VOIP, są niedozwolone. Uwierzytelnianie przez PSTN i SMS jest obecnie uznawane przez NIST za mechanizmy

„ograniczone” i powinno być wycofywane na rzecz haseł jednorazowych opartych na czasie (TOTP), mechanizmu kryptograficznego lub podobnego. NIST SP 800-63B § 5.1.3.3 zaleca zajęcie się ryzykami podmiany urządzenia, zmiany karty SIM, przeniesienia numeru lub innych nietypowych zachowań, jeśli uwierzytelnianie pozapasmowe przez telefon lub SMS absolutnie musi być wspierane. Choć ta sekcja ASVS nie czyni z tego wymagania, brak tych środków ostrożności w przypadku wrażliwej aplikacji L2 lub aplikacji L3 należy traktować jako istotny sygnał ostrzegawczy.

Warto zauważyć, że NIST wydał również niedawno wytyczne, które odradzają stosowanie powiadomień push. Choć ta sekcja ASVS tego nie robi, należy mieć świadomość ryzyk związanych z „push bombingiem”.

#	Opis	Poziom
6.6.1	Zweryfikuj, że mechanizmy uwierzytelniania wykorzystujące publiczną komutowaną sieć telefoniczną (PSTN) do dostarczania haseł jednorazowych (OTP) przez telefon lub SMS są oferowane wyłącznie wtedy, gdy numer telefonu został wcześniej zweryfikowany, oferowane są także alternatywne, silniejsze metody (takie jak hasła jednorazowe oparte na czasie), a usługa informuje użytkowników o związanych z nimi ryzykach bezpieczeństwa. Dla aplikacji L3 telefon i SMS nie mogą być dostępne jako opcje.	2
6.6.2	Zweryfikuj, że pozapasmowe żądania, kody lub tokeny uwierzytelniania są powiązane z pierwotnym żądaniem uwierzytelnienia, dla którego zostały wygenerowane, i nie mogą być użyte dla żądania wcześniejszego ani późniejszego.	2
6.6.3	Zweryfikuj, że pozapasmowy mechanizm uwierzytelniania oparty na kodach jest chroniony przed atakami siłowymi poprzez ograniczanie częstotliwości żądań. Rozważ również użycie kodu o entropii co najmniej 64 bitów.	2
6.6.4	Zweryfikuj, że tam, gdzie do uwierzytelniania wieloskładnikowego używane są powiadomienia push, stosowane jest ograniczanie częstotliwości żądań, aby zapobiec atakom push bombing. Ryzyko to może ograniczyć również dopasowanie numeru (number matching).	3

V6.7 Kryptograficzny mechanizm uwierzytelniania

Kryptograficzne mechanizmy uwierzytelniania obejmują karty inteligentne lub klucze FIDO, gdzie użytkownik musi podłączyć lub sparować urządzenie kryptograficzne z komputerem, aby dokończyć uwierzytelnianie. Serwer uwierzytelniania wysyła wyzwanie (challenge nonce) do urządzenia lub oprogramowania kryptograficznego, a urządzenie lub oprogramowanie oblicza odpowiedź na podstawie bezpiecznie przechowywanego klucza kryptograficznego. Wymagania tej sekcji zawierają wskazówki implementacyjne dla tych mechanizmów; wytyczne dotyczące algorytmów kryptograficznych omówiono w rozdziale „Kryptografia”.

Tam, gdzie do uwierzytelniania kryptograficznego używane są klucze współdzielone lub tajne, powinny być one przechowywane z użyciem tych samych mechanizmów, co inne sekrety systemowe —zgodnie z sekcją „Zarządzanie sekretami”w rozdziale „Konfiguracja”.

Wymagania tej sekcji odnoszą się głównie do § 5.1.7.2 wytycznych NIST.

#	Opis	Poziom
6.7.1	Zweryfikuj, że certyfikaty używane do weryfikacji kryptograficznych asercji uwierzytelniania są przechowywane w sposób chroniący je przed modyfikacją.	3
6.7.2	Zweryfikuj, że wyzwanie (nonce) ma długość co najmniej 64 bitów i jest statystycznie unikalne lub unikalne w całym cyklu życia urządzenia kryptograficznego.	3

V6.8 Uwierzytelnianie z dostawcą tożsamości

Dostawcy tożsamości (IdP) zapewniają użytkownikom tożsamość federacyjną. Użytkownicy często mają więcej niż jedną tożsamość u wielu dostawców —na przykład tożsamość firmową w Azure AD, Okta, Ping Identity lub Google albo tożsamość konsumencką w serwisach Facebook, Twitter, Google czy WeChat, żeby wymienić tylko kilka popularnych opcji. Lista ta nie stanowi rekomendacji tych firm ani usług, a jedynie zachętę dla programistów, aby uwzględniali fakt, że wielu użytkowników posiada wiele ustanowionych tożsamości. Organizacje powinny rozważyć integrację z istniejącymi tożsamościami użytkowników, stosownie do profilu ryzyka i siły potwierdzania tożsamości danego IdP. Przykładowo mało prawdopodobne jest, aby organizacja rządowa zaakceptowała tożsamość z mediów społecznościowych jako login do systemów wrażliwych —łatwo bowiem tworzyć tożsamości fałszywe lub jednorazowe —podczas gdy firma produkująca gry mobilne może jak najbardziej potrzebować integracji z głównymi platformami społecznościowymi, aby powiększać bazę aktywnych graczy.

Bezpieczne korzystanie z zewnętrznych dostawców tożsamości wymaga starannej konfiguracji i weryfikacji, aby zapobiec podszywaniu się pod tożsamość lub sfalszowanym asercjom. Ta sekcja zawiera wymagania odnoszące się do tych ryzyk.

#	Opis	Poziom
6.8.1	Zweryfikuj, że jeśli aplikacja wspiera wielu dostawców tożsamości (IdP), tożsamości użytkownika nie można podrobić za pośrednictwem innego wspieranego dostawcy (np. używając tego samego identyfikatora użytkownika). Standardowym środkiem zaradczym jest rejestrowanie i identyfikowanie użytkownika przez aplikację na podstawie kombinacji identyfikatora IdP (pełniącego rolę przestrzeni nazw) oraz identyfikatora użytkownika w ramach danego IdP.	2
6.8.2	Zweryfikuj, że obecność i integralność podpisów cyfrowych na asercjach uwierzytelniania (na przykład na tokenach JWT lub asercjach SAML) jest zawsze walidowana, z odrzucaniem wszelkich asercji niepodpisanych lub z nieprawidłowymi podpisami.	2
6.8.3	Zweryfikuj, że asercje SAML są przetwarzane w sposób unikalny i używane tylko raz w okresie ważności, aby zapobiec atakom powtórzeniowym.	2
6.8.4	Zweryfikuj, że jeśli aplikacja korzysta z odrębnego dostawcy tożsamości (IdP) i oczekuje określonej siły, metod lub aktualności uwierzytelnienia dla konkretnych funkcji, weryfikuje to na podstawie informacji zwróconych przez IdP. Na przykład przy użyciu OIDC można to osiągnąć poprzez walidację oświadczeń tokena ID, takich jak 'acr', 'amr' i 'auth_time' (jeśli występują). Jeśli IdP nie dostarcza tych informacji, aplikacja musi mieć udokumentowane podejście awaryjne zakładające, że użyto mechanizmu uwierzytelniania o minimalnej sile (na przykład jednoskładnikowego z nazwą użytkownika i hasłem).	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- NIST SP 800-63 - Digital Identity Guidelines
- NIST SP 800-63B - Authentication and Lifecycle Management
- NIST SP 800-63 FAQ
- OWASP Web Security Testing Guide: Testing for Authentication
- OWASP Password Storage Cheat Sheet
- OWASP Forgot Password Cheat Sheet
- OWASP Choosing and Using Security Questions Cheat Sheet
- Wytyczne CISA dotyczące „dopasowania numeru” (Number Matching)
- Informacje o FIDO Alliance

V7 Zarządzanie sesją

Cel kontrolny

Mechanizmy zarządzania sesją pozwalają aplikacjom korelować interakcje użytkowników i urządzeń w czasie, nawet przy korzystaniu z bezstanowych protokołów komunikacyjnych (takich jak HTTP). Nowoczesne aplikacje mogą używać wielu tokenów sesji o odmiennych cechach i przeznaczeniu. Bezpieczny system zarządzania sesją to taki, który uniemożliwia atakującym pozyskanie, wykorzystanie lub inne nadużycie sesji ofiary. Aplikacje utrzymujące sesje muszą zapewnić spełnienie następujących wysokopoziomowych wymagań zarządzania sesją:

- Sesje są unikalne dla każdej osoby i nie mogą być odgadnięte ani współdzielone.
- Sesje są unieważniane, gdy nie są już potrzebne, oraz wygasają po okresach bezczynności.

Wiele wymagań tego rozdziału odnosi się do wybranych mechanizmów NIST SP 800-63 Digital Identity Guidelines, koncentrując się na powszechnych zagrożeniach i często wykorzystywanych lukach w ufności.

Warto zauważyć, że wymagania dotyczące konkretnych szczegółów implementacyjnych niektórych mechanizmów zarządzania sesją znajdują się w innych miejscach:

- Ciasteczka HTTP to powszechny mechanizm zabezpieczania tokenów sesji. Szczegółowe wymagania bezpieczeństwa dla ciasteczek znajdują się w rozdziale „Bezpieczeństwo frontendu webowego”.
- Tokeny samowystarczalne są często używane jako sposób utrzymywania sesji. Szczegółowe wymagania bezpieczeństwa znajdują się w rozdziale „Tokeny samowystarczalne”.

V7.1 Dokumentacja zarządzania sesją

Nie istnieje jeden wzorzec pasujący do wszystkich aplikacji. Nie jest zatem możliwe zdefiniowanie uniwersalnych granic i limitów odpowiednich dla wszystkich przypadków. Warunkiem wstępnym implementacji i testowania musi być analiza ryzyka wraz z udokumentowanymi decyzjami bezpieczeństwa dotyczącymi obsługi sesji. Zapewnia to dopasowanie systemu zarządzania sesją do konkretnych wymagań aplikacji.

Niezależnie od tego, czy wybrano mechanizm sesji stanowy, czy „bezstanowy”, analiza musi być kompletna i udokumentowana, aby wykazać, że wybrane rozwiązanie jest w stanie spełnić wszystkie istotne wymagania bezpieczeństwa. Należy również uwzględnić interakcję z ewentualnie używanymi mechanizmami jednokrotnego logowania (SSO).

#	Opis	Poziom
7.1.1	Zweryfikuj, że limit czasu bezczynności sesji użytkownika oraz bezwzględny maksymalny czas życia sesji są udokumentowane, odpowiednie w połączeniu z innymi mechanizmami, a dokumentacja zawiera uzasadnienie wszelkich odstępstw od wymagań ponownego uwierzytelniania NIST SP 800-63B.	2
7.1.2	Zweryfikuj, że dokumentacja definiuje, ile równoczesnych (równoległych) sesji jest dozwolonych dla jednego konta, a także zamierzone zachowania i działania podejmowane po osiągnięciu maksymalnej liczby aktywnych sesji.	2
7.1.3	Zweryfikuj, że wszystkie systemy tworzące i zarządzające sesjami użytkowników w ramach ekosystemu federacyjnego zarządzania tożsamością (takie jak systemy SSO) są udokumentowane wraz z mechanizmami koordynacji czasów życia sesji, ich kończenia oraz wszelkich innych warunków wymagających ponownego uwierzytelnienia.	2

V7.2 Podstawowe bezpieczeństwo zarządzania sesją

Ta sekcja realizuje zasadnicze wymagania bezpiecznych sesji poprzez weryfikację, że tokeny sesji są bezpiecznie generowane i walidowane.

#	Opis	Poziom
7.2.1	Zweryfikuj, że aplikacja wykonuje całą weryfikację tokenów sesji z użyciem zaufanej usługi backendowej.	1
7.2.2	Zweryfikuj, że aplikacja używa do zarządzania sesją tokenów samowystarczalnych albo referencyjnych, generowanych dynamicznie —tj. nie używa statycznych sekretów i kluczy API.	1
7.2.3	Zweryfikuj, że jeśli do reprezentowania sesji użytkowników używane są tokeny referencyjne, są one unikalne, generowane przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych (CSPRNG) i mają entropię co najmniej 128 bitów.	1
7.2.4	Zweryfikuj, że aplikacja generuje nowy token sesji przy uwierzytelnieniu użytkownika, w tym przy ponownym uwierzytelnieniu, oraz unieważnia dotychczasowy token sesji.	1

V7.3 Wygasanie sesji

Mechanizmy wygasania sesji służą minimalizacji okna czasowego dla przejęcia sesji i innych form jej nadużycia. Limity czasowe muszą odpowiadać udokumentowanym decyzjom bezpieczeństwa.

#	Opis	Poziom
7.3.1	Zweryfikuj, że istnieje limit czasu beczynności, po którym wymuszane jest ponowne uwierzytelnienie, zgodnie z analizą ryzyka i udokumentowanymi decyzjami bezpieczeństwa.	2
7.3.2	Zweryfikuj, że istnieje bezwzględny maksymalny czas życia sesji, po którym wymuszane jest ponowne uwierzytelnienie, zgodnie z analizą ryzyka i udokumentowanymi decyzjami bezpieczeństwa.	2

V7.4 Kończenie sesji

Kończenie sesji może być obsługiwane przez samą aplikację albo przez dostawcę SSO, jeśli to on zarządza sesją zamiast aplikacji. Rozważając wymagania tej sekcji, może być konieczne rozstrzygnięcie, czy dostawca SSO znajduje się w zakresie weryfikacji, ponieważ część z nich może być kontrolowana przez dostawcę.

Zakończenie sesji powinno skutkować koniecznością ponownego uwierzytelnienia i być skuteczne w całej aplikacji, logowaniu federacyjnym (jeśli występuje) oraz u wszystkich stron ufających.

Dla stanowych mechanizmów sesji zakończenie zwykle polega na unieważnieniu sesji w backendzie. W przypadku tokenów samowystarczalnych wymagane są dodatkowe środki unieważnienia lub zablokowania tych tokenów — w przeciwnym razie mogą one pozostać ważne aż do wygaśnięcia.

#	Opis	Poziom
7.4.1	Zweryfikuj, że po wyzwoleniu zakończenia sesji (np. wylogowanie lub wygaśnięcie) aplikacja uniemożliwia dalsze korzystanie z sesji. Dla tokenów referencyjnych lub sesji stanowych oznacza to unieważnienie danych sesji w backendzie aplikacji. Aplikacje używające tokenów samowystarczalnych będą potrzebowały rozwiązania takiego jak utrzymywanie listy zakończonych tokenów, odrzucanie tokenów wystawionych przed określoną dla każdego użytkownika datą i godziną lub rotacja klucza podpisującego dla każdego użytkownika.	1
7.4.2	Zweryfikuj, że aplikacja kończy wszystkie aktywne sesje, gdy konto użytkownika zostaje wyłączone lub usunięte (np. gdy pracownik odchodzi z firmy).	1

#	Opis	Poziom
7.4.3	Zweryfikuj, że aplikacja daje możliwość zakończenia wszystkich pozostałych aktywnych sesji po udanej zmianie lub usunięciu dowolnego czynnika uwierzytelniania (w tym zmianie hasła poprzez reset lub odzyskiwanie oraz –jeśli występuje –zmianie ustawień MFA).	2
7.4.4	Zweryfikuj, że wszystkie strony wymagające uwierzytelnienia mają łatwy i widoczny dostęp do funkcji wylogowania.	2
7.4.5	Zweryfikuj, że administratorzy aplikacji mają możliwość kończenia aktywnych sesji pojedynczego użytkownika lub wszystkich użytkowników.	2

V7.5 Obrona przed nadużyciem sesji

Ta sekcja zawiera wymagania ograniczające ryzyko stwarzane przez aktywne sesje, które zostały przejęte lub są nadużywane poprzez wektory bazujące na istnieniu i możliwościach aktywnych sesji użytkowników. Przykładem jest wykorzystanie wykonania złośliwej treści do zmuszenia uwierzytelnionej przeglądarki ofiary do wykonania akcji z użyciem jej sesji.

Rozważając wymagania tej sekcji, należy wziąć pod uwagę wytyczne dla poszczególnych poziomów z rozdziału „Uwierzytelnianie”.

#	Opis	Poziom
7.5.1	Zweryfikuj, że aplikacja wymaga pełnego ponownego uwierzytelnienia przed zezwoleniem na modyfikację wrażliwych atrybutów konta, które mogą wpływać na uwierzytelnianie –takich jak adres e-mail, numer telefonu, konfiguracja MFA lub inne informacje używane przy odzyskiwaniu konta.	2
7.5.2	Zweryfikuj, że użytkownicy mogą przeglądać i (po ponownym uwierzytelnieniu co najmniej jednym czynnikiem) kończyć dowolne lub wszystkie aktualnie aktywne sesje.	2
7.5.3	Zweryfikuj, że aplikacja wymaga dodatkowego uwierzytelnienia co najmniej jednym czynnikiem lub weryfikacji wtórnej przed wykonaniem wysoce wrażliwych transakcji lub operacji.	3

V7.6 Ponowne uwierzytelnianie federacyjne

Ta sekcja dotyczy osób tworzących kod strony ufającej (Relying Party, RP) lub dostawcy tożsamości (IdP). Wymagania te wywodzą się z NIST SP 800-63C dotyczącego federacji i asercji.

#	Opis	Poziom
7.6.1	Zweryfikuj, że czas życia i kończenie sesji między stronami ufającymi (RP) a dostawcami tożsamości (IdP) zachowują się zgodnie z dokumentacją, wymagając ponownego uwierzytelnienia w razie potrzeby – na przykład po osiągnięciu maksymalnego czasu między zdarzeniami uwierzytelnienia w IdP.	2
7.6.2	Zweryfikuj, że utworzenie sesji wymaga zgody użytkownika albo jego jawnego działania, uniemożliwiając tworzenie nowych sesji aplikacji bez interakcji użytkownika.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Session Management Testing
- OWASP Session Management Cheat Sheet

V8 Autoryzacja

Cel kontrolny

Autoryzacja zapewnia, że dostęp jest przyznawany wyłącznie uprawnionym konsumentom (użytkownikom, serwerom i innym klientom). Aby wymusić zasadę najmniejszych uprawnień (Principle of Least Privilege, POLP), weryfikowane aplikacje muszą spełniać następujące wysokopoziomowe wymagania:

- Reguły autoryzacji są udokumentowane, wraz z czynnikami decyzyjnymi i kontekstami środowiskowymi.
- Konsumenci powinni mieć dostęp wyłącznie do zasobów dozwolonych przez zdefiniowane dla nich uprawnienia.

V8.1 Dokumentacja autoryzacji

Kompleksowa dokumentacja autoryzacji jest niezbędna, aby decyzje bezpieczeństwa były stosowane spójnie, audytowalne i zgodne z politykami organizacji. Zmniejsza to ryzyko nieautoryzowanego dostępu, czyniąc wymagania bezpieczeństwa jasnymi i wykonalnymi dla programistów, administratorów i testerów.

#	Opis	Poziom
8.1.1	Zweryfikuj, że dokumentacja autoryzacji definiuje reguły ograniczania dostępu na poziomie funkcji oraz dostępu do konkretnych danych na podstawie uprawnień konsumenta i atrybutów zasobu.	1
8.1.2	Zweryfikuj, że dokumentacja autoryzacji definiuje reguły ograniczeń dostępu na poziomie pól (zarówno odczytu, jak i zapisu) na podstawie uprawnień konsumenta i atrybutów zasobu. Zwróć uwagę, że reguły te mogą zależeć od innych wartości atrybutów danego obiektu danych, takich jak stan lub status.	2
8.1.3	Zweryfikuj, że dokumentacja aplikacji definiuje atrybuty środowiskowe i kontekstowe (w tym między innymi porę dnia, lokalizację użytkownika, adres IP lub urządzenie), które są używane w aplikacji do podejmowania decyzji bezpieczeństwa, w tym dotyczących uwierzytelniania i autoryzacji.	3
8.1.4	Zweryfikuj, że dokumentacja uwierzytelniania i autoryzacji definiuje, w jaki sposób czynniki środowiskowe i kontekstowe są wykorzystywane w podejmowaniu decyzji, obok autoryzacji na poziomie funkcji, konkretnych danych i pól. Powinno to obejmować oceniane atrybuty, progi ryzyka oraz podejmowane działania (np. zezwól, zażądaj potwierdzenia, odmów, uwierzytelnianie stopniowe).	3

V8.2 Ogólny projekt autoryzacji

Wdrożenie granularnych mechanizmów autoryzacji na poziomie funkcji, danych i pól zapewnia, że konsumenci mogą uzyskać dostęp wyłącznie do tego, co zostało im jawnie przyznane.

#	Opis	Poziom
8.2.1	Zweryfikuj, że aplikacja zapewnia ograniczenie dostępu na poziomie funkcji do konsumentów posiadających jawne uprawnienia.	1
8.2.2	Zweryfikuj, że aplikacja zapewnia ograniczenie dostępu do konkretnych danych do konsumentów posiadających jawne uprawnienia do konkretnych elementów danych, aby ograniczyć ryzyko insecure direct object reference (IDOR) oraz broken object level authorization (BOLA).	1
8.2.3	Zweryfikuj, że aplikacja zapewnia ograniczenie dostępu na poziomie pól do konsumentów posiadających jawne uprawnienia do konkretnych pól, aby ograniczyć ryzyko broken object property level authorization (BOPLA).	2

#	Opis	Poziom
8.2.4	Zweryfikuj, że adaptacyjne mechanizmy bezpieczeństwa oparte na atrybutach środowiskowych i kontekstowych konsumenta (takich jak pora dnia, lokalizacja, adres IP lub urządzenie) są wdrożone dla decyzji uwierzytelniania i autoryzacji, zgodnie z dokumentacją aplikacji. Mechanizmy te muszą być stosowane zarówno gdy konsument próbuje rozpocząć nową sesję, jak i w trakcie istniejącej sesji.	3

V8.3 Autoryzacja na poziomie operacji

Natychmiastowe stosowanie zmian autoryzacji w odpowiedniej warstwie architektury aplikacji jest kluczowe dla zapobiegania nieautoryzowanym działaniom, zwłaszcza w środowiskach dynamicznych.

#	Opis	Poziom
8.3.1	Zweryfikuj, że aplikacja egzekwuje reguły autoryzacji w zaufanej warstwie usługowej i nie polega na mechanizmach, którymi niezaufany konsument mógłby manipulować, takich jak JavaScript po stronie klienta.	1
8.3.2	Zweryfikuj, że zmiany wartości, na podstawie których podejmowane są decyzje autoryzacyjne, są stosowane natychmiast. Tam, gdzie zmian nie da się zastosować natychmiast (na przykład przy poleganiu na danych w tokenach samowystarczalnych), muszą istnieć mechanizmy kompensujące, które alarmują, gdy konsument wykonuje działanie, do którego nie jest już uprawniony, i wycofują zmianę. Zwróć uwagę, że ta alternatywa nie ograniczy wycieku informacji.	3
8.3.3	Zweryfikuj, że dostęp do obiektu opiera się na uprawnieniach pierwotnego podmiotu (np. konsumenta), a nie na uprawnieniach pośrednika lub usługi działającej w jego imieniu. Na przykład jeśli konsument wywołuje usługę sieciową, uwierzytelniając się tokenem samowystarczalnym, a usługa następnie żąda danych od innej usługi, druga usługa użyje do decyzji o uprawnieniach tokena konsumenta, a nie tokena maszyna-maszyna pierwszej usługi.	3

V8.4 Pozostałe kwestie autoryzacji

Dodatkowe kwestie autoryzacji —w szczególności dotyczące interfejsów administracyjnych i środowisk wielodostępnych —pomagają zapobiegać nieautoryzowanemu dostępowi.

#	Opis	Poziom
8.4.1	Zweryfikuj, że aplikacje wielodostępne stosują mechanizmy separacji między najemcami, zapewniające, że operacje konsumenta nigdy nie wpłyną na najemców, z którymi nie ma on uprawnień do interakcji.	2
8.4.2	Zweryfikuj, że dostęp do interfejsów administracyjnych obejmuje wiele warstw bezpieczeństwa, w tym ciągłą weryfikację tożsamości konsumenta, ocenę stanu bezpieczeństwa urządzenia oraz kontekstową analizę ryzyka — zapewniając, że lokalizacja sieciowa lub zaufane endpointy nie są jedynymi czynnikami autoryzacji, nawet jeśli zmniejszają prawdopodobieństwo nieautoryzowanego dostępu.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Authorization
- OWASP Authorization Cheat Sheet

V9 Tokeny samowystarczalne

Cel kontrolny

Pojęcie tokena samowystarczального pojawia się już w pierwotnym RFC 6749 OAuth 2.0 z 2012 roku. Odnosi się do tokena zawierającego dane lub oświadczenia, na których usługa odbierająca będzie polegać przy podejmowaniu decyzji bezpieczeństwa. Należy go odróżnić od prostego tokena zawierającego wyłącznie identyfikator, na podstawie którego usługa odbierająca wyszukuje dane lokalnie. Najczęstsze przykłady tokenów samowystarczalnych to JSON Web Tokens (JWT) oraz asercje SAML.

Stosowanie tokenów samowystarczalnych stało się bardzo powszechne, również poza OAuth i OIDC. Jednocześnie bezpieczeństwo tego mechanizmu opiera się na zdolności walidacji integralności tokena oraz zapewnieniu, że token jest ważny w danym kontekście. W procesie tym istnieje wiele pułapek, a niniejszy rozdział szczegółowo opisuje mechanizmy, które aplikacje powinny wdrożyć, aby ich uniknąć.

V9.1 Źródło i integralność tokena

Ta sekcja zawiera wymagania zapewniające, że token został wytworzony przez zaufaną stronę i nie został zmanipulowany.

#	Opis	Poziom
9.1.1	Zweryfikuj, że tokeny samowystarczalne są walidowane z użyciem ich podpisu cyfrowego lub kodu MAC w celu ochrony przed manipulacją, zanim zawartość tokena zostanie zaakceptowana.	1
9.1.2	Zweryfikuj, że do tworzenia i weryfikacji tokenów samowystarczalnych w danym kontekście mogą być używane wyłącznie algorytmy z listy dozwolonych. Lista dozwolonych musi zawierać dopuszczone algorytmy — najlepiej wyłącznie symetryczne albo wyłącznie asymetryczne — i nie może zawierać algorytmu 'None'. Jeśli muszą być wspierane zarówno algorytmy symetryczne, jak i asymetryczne, konieczne będą dodatkowe mechanizmy zapobiegające pomyleniu kluczy (key confusion).	1
9.1.3	Zweryfikuj, że materiał klucza używany do walidacji tokenów samowystarczalnych pochodzi z zaufanych, wstępnie skonfigurowanych źródeł dla danego wystawcy tokena, uniemożliwiając atakującym wskazanie niezaufanych źródeł i kluczy. Dla JWT i innych struktur JWS nagłówki takie jak 'jku', 'x5u' i 'jwk' muszą być walidowane względem listy dozwolonych zaufanych źródeł.	1

V9.2 Zawartość tokena

Przed podjęciem decyzji bezpieczeństwa na podstawie zawartości tokena samowystarczального konieczna jest walidacja, że token został przedstawiony w swoim okresie ważności oraz że jest przeznaczony do użytku przez usługę odbierającą i w celu, w jakim został przedstawiony. Pomaga to uniknąć niebezpiecznego użycia krzyżowego między różnymi usługami lub z różnymi typami tokenów od tego samego wystawcy.

Szczegółowe wymagania dla OAuth i OIDC omówiono w dedykowanym rozdziale.

#	Opis	Poziom
9.2.1	Zweryfikuj, że jeśli w danych tokena obecny jest przedział czasu ważności, token i jego zawartość są akceptowane tylko wtedy, gdy czas weryfikacji mieści się w tym przedziale. Na przykład dla JWT muszą być weryfikowane oświadczenia 'nbf' i 'exp'.	1
9.2.2	Zweryfikuj, że usługa odbierająca token waliduje, czy token jest właściwego typu i przeznaczony do zamierzonego celu, zanim zaakceptuje jego zawartość. Na przykład do decyzji autoryzacyjnych mogą być akceptowane wyłącznie tokeny dostępu, a do potwierdzania uwierzytelnienia użytkownika — wyłącznie tokeny ID.	2

#	Opis	Poziom
9.2.3	Zweryfikuj, że usługa akceptuje wyłącznie tokeny przeznaczone do użytku z tą usługą (audience). Dla JWT można to osiągnąć, walidując oświadczenie 'aud' względem listy dozwolonych zdefiniowanej w usłudze.	2
9.2.4	Zweryfikuj, że jeśli wystawca tokenów używa tego samego klucza prywatnego do wystawiania tokenów dla różnych odbiorców (audiences), wystawiane tokeny zawierają ograniczenie odbiorcy jednoznacznie identyfikujące zamierzonych odbiorców. Zapobiegnie to ponownemu użyciu tokena z niezamierzonym odbiorcą. Jeśli identyfikator odbiorcy jest przydzielany dynamicznie, wystawca tokenów musi walidować tych odbiorców, aby upewnić się, że nie prowadzi to do podszywania się pod odbiorcę.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP JSON Web Token Cheat Sheet for Java Cheat Sheet (ale zawiera przydatne ogólne wskazówki)

V10 OAuth i OIDC

Cel kontrolny

OAuth2 (w tym rozdziale nazywany OAuth) to branżowy standard delegowanej autoryzacji. Przykładowo z użyciem OAuth aplikacja kliencka może uzyskać dostęp do API (zasobów serwera) w imieniu użytkownika, o ile użytkownik ją do tego upoważnił.

Sam OAuth nie został zaprojektowany do uwierzytelniania użytkowników. Framework OpenID Connect (OIDC) rozszerza OAuth, dodając nad nim warstwę tożsamości użytkownika. OIDC zapewnia wsparcie dla funkcji obejmujących ustandaryzowane informacje o użytkowniku, jednokrotne logowanie (SSO) oraz zarządzanie sesją. Ponieważ OIDC jest rozszerzeniem OAuth, wymagania OAuth z tego rozdziału mają zastosowanie również do OIDC.

W OAuth zdefiniowane są następujące role:

- Klient OAuth to aplikacja, która próbuje uzyskać dostęp do zasobów serwera (np. wywołując API z użyciem wystawionego tokena dostępu). Klient OAuth jest często aplikacją serwerową.
 - Klient poufny (confidential client) to klient zdolny do zachowania poufności poświadczeń, których używa do uwierzytelniania się wobec serwera autoryzacji.

- Klient publiczny (public client) nie jest zdolny do zachowania poufności poświadczeń służących do uwierzytelniania wobec serwera autoryzacji. Dlatego zamiast się uwierzytelniać (np. parametrami 'client_id' i 'client_secret'), jedynie się identyfikuje (parametrem 'client_id').
- Serwer zasobów OAuth (resource server, RS) to serwerowe API udostępniające zasoby klientom OAuth.
- Serwer autoryzacji OAuth (authorization server, AS) to aplikacja serwerowa wystawiająca tokeny dostępu klientom OAuth. Tokeny te pozwalają klientom OAuth uzyskiwać dostęp do zasobów RS – w imieniu użytkownika końcowego albo we własnym imieniu klienta OAuth. AS jest często odrębną aplikacją, ale (jeśli to właściwe) może być zintegrowany z odpowiednim RS.
- Właściciel zasobu (resource owner, RO) to użytkownik końcowy, który upoważnia klientów OAuth do uzyskania ograniczonego dostępu do zasobów hostowanych na serwerze zasobów w jego imieniu. Właściciel zasobu wyraża zgodę na tę delegowaną autoryzację poprzez interakcję z serwerem autoryzacji.

W OIDC zdefiniowane są następujące role:

- Strona ufająca (relying party, RP) to aplikacja kliencka żądająca uwierzytelnienia użytkownika końcowego za pośrednictwem dostawcy OpenID. Pełni rolę klienta OAuth.
- Dostawca OpenID (OpenID Provider, OP) to serwer autoryzacji OAuth zdolny do uwierzytelnienia użytkownika końcowego i dostarczający stronie ufającej oświadczenia OIDC. OP może być dostawcą tożsamości (IdP), ale w scenariuszach federacyjnych OP i dostawca tożsamości (u którego uwierzytelnia się użytkownik końcowy) mogą być różnymi aplikacjami serwerowymi.

OAuth i OIDC zostały pierwotnie zaprojektowane dla aplikacji stron trzecich. Obecnie są często używane również przez aplikacje własne (first-party). Jednak w scenariuszach first-party – takich jak uwierzytelnianie i zarządzanie sesją – protokół wprowadza dodatkową złożoność, która może rodzić nowe wyzwania bezpieczeństwa.

OAuth i OIDC mogą być stosowane w wielu typach aplikacji, ale ASVS i wymagania tego rozdziału koncentrują się na aplikacjach internetowych i API.

Ponieważ OAuth i OIDC można traktować jako logikę zbudowaną na technologiach webowych, ogólne wymagania z pozostałych rozdziałów zawsze mają zastosowanie, a tego rozdziału nie można rozpatrywać w oderwaniu od kontekstu.

Niniejszy rozdział odzwierciedla aktualne najlepsze praktyki dla OAuth2 i OIDC, zgodne ze specyfikacjami dostępnymi pod adresami <https://oauth.net/2/> oraz <https://openid.net/developers/specs/>. Nawet jeśli RFC są uznawane za dojrzałe, są często aktualizowane – dlatego stosując wymagania tego rozdziału, ważne jest trzymanie się najnowszych wersji. Więcej szczegółów w sekcji źródeł.

Zważywszy na złożoność tego obszaru, dla bezpiecznego rozwiązania OAuth lub OIDC absolutnie kluczowe jest korzystanie z uznanych, branżowych serwerów autoryzacji oraz stosowanie zalecanej konfiguracji bezpieczeństwa.

Terminologia użyta w tym rozdziale jest zgodna z RFC OAuth i specyfikacjami OIDC, przy czym terminologia OIDC jest używana wyłącznie w wymaganiach specyficznych dla OIDC; w pozostałych przypadkach stosowana jest terminologia OAuth.

W kontekście OAuth i OIDC termin „token” w tym rozdziale odnosi się do:

- Tokenów dostępu, które mogą być konsumowane wyłącznie przez RS i mogą być tokenami referencyjnymi walidowanymi poprzez introspekcję albo tokenami samowystarczalnymi walidowanymi z użyciem materiału klucza.
- Tokenów odświeżania, które mogą być konsumowane wyłącznie przez serwer autoryzacji, który je wystawił.
- Tokenów ID OIDC, które mogą być konsumowane wyłącznie przez klienta, który zainicjował przepływ autoryzacji.

Poziomy ryzyka niektórych wymagań tego rozdziału zależą od tego, czy klient jest klientem poufnym, czy uznawany jest za klienta publicznego. Ponieważ silne uwierzytelnianie klienta ogranicza wiele wektorów ataku, kilka wymagań może zostać złagodzonych przy stosowaniu klienta poufnego w aplikacjach L1.

V10.1 Ogólne bezpieczeństwo OAuth i OIDC

Ta sekcja obejmuje ogólne wymagania architektoniczne mające zastosowanie do wszystkich aplikacji korzystających z OAuth lub OIDC.

#	Opis	Poziom
10.1.1	Zweryfikuj, że tokeny są wysyłane wyłącznie do komponentów, które ściśle ich potrzebują. Na przykład przy stosowaniu wzorca backend-for-frontend dla przeglądarek aplikacji JavaScript tokeny dostępu i odświeżania mogą być dostępne wyłącznie dla backendu.	2
10.1.2	Zweryfikuj, że klient akceptuje wartości od serwera autoryzacji (takie jak kod autoryzacyjny lub token ID) tylko wtedy, gdy wartości te pochodzą z przepływu autoryzacji zainicjowanego przez tę samą sesję agenta użytkownika i tę samą transakcję. Wymaga to, aby sekrety generowane przez klienta —takie jak ‘code_verifier’ z proof key for code exchange (PKCE), ‘state’ lub ‘nonce’ OIDC —były nieodgadnywalne, specyficzne dla transakcji oraz bezpiecznie powiązane zarówno z klientem, jak i z sesją agenta użytkownika, w której transakcja została rozpoczęta.	2

V10.2 Klient OAuth

Te wymagania określają obowiązki aplikacji klienckich OAuth. Klientem może być na przykład backend serwera WWW (często działający jako Backend For Frontend, BFF), integracja usługi backendowej albo frontendowa aplikacja typu Single Page Application (SPA, zwana też aplikacją przeglądarkową).

Zasadniczo klienci backendowi są uznawani za klientów poufnych, a klienci frontendowi – za klientów publicznych. Jednak aplikacje natywne działające na urządzeniu użytkownika końcowego mogą być uznane za poufne przy stosowaniu dynamicznej rejestracji klienta OAuth.

#	Opis	Poziom
10.2.1	Zweryfikuj, że jeśli używany jest przepływ kodu (code flow), klient OAuth posiada ochronę przed przeglądarkowymi atakami fałszowania żądań – powszechnie znanymi jako cross-site request forgery (CSRF) – wyzwalającymi żądania tokenów, poprzez użycie mechanizmu proof key for code exchange (PKCE) albo sprawdzanie parametru ‘state’ wysłanego w żądaniu autoryzacji.	2
10.2.2	Zweryfikuj, że jeśli klient OAuth może współpracować z więcej niż jednym serwerem autoryzacji, posiada obronę przed atakami mix-up. Na przykład może wymagać, aby serwer autoryzacji zwracał wartość parametru ‘iss’, i walidować ją w odpowiedzi autoryzacji oraz odpowiedzi tokena.	2
10.2.3	Zweryfikuj, że klient OAuth żąda w żądaniach do serwera autoryzacji wyłącznie wymaganych zakresów (scopes) lub innych parametrów autoryzacji.	3

V10.3 Serwer zasobów OAuth

W kontekście ASVS i tego rozdziału serwer zasobów to API. Aby zapewnić bezpieczny dostęp, serwer zasobów musi:

- Zwalidować token dostępu zgodnie z formatem tokena i właściwymi specyfikacjami protokołu, np. walidacją JWT lub introspekcją tokena OAuth.
- Jeśli token jest ważny – egzekwować decyzje autoryzacyjne na podstawie informacji z tokena dostępu oraz przyznanych uprawnień. Na przykład serwer zasobów musi zweryfikować, że klient (działający w imieniu RO) jest uprawniony do dostępu do żądanego zasobu.

Wymienione tu wymagania są zatem specyficzne dla OAuth lub OIDC i powinny być realizowane po walidacji tokena, a przed wykonaniem autoryzacji na podstawie informacji z tokena.

#	Opis	Poziom
10.3.1	Zweryfikuj, że serwer zasobów akceptuje wyłącznie tokeny dostępu przeznaczone do użytku z tą usługą (audience). Odbiorca może być zawarty w ustrukturyzowanym tokenie dostępu (np. oświadczenie 'aud' w JWT) albo sprawdzany z użyciem endpointu introspekcji tokenów.	2
10.3.2	Zweryfikuj, że serwer zasobów egzekwuje decyzje autoryzacyjne na podstawie oświadczeń z tokena dostępu definiujących delegowaną autoryzację. Jeśli obecne są oświadczenia takie jak 'sub', 'scope' i 'authorization_details', muszą one stanowić część decyzji.	2
10.3.3	Zweryfikuj, że jeśli decyzja kontroli dostępu wymaga zidentyfikowania unikalnego użytkownika na podstawie tokena dostępu (JWT lub powiązanej odpowiedzi introspekcji tokena), serwer zasobów identyfikuje użytkownika na podstawie oświadczeń, których nie można przypisać ponownie innym użytkownikom. Zwykle oznacza to użycie kombinacji oświadczeń 'iss' i 'sub'.	2
10.3.4	Zweryfikuj, że jeśli serwer zasobów wymaga określonej siły, metod lub aktualności uwierzytelnienia, weryfikuje, czy przedstawiony token dostępu spełnia te ograniczenia – na przykład, jeśli występują, z użyciem oświadczeń OIDC odpowiednio 'acr', 'amr' i 'auth_time'.	2
10.3.5	Zweryfikuj, że serwer zasobów zapobiega użyciu skradzionych tokenów dostępu lub ich powtórzeniu (przez strony nieuprawnione), wymagając tokenów dostępu ograniczonych do nadawcy (sender-constrained) – z użyciem Mutual TLS dla OAuth 2 albo OAuth 2 Demonstration of Proof of Possession (DPoP).	3

V10.4 Serwer autoryzacji OAuth

Te wymagania określają obowiązki serwerów autoryzacji OAuth, w tym dostawców OpenID.

Dla uwierzytelniania klienta dozwolona jest metoda 'self_signed_tls_client_auth', z zastrzeżeniem warunków wstępnych wymaganych przez sekcję 2.2 dokumentu RFC 8705.

#	Opis	Poziom
10.4.1	Zweryfikuj, że serwer autoryzacji waliduje URI przekierowania na podstawie specyficznej dla klienta listy dozwolonych wstępnie zarejestrowanych URI, stosując porównanie dokładne (exact string comparison).	1

#	Opis	Poziom
10.4.2	Zweryfikuj, że jeśli serwer autoryzacji zwraca kod autoryzacyjny w odpowiedzi autoryzacji, kod ten może zostać użyty tylko raz w żądaniu tokena. Przy drugim poprawnym żądaniu z kodem autoryzacyjnym, który został już użyty do wystawienia tokena dostępu, serwer autoryzacji musi odrzucić żądanie tokena oraz unieważnić wszystkie wystawione tokeny powiązane z tym kodem autoryzacyjnym.	1
10.4.3	Zweryfikuj, że kod autoryzacyjny jest krótkotrwały. Maksymalny czas życia może wynosić do 10 minut dla aplikacji L1 i L2 oraz do 1 minuty dla aplikacji L3.	1
10.4.4	Zweryfikuj, że dla danego klienta serwer autoryzacji zezwala wyłącznie na użycie grantów, których ten klient potrzebuje. Zwróć uwagę, że granty 'token' (przepływ Implicit) oraz 'password' (przepływ Resource Owner Password Credentials) nie mogą być już używane.	1
10.4.5	Zweryfikuj, że serwer autoryzacji ogranicza ryzyko ataków powtórzeniowych na tokeny odświeżania dla klientów publicznych, najlepiej z użyciem tokenów odświeżania ograniczonych do nadawcy, tj. Demonstrating Proof of Possession (DPoP) lub tokenów dostępu powiązanych z certyfikatem z użyciem mutual TLS (mTLS). Dla aplikacji L1 i L2 można stosować rotację tokenów odświeżania. Jeśli stosowana jest rotacja, serwer autoryzacji musi unieważnić token odświeżania po użyciu oraz odwołać wszystkie tokeny odświeżania dla danej autoryzacji, jeśli przedstawiony zostanie token odświeżania już użyty i unieważniony.	1
10.4.6	Zweryfikuj, że jeśli używany jest grant kodu, serwer autoryzacji ogranicza ryzyko ataków przechwycenia kodu autoryzacyjnego, wymagając proof key for code exchange (PKCE). Dla żądań autoryzacji serwer autoryzacji musi wymagać prawidłowej wartości 'code_challenge' i nie może akceptować wartości 'code_challenge_method' równej 'plain'. Dla żądania tokena musi wymagać walidacji parametru 'code_verifier'.	2
10.4.7	Zweryfikuj, że jeśli serwer autoryzacji wspiera niewierzytelną dynamiczną rejestrację klientów, ogranicza ryzyko złośliwych aplikacji klienckich. Musi walidować metadane klienta, takie jak wszelkie rejestrowane URI, zapewnić zgodę użytkownika oraz ostrzec użytkownika przed przetworzeniem żądania autoryzacji z niezaufaną aplikacją kliencką.	2
10.4.8	Zweryfikuj, że tokeny odświeżania mają bezwzględny termin wygaśnięcia, również wtedy, gdy stosowane jest przesuwne wygasanie tokenów odświeżania.	2

#	Opis	Poziom
10.4.9	Zweryfikuj, że tokeny odświeżania i referencyjne tokeny dostępu mogą zostać odwołane przez uprawnionego użytkownika z poziomu interfejsu użytkownika serwera autoryzacji, aby ograniczyć ryzyko złośliwych klientów lub skradzionych tokenów.	2
10.4.10	Zweryfikuj, że klient poufny jest uwierzytelniany dla żądań kanałem zwrotnym (backchannel) od klienta do serwera autoryzacji, takich jak żądania tokenów, pushed authorization requests (PAR) oraz żądania odwołania tokenów.	2
10.4.11	Zweryfikuj, że konfiguracja serwera autoryzacji przypisuje klientowi OAuth wyłącznie wymagane zakresy.	2
10.4.12	Zweryfikuj, że dla danego klienta serwer autoryzacji zezwala wyłącznie na wartość 'response_mode', której ten klient potrzebuje — na przykład poprzez walidację tej wartości przez serwer autoryzacji względem wartości oczekiwanych albo użycie pushed authorization request (PAR) lub JWT-secured Authorization Request (JAR).	3
10.4.13	Zweryfikuj, że grant typu 'code' jest zawsze używany łącznie z pushed authorization requests (PAR).	3
10.4.14	Zweryfikuj, że serwer autoryzacji wystawia wyłącznie tokeny dostępu ograniczone do nadawcy (Proof-of-Possession) — z tokenami dostępu powiązanymi z certyfikatem z użyciem mutual TLS (mTLS) albo tokenami powiązanymi z DPOP (Demonstration of Proof of Possession).	3
10.4.15	Zweryfikuj, że dla klienta serwerowego (niewykonywanego na urządzeniu użytkownika końcowego) serwer autoryzacji zapewnia, że wartość parametru 'authorization_details' pochodzi z backendu klienta i nie została zmanipulowana przez użytkownika — na przykład poprzez wymaganie użycia pushed authorization request (PAR) lub JWT-secured Authorization Request (JAR).	3
10.4.16	Zweryfikuj, że klient jest poufny, a serwer autoryzacji wymaga stosowania silnych metod uwierzytelniania klienta (opartych na kryptografii klucza publicznego i odpornych na ataki powtórzeniowe), takich jak mutual TLS ('tls_client_auth', 'self_signed_tls_client_auth') lub private key JWT ('private_key_jwt').	3

V10.5 Klient OIDC

Ponieważ strona ufająca OIDC działa jako klient OAuth, zastosowanie mają również wymagania z sekcji „Klient OAuth”.

Zwróć uwagę, że sekcja „Uwierzytelnianie z dostawcą tożsamości” w rozdziale „Uwierzytelnianie” również zawiera istotne wymagania ogólne.

#	Opis	Poziom
10.5.1	Zweryfikuj, że klient (jako strona ufająca) ogranicza ryzyko ataków powtórzeniowych na token ID – na przykład zapewniając, że oświadczenie ‘nonce’ w tokenie ID odpowiada wartości ‘nonce’ wysłanej w żądaniu uwierzytelnienia do dostawcy OpenID (w OAuth2 nazywanym żądaniem autoryzacji wysyłanym do serwera autoryzacji).	2
10.5.2	Zweryfikuj, że klient jednoznacznie identyfikuje użytkownika na podstawie oświadczeń tokena ID – zwykle oświadczenia ‘sub’ – których nie można przypisać ponownie innym użytkownikom (w zakresie danego dostawcy tożsamości).	2
10.5.3	Zweryfikuj, że klient odrzuca próby podszycia się złośliwego serwera autoryzacji pod inny serwer autoryzacji za pośrednictwem metadanych serwera autoryzacji. Klient musi odrzucić metadane serwera autoryzacji, jeśli URL wystawcy w tych metadanych nie odpowiada dokładnie wstępnie skonfigurowanemu URL wystawcy oczekiwanemu przez klienta.	2
10.5.4	Zweryfikuj, że klient waliduje, iż token ID jest przeznaczony do użytku przez tego klienta (audience), sprawdzając, że oświadczenie ‘aud’ z tokena jest równe wartości ‘client_id’ klienta.	2
10.5.5	Zweryfikuj, że przy korzystaniu z wylogowania kanałem zwrotnym OIDC (back-channel logout) strona ufająca ogranicza ryzyko odmowy usługi poprzez wymuszone wylogowanie oraz pomylenia różnych JWT (cross-JWT confusion) w przepływie wylogowania. Klient musi zweryfikować, że token wylogowania ma poprawny typ o wartości ‘logout+jwt’, zawiera oświadczenie ‘event’ z poprawną nazwą elementu oraz nie zawiera oświadczenia ‘nonce’. Zwróć uwagę, że zalecany jest również krótki czas wygaśnięcia (np. 2 minuty).	2

V10.6 Dostawca OpenID

Ponieważ dostawcy OpenID działają jako serwery autoryzacji OAuth, zastosowanie mają również wymagania z sekcji „Serwer autoryzacji OAuth”.

Zwróć uwagę, że przy stosowaniu przepływu tokena ID (a nie przepływu kodu) tokeny dostępu nie są wystawiane i wiele wymagań dla serwera autoryzacji OAuth nie ma zastosowania.

#	Opis	Poziom
10.6.1	Zweryfikuj, że dostawca OpenID zezwala wyłącznie na wartości 'code', 'ciba', 'id_token' lub 'id_token code' dla trybu odpowiedzi. Zwróć uwagę, że 'code' jest preferowane względem 'id_token code' (przepływ hybrydowy OIDC), a 'token' (jakikolwiek przepływ Implicit) nie może być używany.	2
10.6.2	Zweryfikuj, że dostawca OpenID ogranicza ryzyko odmowy usługi poprzez wymuszone wylogowanie – uzyskując jawne potwierdzenie od użytkownika końcowego lub, jeśli występują, walidując parametry żądania wylogowania (zainicjowanego przez stronę ufającą), takie jak 'id_token_hint'.	2

V10.7 Zarządzanie zgodami

Te wymagania obejmują weryfikację zgody użytkownika przez serwer autoryzacji. Bez właściwej weryfikacji zgody użytkownika złośliwy aktor może uzyskać uprawnienia w imieniu użytkownika poprzez podszywanie się lub socjotechnikę.

#	Opis	Poziom
10.7.1	Zweryfikuj, że serwer autoryzacji zapewnia, iż użytkownik wyraża zgodę na każde żądanie autoryzacji. Jeśli tożsamość klienta nie można zapewnić, serwer autoryzacji musi zawsze jawnie prosić użytkownika o zgodę.	2
10.7.2	Zweryfikuj, że gdy serwer autoryzacji prosi użytkownika o zgodę, przedstawia wystarczające i jasne informacje o tym, czego zgoda dotyczy. Tam, gdzie ma to zastosowanie, powinno to obejmować charakter żądanych autoryzacji (zwykle na podstawie zakresu, serwera zasobów, szczegółów autoryzacji Rich Authorization Requests (RAR)), tożsamość autoryzowanej aplikacji oraz czas życia tych autoryzacji.	2
10.7.3	Zweryfikuj, że użytkownik może przeglądać, modyfikować i odwoływać zgody, których udzielił za pośrednictwem serwera autoryzacji.	2

Źródła

Więcej informacji o OAuth można znaleźć pod adresami:

- oauth.net
- OWASP OAuth 2.0 Protocol Cheat Sheet

Dla wymagań ASVS związanych z OAuth wykorzystywane są następujące RFC – opublikowane oraz w statusie draft:

- RFC6749 The OAuth 2.0 Authorization Framework
- RFC6750 The OAuth 2.0 Authorization Framework: Bearer Token Usage
- RFC6819 OAuth 2.0 Threat Model and Security Considerations
- RFC7636 Proof Key for Code Exchange by OAuth Public Clients
- RFC7591 OAuth 2.0 Dynamic Client Registration Protocol
- RFC8628 OAuth 2.0 Device Authorization Grant
- RFC8707 Resource Indicators for OAuth 2.0
- RFC9068 JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens
- RFC9126 OAuth 2.0 Pushed Authorization Requests
- RFC9207 OAuth 2.0 Authorization Server Issuer Identification
- RFC9396 OAuth 2.0 Rich Authorization Requests
- RFC9449 OAuth 2.0 Demonstrating Proof of Possession (DPoP)
- RFC9700 Best Current Practice for OAuth 2.0 Security
- draft OAuth 2.0 for Browser-Based Applications
- draft The OAuth 2.1 Authorization Framework

Więcej informacji o OpenID Connect można znaleźć pod adresami:

- OpenID Connect Core 1.0
- FAPI 2.0 Security Profile

V11 Kryptografia

Cel kontrolny

Celem tego rozdziału jest zdefiniowanie najlepszych praktyk ogólnego stosowania kryptografii, a także zaszczepienie fundamentalnego zrozumienia zasad kryptograficznych i zainspirowanie zwrotu ku bardziej odpornym i nowoczesnym podejściom. Rozdział zachęca do:

- Wdrażania solidnych systemów kryptograficznych, które zawodzą w sposób bezpieczny, adaptując się do ewoluujących zagrożeń i są przygotowane na przyszłość.
- Stosowania mechanizmów kryptograficznych, które są zarówno bezpieczne, jak i zgodne z najlepszymi praktykami branżowymi.
- Utrzymywania bezpiecznego systemu zarządzania kluczami kryptograficznymi z odpowiednią kontrolą dostępu i audytem.
- Regularnej oceny krajobrazu kryptograficznego w celu analizy nowych ryzyk i odpowiedniego dostosowywania algorytmów.
- Wykrywania i zarządzania przypadkami użycia kryptografii w całym cyklu życia aplikacji, aby wszystkie zasoby kryptograficzne były zewidencjonowane i zabezpieczone.

Oprócz przedstawienia ogólnych zasad i najlepszych praktyk dokument zawiera również bardziej szczegółowe informacje techniczne o wymaganiach w Załączniku C –Standardy kryptograficzne. Obejmuje to algorytmy i tryby uznawane za „zatwierdzone” na potrzeby wymagań tego rozdziału.

Wymagania wykorzystujące kryptografię do rozwiązania odrębnego problemu –takiego jak zarządzanie sekretami czy bezpieczeństwo komunikacji –znajdują się w innych częściach standardu.

V11.1 Inwentarz kryptograficzny i dokumentacja

Aplikacje muszą być projektowane z silną architekturą kryptograficzną, chroniącą zasoby danych stosownie do ich klasyfikacji. Szyfrowanie wszystkiego jest marnotrawstwem; nieszyfrowanie niczego to prawne zaniedbanie. Należy znaleźć równowagę –zwykle na etapie projektowania architektury lub projektu wysokopoziomowego, sprintów projektowych albo tzw. architectural spikes. Projektowanie kryptografii „na bieżąco” lub jej późniejsze doszywanie nieuchronnie będzie kosztować znacznie więcej niż wbudowanie jej od samego początku.

Ważne jest zapewnienie, że wszystkie zasoby kryptograficzne są regularnie wykrywane, inwentaryzowane i oceniane. Więcej informacji o tym, jak można to zrobić, znajduje się w załączniku.

Krytyczna jest również potrzeba zabezpieczenia systemów kryptograficznych na przyszłość –wobec nadciągającej ery komputerów kwantowych. Kryptografia postkwantowa (Post-Quantum Cryptography, PQC) odnosi się do algorytmów kryptograficznych zaprojektowanych tak, aby pozostały bezpieczne wobec ataków komputerów kwantowych, które –jak się oczekuje –złamią powszechnie stosowane algorytmy, takie jak RSA i kryptografia krzywych eliptycznych (ECC).

Aktualne wytyczne dotyczące zweryfikowanych prymitywów i standardów PQC znajdują się w załączniku.

#	Opis	Poziom
11.1.1	Zweryfikuj, że istnieje udokumentowana polityka zarządzania kluczami kryptograficznymi oraz cykl życia klucza kryptograficznego zgodny ze standardem zarządzania kluczami, takim jak NIST SP 800-57. Powinno to obejmować zapewnienie, że klucze nie są nadmiernie współdzielone (na przykład z więcej niż dwoma podmiotami dla sekretów współdzielonych i więcej niż jednym podmiotem dla kluczy prywatnych).	2
11.1.2	Zweryfikuj, że inwentarz kryptograficzny jest wykonywany, utrzymywany, regularnie aktualizowany i obejmuje wszystkie klucze kryptograficzne, algorytmy oraz certyfikaty używane przez aplikację. Musi on również dokumentować, gdzie klucze mogą i nie mogą być używane w systemie, oraz typy danych, które mogą i nie mogą być chronione tymi kluczami.	2

#	Opis	Poziom
11.1.3	Zweryfikuj, że stosowane są mechanizmy wykrywania kryptografii, identyfikujące wszystkie wystąpienia kryptografii w systemie, w tym operacje szyfrowania, haszowania i podpisywania.	3
11.1.4	Zweryfikuj, że utrzymywany jest inwentarz kryptograficzny. Musi on obejmować udokumentowany plan określający ścieżkę migracji do nowych standardów kryptograficznych, takich jak kryptografia postkwantowa, aby móc reagować na przyszłe zagrożenia.	3

V11.2 Bezpieczna implementacja kryptografii

Ta sekcja definiuje wymagania dotyczące wyboru, implementacji i bieżącego zarządzania podstawowymi algorytmami kryptograficznymi aplikacji. Celem jest zapewnienie, że wdrażane są wyłącznie solidne, akceptowane branżowo prymitywy kryptograficzne, zgodne z aktualnymi standardami (np. NIST, ISO/IEC) i najlepszymi praktykami. Organizacje muszą zapewnić, że każdy komponent kryptograficzny jest wybierany na podstawie recenzowanych dowodów i praktycznych testów bezpieczeństwa.

#	Opis	Poziom
11.2.1	Zweryfikuj, że do operacji kryptograficznych używane są implementacje zweryfikowane branżowo (w tym biblioteki i implementacje akcelеровane sprzętowo).	2
11.2.2	Zweryfikuj, że aplikacja została zaprojektowana z zachowaniem zwinności kryptograficznej (crypto agility) – tak aby algorytmy liczb losowych, szyfrowania uwierzytelnionego, MAC lub haszowania, długości kluczy, liczby rund, szyfry i tryby mogły być w dowolnym momencie rekonfigurowane, aktualizowane lub wymieniane, w celu ochrony przed złamaniami kryptograficznymi. Analogicznie musi istnieć możliwość wymiany kluczy i haseł oraz ponownego zaszyfrowania danych. Pozwoli to na płynne przejście na kryptografię postkwantową (PQC), gdy szeroko dostępne staną się wysokopewne implementacje zatwierdzonych schematów lub standardów PQC.	2
11.2.3	Zweryfikuj, że wszystkie prymitywy kryptograficzne zapewniają co najmniej 128 bitów bezpieczeństwa, biorąc pod uwagę algorytm, rozmiar klucza i konfigurację. Na przykład 256-bitowy klucz ECC zapewnia około 128 bitów bezpieczeństwa, podczas gdy RSA wymaga klucza 3072-bitowego, aby osiągnąć 128 bitów bezpieczeństwa.	2

#	Opis	Poziom
11.2.4	Zweryfikuj, że wszystkie operacje kryptograficzne są wykonywane w czasie stałym, bez operacji „skracających” (short-circuit) w porównaniach, obliczeniach i zwracanych wartościach, aby uniknąć wycieku informacji.	3
11.2.5	Zweryfikuj, że wszystkie moduły kryptograficzne zawodzą w sposób bezpieczny, a błędy są obsługiwane tak, aby nie umożliwiały podatności, takich jak ataki Padding Oracle.	3

V11.3 Algorytmy szyfrowania

Algorytmy szyfrowania uwierzytelnionego zbudowane na AES i CHACHA20 stanowią kręgosłup nowoczesnej praktyki kryptograficznej.

#	Opis	Poziom
11.3.1	Zweryfikuj, że nie są używane niebezpieczne tryby blokowe (np. ECB) ani słabe schematy dopełnienia (np. PKCS#1 v1.5).	1
11.3.2	Zweryfikuj, że używane są wyłącznie zatwierdzone szyfry i tryby, takie jak AES z GCM.	1
11.3.3	Zweryfikuj, że zaszyfrowane dane są chronione przed nieautoryzowaną modyfikacją — najlepiej z użyciem zatwierdzonej metody szyfrowania uwierzytelnionego albo poprzez połączenie zatwierdzonej metody szyfrowania z zatwierdzonym algorytmem MAC.	2
11.3.4	Zweryfikuj, że wartości nonce, wektory inicjujące i inne liczby jednorazowe nie są używane dla więcej niż jednej pary klucz szyfrujący–element danych. Metoda ich generowania musi być odpowiednia dla stosowanego algorytmu.	3
11.3.5	Zweryfikuj, że każda kombinacja algorytmu szyfrowania i algorytmu MAC działa w trybie encrypt-then-MAC.	3

V11.4 Haszowanie i funkcje oparte na skrótach

Skróty kryptograficzne są wykorzystywane w bardzo wielu protokołach kryptograficznych, takich jak podpisy cyfrowe, HMAC, funkcje wyprowadzania klucza (KDF), generowanie losowych bitów oraz przechowywanie haseł. Bezpieczeństwo systemu kryptograficznego jest tylko tak silne, jak leżące u jego podstaw funkcje skrótu. Ta sekcja przedstawia wymagania dotyczące stosowania bezpiecznych funkcji skrótu w operacjach kryptograficznych.

W kwestii przechowywania haseł –obok załącznika kryptograficznego –użyteczny kontekst i wskazówki zapewnia również OWASP Password Storage Cheat Sheet.

#	Opis	Poziom
11.4.1	Zweryfikuj, że dla ogólnych kryptograficznych przypadków użycia –w tym podpisów cyfrowych, HMAC, KDF i generowania losowych bitów –używane są wyłącznie zatwierdzone funkcje skrótu. Niedozwolone funkcje skrótu, takie jak MD5, nie mogą być używane do żadnych celów kryptograficznych.	1
11.4.2	Zweryfikuj, że hasła są przechowywane z użyciem zatwierdzonej, wymagającej obliczeniowo funkcji wyprowadzania klucza (znanej też jako „funkcja haszowania haseł”), z parametrami skonfigurowanymi zgodnie z aktualnymi wytycznymi. Ustawienia powinny równoważyć bezpieczeństwo i wydajność tak, aby ataki siłowe były wystarczająco trudne dla wymaganego poziomu bezpieczeństwa.	2
11.4.3	Zweryfikuj, że funkcje skrótu używane w podpisach cyfrowych, jako element uwierzytelniania danych lub ich integralności, są odporne na kolizje i mają odpowiednie długości bitowe. Jeśli wymagana jest odporność na kolizje, długość wyjścia musi wynosić co najmniej 256 bitów. Jeśli wymagana jest wyłącznie odporność na ataki drugiego przeciwobrazu (second pre-image), długość wyjścia musi wynosić co najmniej 128 bitów.	2
11.4.4	Zweryfikuj, że aplikacja używa zatwierdzonych funkcji wyprowadzania klucza z parametrami rozciągania klucza (key stretching) przy wyprowadzaniu kluczy tajnych z haseł. Stosowane parametry muszą równoważyć bezpieczeństwo i wydajność, aby ataki siłowe nie mogły skompromitować wynikowego klucza kryptograficznego.	2

V11.5 Wartości losowe

Kryptograficznie bezpieczne generowanie liczb pseudolosowych (CSPRNG) jest niezwykle trudne do poprawnego zrealizowania. Zasadniczo dobre źródła entropii w systemie szybko się wyczerpują przy nadużywaniu, natomiast źródła o mniejszej losowości mogą prowadzić do przewidywalnych kluczy i sekretów.

#	Opis	Poziom
11.5.1	Zweryfikuj, że wszystkie liczby losowe i ciągi znaków, które mają być nieodgadnywalne, są generowane przy użyciu kryptograficznie bezpiecznego generatora liczb pseudolosowych (CSPRNG) i mają entropię co najmniej 128 bitów. Zwróć uwagę, że UUID-y nie spełniają tego warunku.	2

#	Opis	Poziom
11.5.2	Zweryfikuj, że stosowany mechanizm generowania liczb losowych został zaprojektowany do bezpiecznego działania nawet pod dużym obciążeniem.	3

V11.6 Kryptografia klucza publicznego

Kryptografia klucza publicznego znajduje zastosowanie tam, gdzie współdzielenie klucza tajnego między wieloma stronami nie jest możliwe lub pożądane.

W ramach tego istnieje potrzeba stosowania zatwierdzonych mechanizmów wymiany kluczy, takich jak Diffie-Hellman i Elliptic Curve Diffie-Hellman (ECDH), aby kryptosystem pozostał bezpieczny wobec współczesnych zagrożeń. Rozdział „Bezpieczna komunikacja” zawiera wymagania dotyczące TLS, dlatego wymagania tej sekcji są przeznaczone dla sytuacji, w których kryptografia klucza publicznego jest wykorzystywana w przypadkach użycia innych niż TLS.

#	Opis	Poziom
11.6.1	Zweryfikuj, że do generowania kluczy i zasilania generatora (seeding) oraz do generowania i weryfikacji podpisów cyfrowych używane są wyłącznie zatwierdzone algorytmy kryptograficzne i tryby działania. Algorytmy generowania kluczy nie mogą generować kluczy niebezpiecznych, podatnych na znane ataki – na przykład kluczy RSA podatnych na faktoryzację Fermata.	2
11.6.2	Zweryfikuj, że do wymiany kluczy używane są zatwierdzone algorytmy kryptograficzne (takie jak Diffie-Hellman), ze szczególnym naciskiem na to, aby mechanizmy wymiany kluczy używały bezpiecznych parametrów. Zapobieganie to atakom na proces ustanawiania klucza, które mogłyby prowadzić do ataków typu adversary-in-the-middle lub złamań kryptograficznych.	3

V11.7 Kryptografia danych w użyciu

Ochrona danych podczas ich przetwarzania jest sprawą nadrzędną. Zalecane są techniki takie jak pełne szyfrowanie pamięci, szyfrowanie danych w tranzycie oraz zapewnienie, że dane są szyfrowane możliwie najszybciej po użyciu.

#	Opis	Poziom
11.7.1	Zweryfikuj, że stosowane jest pełne szyfrowanie pamięci, chroniące dane wrażliwe w trakcie użycia i uniemożliwiające dostęp nieuprawnionym użytkownikom lub procesom.	3
11.7.2	Zweryfikuj, że minimalizacja danych zapewnia, iż podczas przetwarzania ekspozowana jest minimalna ilość danych, oraz upewnij się, że dane są szyfrowane natychmiast po użyciu lub najszybciej, jak to wykonalne.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Testing for Weak Cryptography
- OWASP Cryptographic Storage Cheat Sheet
- FIPS 140-3
- NIST SP 800-57

V12 Bezpieczna komunikacja

Cel kontrolny

Niniejszy rozdział zawiera wymagania dotyczące konkretnych mechanizmów, które powinny chronić dane w transzycie —zarówno między klientem użytkownika końcowego a usługą backendową, jak i między usługami wewnętrznymi i backendowymi.

Ogólne koncepcje promowane przez ten rozdział obejmują:

- Zapewnienie, że komunikacja jest szyfrowana na zewnątrz, a najlepiej również wewnętrznie.
- Konfigurowanie mechanizmów szyfrowania zgodnie z najnowszymi wytycznymi, w tym preferowanymi algorytmami i szyframi.
- Zapewnienie, że komunikacja nie jest przechwytywana przez strony nieuprawnione, poprzez stosowanie podpisanych certyfikatów.

Oprócz przedstawienia ogólnych zasad i najlepszych praktyk ASVS dostarcza również bardziej szczegółowych informacji technicznych o sile kryptograficznej w Załączniku C —Standardy kryptograficzne.

V12.1 Ogólne wytyczne bezpieczeństwa TLS

Ta sekcja zawiera wstępne wskazówki dotyczące zabezpieczania komunikacji TLS. Do bieżącego przeglądu konfiguracji TLS należy używać aktualnych narzędzi.

Choć stosowanie certyfikatów TLS typu wildcard nie jest samo w sobie niebezpieczne, kompromitacja certyfikatu wdrożonego we wszystkich posiadanych środowiskach (np. produkcyjnym, staging, deweloperskim i testowym) może prowadzić do kompromitacji poziomu bezpieczeństwa aplikacji, które go używają. Jeśli to możliwe, należy stosować właściwą ochronę, zarządzanie oraz odrębne certyfikaty TLS w różnych środowiskach.

#	Opis	Poziom
12.1.1	Zweryfikuj, że włączone są wyłącznie najnowsze zalecane wersje protokołu TLS, takie jak TLS 1.2 i TLS 1.3. Najnowsza wersja protokołu TLS musi być opcją preferowaną.	1
12.1.2	Zweryfikuj, że włączone są wyłącznie zalecane zestawy szyfrów (cipher suites), z najsilniejszymi zestawami ustawionymi jako preferowane. Aplikacje L3 mogą wspierać wyłącznie zestawy szyfrów zapewniające utajnianie z wyprzedzeniem (forward secrecy).	2
12.1.3	Zweryfikuj, że aplikacja waliduje, iż certyfikaty klienckie mTLS są zaufane, zanim użyje tożsamości z certyfikatu do uwierzytelniania lub autoryzacji.	2
12.1.4	Zweryfikuj, że włączone i skonfigurowane jest właściwe odwoływanie certyfikatów, takie jak Online Certificate Status Protocol (OCSP) Stapling.	3
12.1.5	Zweryfikuj, że w ustawieniach TLS aplikacji włączone jest Encrypted Client Hello (ECH), aby zapobiec ujawnieniu wrażliwych metadanych, takich jak Server Name Indication (SNI), podczas procesu uzgadniania TLS.	3

V12.2 Komunikacja HTTPS z usługami skierowanymi na zewnątrz

Zapewnij, że cały ruch HTTP do usług skierowanych na zewnątrz, które udostępnia aplikacja, jest wysyłany w postaci zaszyfrowanej, z użyciem publicznie zaufanych certyfikatów.

#	Opis	Poziom
12.2.1	Zweryfikuj, że TLS jest używany dla całej łączności między klientem a skierowanymi na zewnątrz usługami opartymi na HTTP i nie następuje powrót do komunikacji niebezpiecznej lub nieszyfrowanej.	1
12.2.2	Zweryfikuj, że usługi skierowane na zewnątrz używają publicznie zaufanych certyfikatów TLS.	1

V12.3 Ogólne bezpieczeństwo komunikacji między usługami

Komunikacja serwerowa (zarówno wewnętrzna, jak i zewnętrzna) to więcej niż tylko HTTP. Połączenia do i z innych systemów również muszą być bezpieczne —najlepiej z użyciem TLS.

#	Opis	Poziom
12.3.1	Zweryfikuj, że szyfrowany protokół, taki jak TLS, jest używany dla wszystkich połączeń przychodzących i wychodzących do i z aplikacji, w tym systemów monitorowania, narzędzi zarządzania, zdalnego dostępu i SSH, oprogramowania pośredniczącego, baz danych, mainframe'ów, systemów partnerskich lub zewnętrznych API. Serwer nie może wracać do protokołów niebezpiecznych lub nieszyfrowanych.	2
12.3.2	Zweryfikuj, że klienci TLS walidują otrzymane certyfikaty przed rozpoczęciem komunikacji z serwerem TLS.	2
12.3.3	Zweryfikuj, że TLS lub inny odpowiedni mechanizm szyfrowania transportu jest używany dla całej łączności między wewnętrznymi usługami opartymi na HTTP w ramach aplikacji i nie następuje powrót do komunikacji niebezpiecznej lub nieszyfrowanej.	2
12.3.4	Zweryfikuj, że połączenia TLS między usługami wewnętrznymi używają zaufanych certyfikatów. Tam, gdzie stosowane są certyfikaty generowane wewnętrznie lub samopodpisane, usługa konsumująca musi być skonfigurowana tak, aby ufać wyłącznie konkretnym wewnętrznym urządzeniom certyfikacji (CA) i konkretnym certyfikatom samopodpisany.	2
12.3.5	Zweryfikuj, że usługi komunikujące się wewnętrznie w ramach systemu (komunikacja intra-service) używają silnego uwierzytelniania, aby każdy endpoint był zweryfikowany. Muszą być stosowane silne metody uwierzytelniania, takie jak uwierzytelnianie klienta TLS, zapewniające tożsamość z użyciem infrastruktury klucza publicznego i mechanizmów odpornych na ataki powtórzeniowe. Dla architektur mikroserwisowych rozważ użycie service mesh w celu uproszczenia zarządzania certyfikatami i podniesienia poziomu bezpieczeństwa.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP - Transport Layer Security Cheat Sheet
- Przewodnik Mozilli po konfiguracji TLS po stronie serwera
- Narzędzie Mozilli do generowania sprawdzonych konfiguracji TLS.

- O-Saft —projekt OWASP do walidacji konfiguracji TLS

V13 Konfiguracja

Cel kontrolny

Domyślna konfiguracja aplikacji musi być bezpieczna do użytku w Internecie.

Niniejszy rozdział zawiera wytyczne dotyczące różnych konfiguracji niezbędnych do osiągnięcia tego celu, w tym stosowanych na etapie wytwarzania, budowania i wdrażania.

Omawiane tematy obejmują zapobieganie wyciekom danych, bezpieczne zarządzanie komunikacją między komponentami oraz ochronę sekretów.

V13.1 Dokumentacja konfiguracji

Ta sekcja przedstawia wymagania dokumentacyjne dotyczące sposobu, w jaki aplikacja komunikuje się z usługami wewnętrznymi i zewnętrznymi, a także technik zapobiegania utracie dostępności wskutek niedostępności usług. Obejmuje również dokumentację związaną z sekretami.

#	Opis	Poziom
13.1.1	Zweryfikuj, że wszystkie potrzeby komunikacyjne aplikacji są udokumentowane. Musi to obejmować usługi zewnętrzne, na których polega aplikacja, oraz przypadki, w których użytkownik końcowy może wskazać zewnętrzną lokalizację, z którą aplikacja następnie się połączy.	2
13.1.2	Zweryfikuj, że dla każdej usługi, z której korzysta aplikacja, dokumentacja definiuje maksymalną liczbę równoczesnych połączeń (np. limity puli połączeń) oraz zachowanie aplikacji po osiągnięciu tego limitu, w tym wszelkie mechanizmy awaryjne lub naprawcze, aby zapobiec warunkom odmowy usługi.	3
13.1.3	Zweryfikuj, że dokumentacja aplikacji definiuje strategie zarządzania zasobami dla każdego zewnętrznego systemu lub usługi, z których korzysta (np. bazy danych, uchwyt plików, wątki, połączenia HTTP). Powinno to obejmować procedury zwalniania zasobów, ustawienia limitów czasu, obsługę awarii oraz —tam, gdzie zaimplementowano logikę ponawiania — określenie limitów ponowień, opóźnień i algorytmów back-off. Dla synchronicznych operacji żądanie-odpowiedź HTTP powinna nakazywać krótkie limity czasu oraz wyłączenie ponowień albo ich ścisłe ograniczenie, aby zapobiec kaskadowym opóźnieniom i wyczerpaniu zasobów.	3

#	Opis	Poziom
13.1.4	Zweryfikuj, że dokumentacja aplikacji definiuje sekrety krytyczne dla bezpieczeństwa aplikacji oraz harmonogram ich rotacji, na podstawie modelu zagrożeń organizacji i wymagań biznesowych.	3

V13.2 Konfiguracja komunikacji backendowej

Aplikacje współpracują z wieloma usługami, w tym API, bazami danych i innymi komponentami. Mogą one być uznawane za wewnętrzne dla aplikacji, lecz nieobjęte jej standardowymi mechanizmami kontroli dostępu, albo mogą być całkowicie zewnętrzne. W obu przypadkach konieczne jest skonfigurowanie aplikacji do bezpiecznej współpracy z tymi komponentami oraz —jeśli to wymagane— ochrona tej konfiguracji.

Uwaga: rozdział „Bezpieczna komunikacja” zawiera wytyczne dotyczące szyfrowania w tranzycie.

#	Opis	Poziom
13.2.1	Zweryfikuj, że komunikacja między backendowymi komponentami aplikacji, które nie wspierają standardowego mechanizmu sesji użytkownika aplikacji —w tym API, oprogramowaniem pośredniczącym i warstwami danych —jest uwierzytelniana. Uwierzytelnianie musi wykorzystywać indywidualne konta usługowe, tokeny krótkoterminowe lub uwierzytelnianie oparte na certyfikatach, a nie niezmiennie poświadczenia, takie jak hasła, klucze API czy konta współdzielone z dostępem uprzywilejowanym.	2
13.2.2	Zweryfikuj, że komunikacja między backendowymi komponentami aplikacji —w tym lokalnymi usługami lub usługami systemu operacyjnego, API, oprogramowaniem pośredniczącym i warstwami danych —odbywa się z użyciem kont o najmniejszych niezbędnych uprawnieniach.	2
13.2.3	Zweryfikuj, że jeśli do uwierzytelniania usługi musi być użyte poświadczenie, poświadczenie używane przez konsumenta nie jest poświadczeniem domyślnym (np. root/root lub admin/admin).	2
13.2.4	Zweryfikuj, że lista dozwolonych jest używana do zdefiniowania zewnętrznych zasobów lub systemów, z którymi aplikacja może się komunikować (np. dla żądań wychodzących, ładowania danych lub dostępu do plików). Lista ta może być zaimplementowana w warstwie aplikacji, na serwerze WWW, na zaporze albo jako kombinacja różnych warstw.	2
13.2.5	Zweryfikuj, że serwer WWW lub serwer aplikacyjny jest skonfigurowany z listą dozwolonych zasobów lub systemów, do których serwer może wysyłać żądania albo z których może łączyć dane lub pliki.	2

#	Opis	Poziom
13.2.6	Zweryfikuj, że tam, gdzie aplikacja łączy się z odrębnymi usługami, przestrzega udokumentowanej konfiguracji dla każdego połączenia – takiej jak maksymalna liczba połączeń równoległych, zachowanie po osiągnięciu maksymalnej dozwolonej liczby połączeń, limity czasu połączeń i strategie ponawiania.	3

V13.3 Zarządzanie sekretami

Zarządzanie sekretami to kluczowe zadanie konfiguracyjne zapewniające ochronę danych używanych w aplikacji. Szczegółowe wymagania dotyczące kryptografii znajdują się w rozdziale „Kryptografia”, natomiast ta sekcja koncentruje się na aspektach zarządzania sekretami i ich obsługi.

#	Opis	Poziom
13.3.1	Zweryfikuj, że do bezpiecznego tworzenia, przechowywania, kontroli dostępu i niszczenia sekretów backendowych używane jest rozwiązanie do zarządzania sekretami, takie jak skarbiec kluczy (key vault). Sekrety mogą obejmować hasła, materiał klucza, integracje z bazami danych i systemami stron trzecich, klucze i ziarna tokenów opartych na czasie, inne sekrety wewnętrzne oraz klucze API. Sekrety nie mogą znajdować się w kodzie źródłowym aplikacji ani w artefaktach builda. Dla aplikacji L3 musi to być rozwiązanie wsparte sprzętowo, takie jak HSM.	2
13.3.2	Zweryfikuj, że dostęp do zasobów sekretnych jest zgodny z zasadą najmniejszych uprawnień.	2
13.3.3	Zweryfikuj, że wszystkie operacje kryptograficzne są wykonywane z użyciem izolowanego modułu bezpieczeństwa (takiego jak skarbiec lub sprzętowy moduł bezpieczeństwa), aby bezpiecznie zarządzać materiałem klucza i chronić go przed ekspozycją poza modułem bezpieczeństwa.	3
13.3.4	Zweryfikuj, że sekrety są skonfigurowane tak, aby wygasły i podlegały rotacji zgodnie z dokumentacją aplikacji.	3

V13.4 Niezamierzony wyciek informacji

Konfiguracje produkcyjne powinny być utwardzone tak, aby nie ujawniać zbędnych danych. Wiele z tych problemów rzadko jest oceniane jako istotne ryzyka, ale często są one łączone w łańcuchy z

innymi podatnościami. Jeśli problemy te nie występują domyślnie, poprzeczka dla ataku na aplikację zostaje podniesiona.

Przykładowo ukrycie wersji komponentów serwerowych nie eliminuje potrzeby łatania wszystkich komponentów, a wyłączenie listowania katalogów nie usuwa potrzeby stosowania kontroli autoryzacji ani trzymania plików poza folderem publicznym —ale podnosi poprzeczkę.

#	Opis	Poziom
13.4.1	Zweryfikuj, że aplikacja jest wdrażana albo bez jakichkolwiek metadanych kontroli wersji, w tym folderów .git lub .svn, albo w sposób uniemożliwiający dostęp do tych folderów zarówno z zewnątrz, jak i samej aplikacji.	1
13.4.2	Zweryfikuj, że tryby debugowania są wyłączone dla wszystkich komponentów w środowiskach produkcyjnych, aby zapobiec ekspozycji funkcji debugowania i wyciekowi informacji.	2
13.4.3	Zweryfikuj, że serwery WWW nie udostępniają klientom listowania katalogów, chyba że jest to wyraźnie zamierzone.	2
13.4.4	Zweryfikuj, że metoda HTTP TRACE nie jest wspierana w środowiskach produkcyjnych, aby uniknąć potencjalnego wycieku informacji.	2
13.4.5	Zweryfikuj, że dokumentacja (np. wewnętrznych API) oraz endpointy monitorowania nie są eksponowane, chyba że jest to wyraźnie zamierzone.	2
13.4.6	Zweryfikuj, że aplikacja nie ujawnia szczegółowych informacji o wersjach komponentów backendowych.	3
13.4.7	Zweryfikuj, że warstwa webowa jest skonfigurowana tak, aby serwować wyłącznie pliki o określonych rozszerzeniach, aby zapobiec niezamierzonemu wyciekowi informacji, konfiguracji i kodu źródłowego.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 Ochrona danych

Cel kontrolny

Aplikacje nie są w stanie przewidzieć wszystkich wzorców użycia i zachowań użytkowników, dlatego powinny wdrażać mechanizmy ograniczające nieautoryzowany dostęp do danych wrażliwych na urządzeniach klienckich.

Niniejszy rozdział zawiera wymagania związane z określeniem, jakie dane wymagają ochrony, jak należy je chronić, oraz konkretne mechanizmy do wdrożenia lub pułapki, których należy unikać.

Kolejną kwestią ochrony danych jest masowa ekstrakcja, modyfikacja lub nadmierne użycie. Wymagania każdego systemu będą zapewne bardzo różne, dlatego ustalenie, co jest „nienormalne”, musi uwzględniać model zagrożeń i ryzyko biznesowe. Z perspektywy ASVS wykrywanie tych problemów jest omawiane w rozdziale „Logowanie zdarzeń bezpieczeństwa i obsługa błędów”, a ustalanie limitów — w rozdziale „Walidacja i logika biznesowa”.

V14.1 Dokumentacja ochrony danych

Kluczowym warunkiem wstępnym zdolności do ochrony danych jest skategoryzowanie, które dane należy uznać za wrażliwe. Prawdopodobnie wystąpi kilka różnych poziomów wrażliwości, a dla każdego z nich mechanizmy wymagane do ochrony danych będą inne.

Istnieją różne regulacje i przepisy dotyczące prywatności, które wpływają na to, jak aplikacje muszą podchodzić do przechowywania, wykorzystywania i przysyłania wrażliwych danych osobowych. Ta sekcja nie próbuje już powielać tego typu przepisów o ochronie danych czy prywatności, lecz koncentruje się na kluczowych kwestiach technicznych ochrony danych wrażliwych. Należy zapoznać się z lokalnymi przepisami i regulacjami oraz — w razie potrzeby — skonsultować się z wykwalifikowanym specjalistą ds. prywatności lub prawnikiem.

#	Opis	Poziom
14.1.1	Zweryfikuj, że wszystkie dane wrażliwe tworzone i przetwarzane przez aplikację zostały zidentyfikowane i sklasyfikowane według poziomów ochrony. Obejmuje to dane, które są jedynie zakodowane i przez to łatwe do zdekodowania, takie jak ciągi Base64 lub jawny ładunek wewnątrz JWT. Poziomy ochrony muszą uwzględniać wszelkie regulacje i standardy ochrony danych i prywatności, z którymi aplikacja musi być zgodna.	2
14.1.2	Zweryfikuj, że wszystkie poziomy ochrony danych wrażliwych mają udokumentowany zestaw wymagań ochronnych. Musi on obejmować (między innymi) wymagania dotyczące ogólnego szyfrowania, weryfikacji integralności, retencji, sposobu logowania danych, kontroli dostępu do danych wrażliwych w logach, szyfrowania na poziomie bazy danych, prywatności i stosowanych technologii wzmacniających prywatność oraz inne wymagania poufności.	2

V14.2 Ogólna ochrona danych

Ta sekcja zawiera różnorodne praktyczne wymagania związane z ochroną danych. Większość dotyczy konkretnych problemów, takich jak niezamierzony wyciek danych, ale znajduje się tu również ogólne

wymaganie wdrożenia mechanizmów ochronnych stosownie do poziomu ochrony wymaganego dla każdego elementu danych.

#	Opis	Poziom
14.2.1	Zweryfikuj, że dane wrażliwe są wysyłane do serwera wyłącznie w treści komunikatu HTTP lub polach nagłówka, a URL i ciąg zapytania nie zawierają informacji wrażliwych, takich jak klucz API czy token sesji.	1
14.2.2	Zweryfikuj, że aplikacja zapobiega buforowaniu danych wrażliwych w komponentach serwerowych, takich jak moduły równoważenia obciążenia i pamięci podręczne aplikacji, albo zapewnia, że dane są bezpiecznie usuwane po użyciu.	2
14.2.3	Zweryfikuj, że zdefiniowane dane wrażliwe nie są wysyłane do stron niezaufanych (np. trackerów użytkowników), aby zapobiec niepożądanemu gromadzeniu danych poza kontrolą aplikacji.	2
14.2.4	Zweryfikuj, że mechanizmy dotyczące danych wrażliwych — związane z szyfrowaniem, weryfikacją integralności, retencją, sposobem logowania danych, kontrolą dostępu do danych wrażliwych w logach, prywatnością i technologiami wzmacniającymi prywatność — są wdrożone zgodnie z dokumentacją dla poziomu ochrony konkretnych danych.	2
14.2.5	Zweryfikuj, że mechanizmy buforowania są skonfigurowane tak, aby buforować wyłącznie odpowiedzi o oczekiwanym typie treści dla danego zasobu, niezawierające wrażliwej, dynamicznej treści. Serwer WWW powinien zwracać odpowiedź 404 lub 302 przy dostępie do nieistniejącego pliku, zamiast zwracać inny, istniejący plik. Powinno to zapobiegać atakom Web Cache Deception.	3
14.2.6	Zweryfikuj, że aplikacja zwraca wyłącznie minimalny zakres danych wrażliwych wymagany dla jej funkcjonalności — na przykład zwraca tylko część cyfr numeru karty płatniczej, a nie pełny numer. Jeśli kompletne dane są wymagane, powinny być maskowane w interfejsie użytkownika, chyba że użytkownik celowo je wyświetli.	3
14.2.7	Zweryfikuj, że informacje wrażliwe podlegają klasyfikacji retencji danych, zapewniającej automatyczne usuwanie danych przestarzałych lub zbędnych — według zdefiniowanego harmonogramu albo stosownie do sytuacji.	3
14.2.8	Zweryfikuj, że informacje wrażliwe są usuwane z metadanych plików przesyłanych przez użytkowników, chyba że użytkownik wyraził zgodę na ich przechowywanie.	3

V14.3 Ochrona danych po stronie klienta

Ta sekcja zawiera wymagania zapobiegające wyciekom danych w określony sposób po stronie klienta lub agenta użytkownika aplikacji.

#	Opis	Poziom
14.3.1	Zweryfikuj, że uwierzytelnione dane są czyszczone z pamięci klienta, takiej jak DOM przeglądarki, po zakończeniu działania klienta lub sesji. Pomocne może być pole nagłówka odpowiedzi HTTP 'Clear-Site-Data', ale strona kliencka powinna również potrafić samodzielnie wyczyścić dane, jeśli połączenie z serwerem jest niedostępne w chwili kończenia sesji.	1
14.3.2	Zweryfikuj, że aplikacja ustawia wystarczające pola nagłówka odpowiedzi HTTP zapobiegające buforowaniu (tj. Cache-Control: no-store), aby dane wrażliwe nie były buforowane w przeglądarkach.	2
14.3.3	Zweryfikuj, że dane przechowywane w pamięci przeglądarki (takiej jak localStorage, sessionStorage, IndexedDB czy ciasteczka) nie zawierają danych wrażliwych, z wyjątkiem tokenów sesji.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- Serwis Security Headers do sprawdzania pól nagłówków bezpieczeństwa i anti-caching
- Dokumentacja Mozilli o nagłówkach anti-caching
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- Przegląd unijnego ogólnego rozporządzenia o ochronie danych (RODO)
- Europejski Inspektor Ochrony Danych — Internet Privacy Engineering Network
- Informacje o nagłówku „Clear-Site-Data”
- White paper o Web Cache Deception

V15 Bezpieczne kodowanie i architektura

Cel kontrolny

Wiele wymagań ASVS dotyczy konkretnego obszaru bezpieczeństwa, takiego jak uwierzytelnianie czy autoryzacja, albo odnosi się do konkretnego typu funkcjonalności aplikacji, jak logowanie zdarzeń czy obsługa plików.

Niniejszy rozdział zawiera ogólne wymagania bezpieczeństwa, które należy uwzględnić przy projektowaniu i tworzeniu aplikacji. Wymagania te koncentrują się nie tylko na czystej architekturze i jakości kodu, lecz również na konkretnych praktykach architektonicznych i programistycznych niezbędnych dla bezpieczeństwa aplikacji.

V15.1 Dokumentacja bezpiecznego kodowania i architektury

Wiele wymagań dotyczących ustanowienia bezpiecznej i możliwej do obrony architektury zależy od jasnej dokumentacji decyzji podjętych w sprawie implementacji konkretnych mechanizmów bezpieczeństwa oraz komponentów używanych w aplikacji.

Ta sekcja przedstawia wymagania dokumentacyjne, w tym identyfikację komponentów uznawanych za zawierające „niebezpieczną funkcjonalność” lub będące „komponentami ryzykownymi”.

Komponent z „niebezpieczną funkcjonalnością” może być komponentem tworzonym wewnętrznie lub zewnętrznym, wykonującym operacje takie jak deserializacja niezauważanych danych, parsowanie surowych plików lub danych binarnych, dynamiczne wykonywanie kodu czy bezpośrednia manipulacja pamięcią. Podatności w tego typu operacjach niosą wysokie ryzyko kompromitacji aplikacji i potencjalnego wystawienia jej infrastruktury.

„Komponent ryzykowny” to biblioteka strony trzeciej (tj. nietworzona wewnętrznie) z brakującymi lub słabo zaimplementowanymi mechanizmami bezpieczeństwa w procesach jej rozwoju lub funkcjonalności. Przykłady obejmują komponenty słabo utrzymywane, niewspierane, będące u kresu cyklu życia lub mające historię istotnych podatności.

Ta sekcja podkreśla również wagę zdefiniowania odpowiednich ram czasowych usuwania podatności w komponentach stron trzecich.

#	Opis	Poziom
15.1.1	Zweryfikuj, że dokumentacja aplikacji definiuje oparte na ryzyku ramy czasowe naprawy dla wersji komponentów stron trzecich z podatnościami oraz ogólnie dla aktualizacji bibliotek, aby zminimalizować ryzyko związane z tymi komponentami.	1
15.1.2	Zweryfikuj, że utrzymywany jest katalog inwentaryzacyjny, taki jak software bill of materials (SBOM), obejmujący wszystkie używane biblioteki stron trzecich, w tym weryfikację, że komponenty pochodzą ze wstępnie zdefiniowanych, zaufanych i stale utrzymywanych repozytoriów.	2

#	Opis	Poziom
15.1.3	Zweryfikuj, że dokumentacja aplikacji identyfikuje funkcjonalność czasochłonną lub zasobożerną. Musi to obejmować sposób zapobiegania utracie dostępności wskutek nadużywania tej funkcjonalności oraz unikania sytuacji, w której budowanie odpowiedzi trwa dłużej niż limit czasu konsumenta. Potencjalne obrony mogą obejmować przetwarzanie asynchroniczne, użycie kolejek oraz ograniczanie procesów równoległych dla każdego użytkownika i każdej aplikacji.	2
15.1.4	Zweryfikuj, że dokumentacja aplikacji wskazuje biblioteki stron trzecich uznawane za „komponenty ryzykowne”.	3
15.1.5	Zweryfikuj, że dokumentacja aplikacji wskazuje części aplikacji, w których używana jest „niebezpieczna funkcjonalność”.	3

V15.2 Architektura bezpieczeństwa i zależności

Ta sekcja zawiera wymagania dotyczące postępowania z ryzykownymi, przestarzałymi lub niebezpiecznymi zależnościami i komponentami poprzez zarządzanie zależnościami.

Obejmuje również stosowanie technik na poziomie architektury —takich jak sandboxing, enkapsulacja, konteneryzacja i izolacja sieciowa —w celu ograniczenia skutków używania „niebezpiecznych operacji” lub „komponentów ryzykownych” (zdefiniowanych w poprzedniej sekcji) oraz zapobiegania utracie dostępności wskutek nadużywania funkcjonalności zasobożernej.

#	Opis	Poziom
15.2.1	Zweryfikuj, że aplikacja zawiera wyłącznie komponenty, które nie przekroczyły udokumentowanych ram czasowych aktualizacji i naprawy.	1
15.2.2	Zweryfikuj, że aplikacja wdrożyła obrony przed utratą dostępności wskutek funkcjonalności czasochłonnej lub zasobożernej, zgodnie z udokumentowanymi decyzjami bezpieczeństwa i strategiami w tym zakresie.	2
15.2.3	Zweryfikuj, że środowisko produkcyjne zawiera wyłącznie funkcjonalność wymaganą do działania aplikacji i nie eksponuje zbędnej funkcjonalności, takiej jak kod testowy, przykładowe fragmenty czy funkcjonalność deweloperska.	2
15.2.4	Zweryfikuj, że komponenty stron trzecich i wszystkie ich zależności przechodnie pochodzą z oczekiwanego repozytorium —wewnętrznego lub zewnętrznego —oraz że nie istnieje ryzyko ataku dependency confusion.	3

#	Opis	Poziom
15.2.5	Zweryfikuj, że aplikacja wdraża dodatkowe zabezpieczenia wokół części aplikacji udokumentowanych jako zawierające „niebezpieczną funkcjonalność” lub używające bibliotek stron trzecich uznawanych za „komponenty ryzykowne”. Może to obejmować techniki takie jak sandboxing, enkapsulacja, konteneryzacja lub izolacja na poziomie sieci, aby opóźnić i zniechęcić atakujących, którzy skompromitowali jedną część aplikacji, przed przemieszczaniem się (pivoting) w inne jej miejsca.	3

V15.3 Kodowanie defensywne

Ta sekcja obejmuje typy podatności – w tym type juggling, prototype pollution i inne – wynikające ze stosowania niebezpiecznych wzorców kodowania w danym języku. Część z nich może nie dotyczyć wszystkich języków, inne będą miały poprawki specyficzne dla języka albo będą związane ze sposobem, w jaki dany język lub framework obsługuje funkcję taką jak parametry HTTP. Sekcja uwzględnia również ryzyko braku kryptograficznej walidacji aktualizacji aplikacji.

Rozważa również ryzyka związane z używaniem obiektów do reprezentowania elementów danych oraz przyjmowaniem i zwracaniem ich przez zewnętrzne API. W takim przypadku aplikacja musi zapewnić, że pola danych, które nie powinny być zapisywalne, nie są modyfikowane danymi od użytkownika (mass assignment), a API selektywnie dobiera zwracane pola danych. Tam, gdzie dostęp do pól zależy od uprawnień użytkownika, należy to rozważać w kontekście wymagania kontroli dostępu na poziomie pól z rozdziału „Autoryzacja”.

#	Opis	Poziom
15.3.1	Zweryfikuj, że aplikacja zwraca wyłącznie wymagany podzbiór pól obiektu danych. Na przykład nie powinna zwracać całego obiektu danych, ponieważ niektóre pojedyncze pola nie powinny być dostępne dla użytkowników.	1
15.3.2	Zweryfikuj, że tam, gdzie backend aplikacji wykonuje wywołania do zewnętrznych URL-i, jest skonfigurowany tak, aby nie podążać za przekierowaniami, chyba że jest to funkcjonalność zamierzona.	2
15.3.3	Zweryfikuj, że aplikacja posiada środki zaradcze chroniące przed atakami mass assignment poprzez ograniczanie dozwolonych pól dla każdego kontrolera i akcji – np. nie jest możliwe wstawienie lub aktualizacja wartości pola, które nie miało być częścią danej akcji.	2

#	Opis	Poziom
15.3.4	Zweryfikuj, że wszystkie komponenty proxy i oprogramowania pośredniczącego przekazują oryginalny adres IP użytkownika poprawnie, z użyciem zaufanych pól danych niepodlegających manipulacji przez użytkownika końcowego, a aplikacja i serwer WWW używają tej poprawnej wartości do logowania zdarzeń i decyzji bezpieczeństwa, takich jak ograniczanie częstotliwości żądań —mając na uwadze, że nawet oryginalny adres IP może nie być wiarygodny ze względu na dynamiczne IP, VPN-y czy zapory korporacyjne.	2
15.3.5	Zweryfikuj, że aplikacja jawnie zapewnia, iż zmienne są właściwego typu, oraz wykonuje operacje ścisłej równości i porównania. Ma to na celu uniknięcie podatności type juggling lub type confusion, wynikających z założeń kodu aplikacji co do typu zmiennej.	2
15.3.6	Zweryfikuj, że kod JavaScript jest pisany w sposób zapobiegający prototype pollution —na przykład z użyciem Set() lub Map() zamiast literałów obiektowych.	2
15.3.7	Zweryfikuj, że aplikacja posiada obrony przed atakami HTTP parameter pollution, w szczególności jeśli framework aplikacji nie rozróżnia źródła parametrów żądania (ciąg zapytania, parametry treści, ciasteczka czy pola nagłówka).	2

V15.4 Bezpieczna współbieżność

Problemy współbieżności —takie jak wyścigi (race conditions), podatności time-of-check to time-of-use (TOCTOU), zakleszczenia (deadlocks), uwięzienia (livelocks), zagłodzenie wątków (thread starvation) czy niewłaściwa synchronizacja —mogą prowadzić do nieprzewidywalnego zachowania i ryzyk bezpieczeństwa. Ta sekcja zawiera różne techniki i strategie pomagające ograniczyć te ryzyka.

#	Opis	Poziom
15.4.1	Zweryfikuj, że obiekty współdzielone w kodzie wielowątkowym (takie jak pamięci podręczne, pliki czy obiekty w pamięci używane przez wiele wątków) są używane w sposób bezpieczny, z zastosowaniem typów bezpiecznych wątkowo i mechanizmów synchronizacji, takich jak blokady czy semaforey, aby uniknąć wyścigów i uszkodzenia danych.	3

#	Opis	Poziom
15.4.2	Zweryfikuj, że sprawdzenia stanu zasobu —takie jak jego istnienie czy uprawnienia —oraz zależne od nich działania są wykonywane jako pojedyncza operacja atomowa, aby zapobiec wyścigom time-of-check to time-of-use (TOCTOU). Przykłady: sprawdzenie istnienia pliku przed jego otwarciem albo weryfikacja dostępu użytkownika przed jego przyznaniem.	3
15.4.3	Zweryfikuj, że blokady są używane konsekwentnie, aby wątki nie ułykały —czy to czekając na siebie nawzajem, czy ponawiając próby w nieskończoność —oraz że logika blokowania pozostaje w kodzie odpowiedzialnym za zarządzanie zasobem, aby blokady nie mogły być nieumyślnie lub złośliwie modyfikowane przez zewnętrzne klasy lub kod.	3
15.4.4	Zweryfikuj, że polityki alokacji zasobów zapobiegają zagłodzeniu wątków poprzez zapewnienie sprawiedliwego dostępu do zasobów —na przykład z wykorzystaniem pul wątków —pozwalając wątkom o niższym priorytecie na wykonanie w rozsądnym czasie.	3

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 Logowanie zdarzeń bezpieczeństwa i obsługa błędów

Cel kontrolny

Logi bezpieczeństwa różnią się od logów błędów czy wydajności —służą do rejestrowania zdarzeń istotnych z punktu widzenia bezpieczeństwa, takich jak decyzje uwierzytelniania, decyzje kontroli dostępu oraz próby obejścia mechanizmów bezpieczeństwa, na przykład walidacji danych wejściowych czy walidacji logiki biznesowej. Ich celem jest wspieranie wykrywania, reagowania i dochodzeń poprzez dostarczanie ustrukturyzowanych danych o wysokiej wartości sygnałowej dla narzędzi analitycznych, takich jak systemy SIEM.

Logi nie powinny zawierać wrażliwych danych osobowych, chyba że wymaga tego prawo, a wszelkie logowane dane muszą być chronione jako zasób o wysokiej wartości. Logowanie nie może naruszać

prywatności ani bezpieczeństwa systemu. Aplikacje muszą również zawodzić w sposób bezpieczny, unikając zbędnego ujawniania informacji lub zakłóceń.

Szczegółowe wskazówki implementacyjne znajdują się w ściągach OWASP wymienionych w sekcji źródeł.

V16.1 Dokumentacja logowania zdarzeń bezpieczeństwa

Ta sekcja zapewnia jasny i kompletny inwentarz logowania w całym stosie technologicznym aplikacji. Jest to niezbędne dla skutecznego monitorowania bezpieczeństwa, reagowania na incydenty i zgodności.

#	Opis	Poziom
16.1.1	Zweryfikuj, że istnieje inwentarz dokumentujący logowanie wykonywane na każdej warstwie stosu technologicznego aplikacji: jakie zdarzenia są logowane, formaty logów, gdzie logi są przechowywane, jak są wykorzystywane, jak kontrolowany jest do nich dostęp oraz jak długo są przechowywane.	2

V16.2 Ogólne logowanie

Ta sekcja zawiera wymagania zapewniające, że logi bezpieczeństwa mają spójną strukturę i zawierają oczekiwane metadane. Celem jest uczynienie logów odczytywalnymi maszynowo i możliwymi do analizy w systemach rozproszonych i różnych narzędziach.

Zdarzenia bezpieczeństwa w naturalny sposób często dotyczą danych wrażliwych. Jeśli takie dane są logowane bezrefleksyjnie, same logi stają się danymi klasyfikowanymi —a więc podlegają wymaganiom szyfrowania, ostrzejszym politykom retencji i potencjalnemu ujawnieniu podczas audytów.

Dlatego krytyczne jest logowanie wyłącznie tego, co niezbędne, oraz traktowanie danych z logów z taką samą starannością, jak innych zasobów wrażliwych.

Poniższe wymagania ustanawiają fundamenty dotyczące metadanych logowania, synchronizacji, formatu i kontroli.

#	Opis	Poziom
16.2.1	Zweryfikuj, że każdy wpis logu zawiera niezbędne metadane (takie jak: kiedy, gdzie, kto, co), które pozwoliłyby na szczegółowe odtworzenie osi czasu w razie wystąpienia zdarzenia.	2

#	Opis	Poziom
16.2.2	Zweryfikuj, że źródła czasu wszystkich komponentów logujących są zsynchronizowane oraz że znaczniki czasu w metadanych zdarzeń bezpieczeństwa używają UTC lub zawierają jawne przesunięcie strefy czasowej. UTC jest zalecane dla zapewnienia spójności w systemach rozproszonych i uniknięcia niejasności przy zmianach czasu letniego.	2
16.2.3	Zweryfikuj, że aplikacja zapisuje lub rozsyła logi wyłącznie do plików i usług udokumentowanych w inwentarzu logów.	2
16.2.4	Zweryfikuj, że logi mogą być odczytywane i korelowane przez używany procesor logów, najlepiej z zastosowaniem powszechnego formatu logowania.	2
16.2.5	Zweryfikuj, że przy logowaniu danych wrażliwych aplikacja egzekwuje logowanie zgodne z poziomem ochrony danych. Na przykład logowanie pewnych danych –takich jak poświadczenia czy dane płatnicze –może być niedozwolone. Inne dane, takie jak tokeny sesji, mogą być logowane wyłącznie w postaci zahaszowanej lub zamaskowanej, w całości lub częściowo.	2

V16.3 Zdarzenia bezpieczeństwa

Ta sekcja definiuje wymagania logowania zdarzeń istotnych dla bezpieczeństwa w aplikacji. Rejestrowanie tych zdarzeń jest krytyczne dla wykrywania podejrzanych zachowań, wspierania dochodzeń i wypełniania obowiązków zgodności.

Sekcja przedstawia typy zdarzeń, które powinny być logowane, ale nie próbuje podawać wyczerpujących szczegółów. Każda aplikacja ma unikalne czynniki ryzyka i kontekst operacyjny.

Zwróć uwagę, że choć ASVS obejmuje logowanie zdarzeń bezpieczeństwa, alarmowanie i korelacja (np. reguły SIEM czy infrastruktura monitorowania) pozostają poza zakresem i są obsługiwane przez systemy operacyjne i monitorujące.

#	Opis	Poziom
16.3.1	Zweryfikuj, że wszystkie operacje uwierzytelniania są logowane, łącznie z próbami udanymi i nieudanymi. Powinny być również zbierane dodatkowe metadane, takie jak typ uwierzytelniania czy użyte czynniki.	2
16.3.2	Zweryfikuj, że nieudane próby autoryzacji są logowane. Dla L3 musi to obejmować logowanie wszystkich decyzji autoryzacyjnych, w tym logowanie dostępu do danych wrażliwych (bez logowania samych danych wrażliwych).	2

#	Opis	Poziom
16.3.3	Zweryfikuj, że aplikacja loguje zdarzenia bezpieczeństwa zdefiniowane w dokumentacji, a także loguje próby obejścia mechanizmów bezpieczeństwa, takich jak walidacja danych wejściowych, logika biznesowa i ochrona przed automatyzacją.	2
16.3.4	Zweryfikuj, że aplikacja loguje nieoczekiwane błędy i awarie mechanizmów bezpieczeństwa, takie jak błędy TLS w backendzie.	2

V16.4 Ochrona logów

Logi to cenne artefakty śledcze i muszą być chronione. Jeśli logi można łatwo zmodyfikować lub usunąć, tracą integralność i stają się niewiarygodne w dochodzeniach po incydentach czy postępowaniach prawnych. Logi mogą ujawniać wewnętrzne zachowanie aplikacji lub wrażliwe metadane, co czyni je atrakcyjnym celem dla atakujących.

Ta sekcja definiuje wymagania zapewniające ochronę logów przed nieautoryzowanym dostępem, manipulacją i ujawnieniem oraz ich bezpieczne przesyłanie i przechowywanie w bezpiecznych, odizolowanych systemach.

#	Opis	Poziom
16.4.1	Zweryfikuj, że wszystkie komponenty logujące odpowiednio kodują dane, aby zapobiec wstrzyknięciu do logów (log injection).	2
16.4.2	Zweryfikuj, że logi są chronione przed nieautoryzowanym dostępem i nie mogą być modyfikowane.	2
16.4.3	Zweryfikuj, że logi są bezpiecznie przesyłane do logicznie odseparowanego systemu do analizy, wykrywania, alarmowania i eskalacji. Celem jest zapewnienie, że w razie włamania do aplikacji logi nie zostaną skompromitowane.	2

V16.5 Obsługa błędów

Ta sekcja definiuje wymagania zapewniające, że aplikacje zawodzą w sposób kontrolowany i bezpieczny, bez ujawniania wrażliwych szczegółów wewnętrznych.

#	Opis	Poziom
16.5.1	Zweryfikuj, że po wystąpieniu nieoczekiwanego lub wrażliwego z punktu widzenia bezpieczeństwa błędu konsumentowi zwracany jest komunikat ogólny, gwarantujący brak ekspozycji wrażliwych wewnętrznych danych systemowych, takich jak ślady stosu, zapytania, klucze tajne i tokeny.	2
16.5.2	Zweryfikuj, że aplikacja kontynuuje bezpieczne działanie, gdy dostęp do zasobu zewnętrznego zawiedzie —na przykład stosując wzorce takie jak circuit breaker czy kontrolowana degradacja (graceful degradation).	2
16.5.3	Zweryfikuj, że aplikacja zawodzi w sposób kontrolowany i bezpieczny, również przy wystąpieniu wyjątku, zapobiegając stanom fail-open, takim jak przetworzenie transakcji pomimo błędów logiki walidacji.	2
16.5.4	Zweryfikuj, że zdefiniowana jest procedura obsługi błędów „ostatniej szansy” , przechwytywająca wszystkie nieobsłużone wyjątki. Ma to zarówno zapobiec utracie szczegółów błędów, które muszą trafić do plików logów, jak i zapewnić, że błąd nie wyłączy całego procesu aplikacji, prowadząc do utraty dostępności.	3

Uwaga: niektóre języki (w tym Swift, Go oraz —poprzez powszechną praktykę projektową —wiele języków funkcyjnych) nie wspierają wyjątków ani procedur obsługi „ostatniej szansy”. W takim przypadku architekt i programiści powinni użyć wzorca właściwego dla danego języka lub frameworka, aby zapewnić, że aplikacje potrafią bezpiecznie obsługiwać zdarzenia wyjątkowe, nieoczekiwane lub związane z bezpieczeństwem.

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- OWASP Web Security Testing Guide: Testing for Error Handling
- Sekcja OWASP Authentication Cheat Sheet o komunikatach błędów
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

Cel kontrolny

Web Real-Time Communication (WebRTC) umożliwia wymianę głosu, wideo i danych w czasie rzeczywistym w nowoczesnych aplikacjach. Wraz ze wzrostem adopcji zabezpieczenie infrastruktury

tury WebRTC staje się krytyczne. Ta sekcja zawiera wymagania bezpieczeństwa dla interesariuszy, którzy tworzą, hostują lub integrują systemy WebRTC.

Rynek WebRTC można z grubsza podzielić na trzy segmenty:

1. Twórcy produktów: dostawcy rozwiązań własnościowych i open source, którzy tworzą i dostarczają produkty oraz rozwiązania WebRTC. Koncentrują się na rozwijaniu solidnych i bezpiecznych technologii WebRTC, z których mogą korzystać inni.
2. Communication Platforms as a Service (CPaaS): dostawcy oferujący API, SDK oraz niezbędną infrastrukturę lub platformy umożliwiające funkcjonalność WebRTC. Dostawcy CPaaS mogą korzystać z produktów pierwszej kategorii albo rozwijać własne oprogramowanie WebRTC, aby świadczyć te usługi.
3. Dostawcy usług: organizacje, które wykorzystują produkty twórców produktów lub dostawców CPaaS albo rozwijają własne rozwiązania WebRTC. Tworzą i wdrażają aplikacje do konferencji online, ochrony zdrowia, e-learningu i innych dziedzin, w których komunikacja w czasie rzeczywistym jest kluczowa.

Przedstawione tu wymagania bezpieczeństwa są skierowane przede wszystkim do twórców produktów, dostawców CPaaS i dostawców usług, którzy:

- Wykorzystują rozwiązania open source do budowy swoich aplikacji WebRTC.
- Używają komercyjnych produktów WebRTC jako części swojej infrastruktury.
- Korzystają z tworzonych wewnętrznie rozwiązań WebRTC lub integrują różne komponenty w spójną ofertę usługową.

Warto zaznaczyć, że te wymagania bezpieczeństwa nie dotyczą programistów, którzy korzystają wyłącznie z SDK i API dostarczanych przez dostawców CPaaS. W ich przypadku to dostawcy CPaaS zwykle odpowiadają za większość podstawowych kwestii bezpieczeństwa swoich platform, a ogólny standard bezpieczeństwa, taki jak ASVS, może nie w pełni odpowiadać ich potrzebom.

V17.1 Serwer TURN

Ta sekcja definiuje wymagania bezpieczeństwa dla systemów, które utrzymują własne serwery TURN (Traversal Using Relays around NAT). Serwery TURN pomagają w przekazywaniu mediów w restrykcyjnych środowiskach sieciowych, ale przy błędnej konfiguracji mogą stwarzać ryzyka. Te mechanizmy koncentrują się na bezpiecznym filtrowaniu adresów i ochronie przed wyczerpaniem zasobów.

#	Opis	Poziom
17.1.1	Zweryfikuj, że usługa Traversal Using Relays around NAT (TURN) zezwala na dostęp wyłącznie do adresów IP, które nie są zarezerwowane do celów specjalnych (np. sieci wewnętrzne, broadcast, loopback). Zwróć uwagę, że dotyczy to zarówno adresów IPv4, jak i IPv6.	2
17.1.2	Zweryfikuj, że usługa Traversal Using Relays around NAT (TURN) nie jest podatna na wyczerpanie zasobów, gdy uprawnieni użytkownicy próbują otworzyć dużą liczbę portów na serwerze TURN.	3

V17.2 Media

Te wymagania dotyczą wyłącznie systemów, które hostują własne serwery mediów WebRTC, takie jak Selective Forwarding Units (SFU), Multipoint Control Units (MCU), serwery nagrywające lub serwery bramek. Serwery mediów obsługują i dystrybuują strumień mediów, przez co ich bezpieczeństwo jest krytyczne dla ochrony komunikacji między uczestnikami. Ochrona strumieni mediów jest w aplikacjach WebRTC sprawą nadrzędną –zapobiega podsłuchowi, manipulacji i atakom odmowy usługi, które mogłyby naruszyć prywatność użytkowników i jakość komunikacji.

W szczególności konieczne jest wdrożenie zabezpieczeń przed atakami zalewowymi (flood), takich jak ograniczanie częstotliwości żądań, walidacja znaczników czasu, użycie zsynchronizowanych zegarów do dopasowania interwałów czasu rzeczywistego oraz zarządzanie buforami w celu zapobiegania przepełnieniom i utrzymania właściwego taktowania. Jeśli pakiety danej sesji medialnej napływają zbyt szybko, nadmiarowe pakiety powinny być odrzucane. Ważna jest również ochrona systemu przed zniekształconymi pakietami poprzez walidację danych wejściowych, bezpieczną obsługę przepełnień liczb całkowitych, zapobieganie przepełnieniom bufora oraz stosowanie innych solidnych technik obsługi błędów.

Systemy opierające się wyłącznie na komunikacji medialnej peer-to-peer między przeglądarkami, bez udziału pośredniczących serwerów mediów, są wyłączone z tych specyficznych wymagań dotyczących mediów.

Ta sekcja odnosi się do użycia Datagram Transport Layer Security (DTLS) w kontekście WebRTC. Wymaganie dotyczące posiadania udokumentowanej polityki zarządzania kluczami kryptograficznymi znajduje się w rozdziale „Kryptografia”. Informacje o zatwierdzonych metodach kryptograficznych można znaleźć w Załączniku kryptograficznym ASVS albo w dokumentach takich jak NIST SP 800-52 Rev. 2 lub BSI TR-02102-2 (wersja 2025-01).

#	Opis	Poziom
17.2.1	Zweryfikuj, że klucz certyfikatu Datagram Transport Layer Security (DTLS) jest zarządzany i chroniony zgodnie z udokumentowaną polityką zarządzania kluczami kryptograficznymi.	2
17.2.2	Zweryfikuj, że serwer mediów jest skonfigurowany do używania i wspierania zatwierdzonych zestawów szyfrów Datagram Transport Layer Security (DTLS) oraz bezpiecznego profilu ochrony dla rozszerzenia DTLS służącego ustanawianiu kluczy dla Secure Real-time Transport Protocol (DTLS-SRTP).	2
17.2.3	Zweryfikuj, że uwierzytelnianie Secure Real-time Transport Protocol (SRTP) jest sprawdzane na serwerze mediów, aby zapobiec sytuacji, w której ataki wstrzyknięcia Real-time Transport Protocol (RTP) prowadzą do odmowy usługi albo wstawienia treści audio lub wideo do strumienia mediów.	2
17.2.4	Zweryfikuj, że serwer mediów jest w stanie kontynuować przetwarzanie przychodzącego ruchu medialnego po napotkaniu zniekształconych pakietów Secure Real-time Transport Protocol (SRTP).	2
17.2.5	Zweryfikuj, że serwer mediów jest w stanie kontynuować przetwarzanie przychodzącego ruchu medialnego podczas zalewu pakietami Secure Real-time Transport Protocol (SRTP) od uprawnionych użytkowników.	3
17.2.6	Zweryfikuj, że serwer mediów nie jest podatny na podatność „ClientHello” Race Condition w Datagram Transport Layer Security (DTLS) —sprawdzając, czy serwer mediów jest publicznie znany jako podatny, albo wykonując test wyścigu.	3
17.2.7	Zweryfikuj, że wszelkie mechanizmy nagrywania audio lub wideo powiązane z serwerem mediów są w stanie kontynuować przetwarzanie przychodzącego ruchu medialnego podczas zalewu pakietami Secure Real-time Transport Protocol (SRTP) od uprawnionych użytkowników.	3
17.2.8	Zweryfikuj, że certyfikat Datagram Transport Layer Security (DTLS) jest sprawdzany względem atrybutu fingerprint Session Description Protocol (SDP), z zakończeniem strumienia mediów w razie niepowodzenia sprawdzenia, aby zapewnić autentyczność strumienia mediów.	3

V17.3 Sygnalizacja

Ta sekcja definiuje wymagania dla systemów, które utrzymują własne serwery sygnalizacyjne WebRTC. Sygnalizacja koordynuje komunikację peer-to-peer i musi być odporna na ataki mogące zakłócić ustanawianie lub kontrolę sesji.

Aby zapewnić bezpieczną sygnalizację, systemy muszą obsługiwać zniekształcone dane wejściowe w

sposób kontrolowany i pozostawać dostępne pod obciążeniem.

#	Opis	Poziom
17.3.1	Zweryfikuj, że serwer sygnalizacyjny jest w stanie kontynuować przetwarzanie prawidłowych przychodzących komunikatów sygnalizacyjnych podczas ataku zalewowego. Powinno to być osiągnięte poprzez wdrożenie ograniczania częstotliwości żądań na poziomie sygnalizacji.	2
17.3.2	Zweryfikuj, że serwer sygnalizacyjny jest w stanie kontynuować przetwarzanie prawidłowych komunikatów sygnalizacyjnych po napotkaniu zniekształconego komunikatu sygnalizacyjnego, który mógłby spowodować odmowę usługi. Może to obejmować walidację danych wejściowych, bezpieczną obsługę przepełnień liczb całkowitych, zapobieganie przepełnieniom bufora oraz stosowanie innych solidnych technik obsługi błędów.	2

Źródła

Więcej informacji można znaleźć w następujących materiałach:

- Podatność WebRTC DTLS ClientHello DoS najlepiej dokumentują wpis na blogu Enable Security skierowany do specjalistów bezpieczeństwa oraz powiązany white paper skierowany do programistów WebRTC
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP)
- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

Załącznik A: Glosariusz

- **Absolute Maximum Session Lifetime** (bezwzględny maksymalny czas życia sesji) – nazywany przez NIST również „Overall Timeout”; maksymalny czas, przez jaki sesja może pozostawać aktywna po uwierzytelnieniu, niezależnie od interakcji użytkownika. Jest to składnik wygasania sesji.

- **Allowlist** (lista dozwolonych) –lista dozwolonych danych lub operacji, na przykład lista znaków dopuszczonych w walidacji danych wejściowych.
- **Anti-forgery token** (token anti-forgery) –mechanizm, w którym jeden lub więcej tokenów jest przekazywanych w żądaniu i walidowanych przez serwer aplikacji w celu zapewnienia, że żądanie pochodzi z oczekiwanego endpointu.
- **Application Security** (bezpieczeństwo aplikacji) –bezpieczeństwo na poziomie aplikacji koncentruje się na analizie komponentów tworzących warstwę aplikacji modelu odniesienia Open Systems Interconnection (modelu OSI), a nie na przykład na bazowym systemie operacyjnym czy podłączonych sieciach.
- **Application Security Verification** (weryfikacja bezpieczeństwa aplikacji) –techniczna ocena aplikacji względem OWASP ASVS.
- **Application Security Verification Report** (raport z weryfikacji bezpieczeństwa aplikacji) –raport dokumentujący ogólne wyniki oraz analizę wspierającą, przygotowany przez weryfikatora dla konkretnej aplikacji.
- **Authentication** (uwierzytelnianie) –weryfikacja deklarowanej tożsamości użytkownika aplikacji.
- **Automated Verification** (weryfikacja automatyczna) –użycie zautomatyzowanych narzędzi (narzędzi analizy dynamicznej, statycznej lub obu), które wykorzystują sygnatury podatności do znajdowania problemów.
- **Black box testing** (testowanie „czarnoskrzynkowe”) –metoda testowania oprogramowania badająca funkcjonalność aplikacji bez wglądu w jej wewnętrzne struktury czy działanie.
- **Common Weakness Enumeration** (CWE) –tworzona przez społeczność lista powszechnych słabości bezpieczeństwa oprogramowania. Służy jako wspólny język, miara dla narzędzi bezpieczeństwa oprogramowania oraz punkt odniesienia dla identyfikacji, ograniczania i zapobiegania słabościom.
- **Component** (komponent) –samodzielna jednostka kodu z powiązаныmi interfejsami dyskowymi i sieciowymi, komunikująca się z innymi komponentami.
- **Credential Service Provider** (CSP) –nazywany także dostawcą tożsamości (IdP). Źródło danych użytkowników, które może być wykorzystywane jako źródło uwierzytelniania przez inne aplikacje.
- **Cross-Site Script Inclusion** (XSSI) –wariant ataku Cross-Site Scripting (XSS), w którym aplikacja internetowa pobiera złośliwy kod z zewnętrznego zasobu i włącza go jako część własnej treści.
- **Cross-Site Scripting** (XSS) –podatność bezpieczeństwa typowa dla aplikacji internetowych, pozwalająca na wstrzykiwanie skryptów po stronie klienta do treści.
- **Cryptographic module** (moduł kryptograficzny) –sprzęt, oprogramowanie i/lub firmware implementujące algorytmy kryptograficzne i/lub generujące klucze kryptograficzne.
- **Cryptographically secure pseudo-random number generator** (CSPRNG) –generator liczb pseudolosowych o właściwościach czyniących go odpowiednim do zastosowań kryptograficznych; nazywany także kryptograficznym generatorem liczb losowych (CRNG).
- **Datagram Transport Layer Security** (DTLS) –protokół kryptograficzny zapewniający bezpieczeństwo komunikacji przez połączenie sieciowe. Oparty na protokole TLS, lecz przys-

tosowany do ochrony protokołów datagramowych (zwykle po UDP). Zdefiniowany w RFC 9147 dla DTLS 1.3.

- **Datagram Transport Layer Security Extension to Establish Keys for the Secure Real-time Transport Protocol** (DTLS-SRTP) –mechanizm wykorzystania uzgadniania DTLS do ustanowienia materiału klucza dla sesji SRTP. Zdefiniowany w RFC 5764.
- **Design Verification** (weryfikacja projektu) –techniczna ocena architektury bezpieczeństwa aplikacji.
- **Dynamic Application Security Testing** (DAST) –technologie zaprojektowane do wykrywania stanów wskazujących na podatność bezpieczeństwa w działającej aplikacji.
- **Dynamic Verification** (weryfikacja dynamiczna) –użycie zautomatyzowanych narzędzi wykorzystujących sygnatury podatności do znajdowania problemów podczas wykonywania aplikacji.
- **Fast IDentity Online** (FIDO) –zestaw standardów uwierzytelniania pozwalający na stosowanie różnorodnych metod uwierzytelniania, w tym biometrii, modułów Trusted Platform Module (TPM), tokenów bezpieczeństwa USB itd.
- **Hardware Security Module** (HSM, sprzętowy moduł bezpieczeństwa) –komponent sprzętowy przechowujący klucze kryptograficzne i inne sekrety w sposób chroniony.
- **Hibernate Query Language** (HQL) –język zapytań z wyglądu podobny do SQL, używany przez bibliotekę ORM Hibernate.
- **HTTP Strict Transport Security** (HSTS) –polityka nakazująca przeglądarce łączenie się z domeną zwracającą nagłówki wyłącznie przez TLS i wyłącznie przy przedstawieniu ważnego certyfikatu. Aktywowana polem nagłówka odpowiedzi Strict-Transport-Security.
- **HyperText Transfer Protocol** (HTTP) –protokół aplikacyjny dla rozproszonych, kolaboracyjnych, hipermedialnych systemów informacyjnych. Fundament komunikacji danych w sieci World Wide Web.
- **HyperText Transfer Protocol over SSL/TLS** (HTTPS) –metoda zabezpieczania komunikacji HTTP poprzez jej szyfrowanie z użyciem Transport Layer Security (TLS).
- **Identity Provider** (IdP, dostawca tożsamości) –w materiałach NIST nazywany także Credential Service Provider (CSP). Podmiot zapewniający źródło uwierzytelniania dla innych aplikacji.
- **Inactivity Timeout** (limit czasu bezczynności) –czas, przez jaki sesja może pozostawać aktywna przy braku interakcji użytkownika z aplikacją. Jest to składnik wygasania sesji.
- **Input Validation** (walidacja danych wejściowych) –kanonikalizacja i walidacja niezaufanych danych wejściowych od użytkownika.
- **JSON Web Token** (JWT) –RFC 7519 definiuje standard obiektu danych JSON składającego się z sekcji nagłówka wyjaśniającej, jak walidować obiekt, sekcji treści zawierającej zestaw oświadczeń oraz sekcji podpisu zawierającej podpis cyfrowy, którym można walidować zawartość sekcji treści. Jest to typ tokena samowystarczalnego.
- **Local File Inclusion** (LFI) –atak wykorzystujący podatne procedury dołączania plików w aplikacji, prowadzący do dołączenia plików lokalnych obecnych już na serwerze.
- **Malicious Code** (złośliwy kod) –kod wprowadzony do aplikacji podczas jej rozwoju bez wiedzy właściciela aplikacji, obchodzący zamierzoną politykę bezpieczeństwa aplikacji. To nie to samo

co malware, taki jak wirus czy robak!

- **Malware** –kod wykonywalny wprowadzony do aplikacji w czasie działania bez wiedzy użytkownika lub administratora aplikacji.
- **Message authentication code** (MAC, kod uwierzytelniania wiadomości) –kryptograficzna suma kontrolna danych, obliczana algorytmem generowania MAC, służąca zapewnieniu ich integralności i autentyczności.
- **Multi-factor authentication** (MFA, uwierzytelnianie wieloskładnikowe) –uwierzytelnianie obejmujące dwa lub więcej pojedynczych czynników.
- **Mutual TLS** (mTLS) –zob. TLS client authentication.
- **Object-relational Mapping** (ORM, mapowanie obiektowo-relacyjne) –system pozwalający odwoływać się do relacyjnej/tabelarycznej bazy danych i odpytywać ją z poziomu programu aplikacji przy użyciu modelu obiektowego zgodnego z aplikacją.
- **One-time Password** (OTP, hasło jednorazowe) –hasło generowane unikalnie do jednorazowego użycia.
- **Open Worldwide Application Security Project** (OWASP) –Open Worldwide Application Security Project (OWASP) to światowa, wolna i otwarta społeczność skoncentrowana na poprawie bezpieczeństwa oprogramowania aplikacyjnego. Naszą misją jest uczynienie bezpieczeństwa aplikacji „widocznym”, aby ludzie i organizacje mogły podejmować świadome decyzje dotyczące ryzyk bezpieczeństwa aplikacji. Zob.: <https://www.owasp.org/>.
- **Password-Based Key Derivation Function 2** (PBKDF2) –specjalny algorytm jednokierunkowy używany do tworzenia silnego klucza kryptograficznego z tekstu wejściowego (takiego jak hasło) i dodatkowej losowej wartości soli; może zatem służyć utrudnieniu łamania hasła offline, jeśli zamiast oryginalnego hasła przechowywana jest wartość wynikowa.
- **Public Key Infrastructure** (PKI, infrastruktura klucza publicznego) –rozwiązanie wiążące klucze publiczne z tożsamościami odpowiednich podmiotów. Powiązanie jest ustanawiane w procesie rejestracji i wystawiania certyfikatów w urzędzie certyfikacji (CA) i przez ten urząd.
- **Public Switched Telephone Network** (PSTN, publiczna komutowana sieć telefoniczna) –tradycyjna sieć telefoniczna obejmująca zarówno telefony stacjonarne, jak i komórkowe.
- **Real-time Transport Protocol** (RTP) i **Real-time Transport Control Protocol** (RTCP) –dwa protokoły używane łącznie do transportu strumieni multimedialnych. Wykorzystywane przez stos WebRTC. Zdefiniowane w RFC 3550.
- **Reference Token** (token referencyjny) –typ tokena działający jako wskaźnik lub identyfikator stanu bądź metadanych przechowywanych na serwerze; czasem nazywany tokenem losowym lub nieprzezroczystym (opaque). W przeciwieństwie do tokenów samowystarczalnych, które osadzają część swoich danych w samym tokenie, tokeny referencyjne nie zawierają żadnych informacji wewnętrznych i polegają na serwerze w kwestii kontekstu. Token referencyjny będzie identyfikatorem sesji albo będzie go zawierał.
- **Relying Party** (RP, strona ufająca) –zasadniczo aplikacja polegająca na tym, że użytkownik uwierzył się u odrębnego dostawcy uwierzytelniania. Aplikacja polega na tokenie lub zestawie podpisanych asercji dostarczonych przez tego dostawcę, aby zaufać, że użytkownik jest tym, za kogo się podaje.

- **Remote File Inclusion** (RFI) –atak wykorzystujący podatne procedury dołączania w aplikacji, skutkujący dołączeniem plików zdalnych.
- **Scalable Vector Graphics** (SVG) –oparty na XML język znaczników do opisu dwuwymiarowej grafiki wektorowej.
- **Secure Real-time Transport Protocol** (SRTP) i **Secure Real-time Transport Control Protocol** (SRTCP) –profil protokołów RTP i RTCP zapewniający wsparcie szyfrowania wiadomości, uwierzytelniania i ochrony integralności. Zdefiniowany w RFC 3711.
- **Security Architecture** (architektura bezpieczeństwa) –abstrakcja projektu aplikacji identyfikująca i opisująca, gdzie i jak używane są mechanizmy bezpieczeństwa, a także identyfikująca i opisująca lokalizację oraz wrażliwość danych użytkowników i aplikacji.
- **Security Assertion Markup Language** (SAML) –otwarty standard uwierzytelniania z jednokrotnym logowaniem, oparty na przekazywaniu podpisanych asercji (zwykle obiektów XML) między dostawcą tożsamości a stroną ufającą.
- **Security Configuration** (konfiguracja bezpieczeństwa) –konfiguracja czasu działania aplikacji wpływająca na sposób użycia mechanizmów bezpieczeństwa.
- **Security Control** (mechanizm bezpieczeństwa) –funkcja lub komponent wykonujący sprawdzenie bezpieczeństwa (np. sprawdzenie autoryzacji) albo wywołujący skutek związany z bezpieczeństwem (np. wygenerowanie wpisu audytowego).
- **Security information and event management** (SIEM) –system wykrywania zagrożeń, zapewniania zgodności i zarządzania incydentami bezpieczeństwa poprzez zbieranie i analizę danych związanych z bezpieczeństwem z różnych źródeł w infrastrukturze IT organizacji.
- **Self-Contained Token** (token samowystarczalny) –token zawierający jeden lub więcej atrybutów, które nie zależą od stanu po stronie serwera ani innego zewnętrznego magazynu. Tokeny te zapewniają autentyczność i integralność zawartych atrybutów, umożliwiając bezpieczną, „bezstanową”wymianę informacji między systemami. Tokeny samowystarczalne są zwykle zabezpieczane technikami kryptograficznymi, takimi jak podpisy cyfrowe lub kody uwierzytelniania wiadomości (MAC), aby zapewnić autentyczność, integralność, a w niektórych przypadkach poufność danych. Typowe przykłady to asercje SAML i JWT.
- **Server-side Request Forgery** (SSRF) –atak nadużywający funkcjonalności serwera do odczytu lub aktualizacji zasobów wewnętrznych. Atakujący dostarcza lub modyfikuje URL, z którego kod działający na serwerze odczyta dane lub do którego je prześle.
- **Session Description Protocol** (SDP) –format komunikatów do ustanawiania sesji multimedialnych (używany na przykład w WebRTC). Zdefiniowany w RFC 4566.
- **Session Identifier** lub **Session ID** (identyfikator sesji) –klucz identyfikujący stanową sesję przechowywaną w backendzie. Przekazywany do i od klienta jako „token referencyjny” albo w jego wnętrzu.
- **Session Token** (token sesji) –zbiórce określenie używane w tym standardzie dla tokena lub wartości używanych zarówno w bezstanowych mechanizmach sesji (używających tokena samowystarczalnego), jak i stanowych mechanizmach sesji (używających tokena referencyjnego).
- **Session Traversal Utilities for NAT** (STUN) –protokół wspomagający przechodzenie przez NAT

w celu ustanawiania komunikacji peer-to-peer. Zdefiniowany w RFC 3489.

- **Single-factor authenticator** (mechanizm uwierzytelniania jednoskładnikowego) –mechanizm sprawdzania, że użytkownik jest uwierzytelniony. Powinien opierać się na czymś, co wiesz (zapamiętane sekrety, hasła, frazy hasłowe, PIN-y), czymś, czym jesteś (biometria, odcisk palca, skan twarzy), albo czymś, co masz (tokeny OTP, urządzenie kryptograficzne takie jak karta inteligentna).
- **Single Sign-on Authentication** (SSO, uwierzytelnianie z jednokrotnym logowaniem) –występuje, gdy użytkownik loguje się do jednej aplikacji, a następnie jest automatycznie logowany do innych aplikacji bez konieczności ponownego uwierzytelniania. Na przykład po zalogowaniu do Google użytkownik zostanie automatycznie zalogowany do innych usług Google, takich jak YouTube, Google Docs czy Gmail.
- **Software bill of materials** (SBOM) –ustrukturyzowana, kompletna lista wszystkich komponentów, modułów, bibliotek, frameworków i innych zasobów wymaganych do zbudowania lub złożenia aplikacji.
- **Software Composition Analysis** (SCA) –zestaw technologii zaprojektowanych do analizy składu aplikacji, zależności, bibliotek i pakietów pod kątem podatności bezpieczeństwa konkretnych używanych wersji komponentów. Nie należy mylić z analizą kodu źródłowego, obecnie powszechnie nazywaną SAST.
- **Software development lifecycle** (SDLC, cykl wytwarzania oprogramowania) –proces krok po kroku, w którym powstaje oprogramowanie –od początkowych wymagań po wdrożenie i utrzymanie.
- **SQL Injection** (SQLi, wstrzyknięcie SQL) –technika wstrzykiwania kodu używana do atakowania aplikacji opartych na danych, w której złośliwe instrukcje SQL są wstawiane w punkcie wejściowym.
- **Stateful Session Mechanism** (stanowy mechanizm sesji) –w stanowym mechanizmie sesji aplikacja utrzymuje stan sesji w backendzie, zwykle odpowiadający tokenowi sesji wygenerowanemu kryptograficznie bezpiecznym generatorem liczb pseudolosowych (CSPRNG) i wydanemu użytkownikowi końcowemu.
- **Stateless Session Mechanism** (bezstanowy mechanizm sesji) –bezstanowy mechanizm sesji używa tokena samowystarczального przekazywanego klientom, zawierającego informacje o sesji, które niekoniecznie są przechowywane w usłudze odbierającej i walidującej token. W praktyce usługa będzie potrzebowała dostępu do pewnych informacji o sesji (takich jak lista odwołanych JWT), aby móc egzekwować wymagane mechanizmy bezpieczeństwa.
- **Static application security testing** (SAST) –zestaw technologii zaprojektowanych do analizy kodu źródłowego, kodu bajtowego i binariów aplikacji pod kątem konstrukcji programistycznych i projektowych wskazujących na podatności bezpieczeństwa. Rozwiązania SAST analizują aplikację „od środka”, w stanie nieuruchomionym.
- **Threat Modeling** (modelowanie zagrożeń) –technika polegająca na opracowywaniu coraz bardziej doprecyzowanych architektur bezpieczeństwa w celu identyfikacji agentów zagrożeń, stref bezpieczeństwa, mechanizmów bezpieczeństwa oraz istotnych zasobów technicznych i biznesowych.

- **Time-of-check to time-of-use (TOCTOU)** – sytuacja, w której aplikacja sprawdza stan zasobu przed jego użyciem, lecz stan ten może się zmienić między sprawdzeniem a użyciem. Może to unieważnić wynik sprawdzenia i doprowadzić do wykonania przez aplikację nieprawidłowych działań wskutek tej niespójności stanu.
- **Time based One-time Passwords (TOTP, hasła jednorazowe oparte na czasie)** – metoda generowania OTP, w której bieżący czas stanowi część algorytmu generowania hasła.
- **TLS client authentication** (uwierzytelnianie klienta TLS), nazywane też **Mutual TLS (mTLS)** – w standardowym połączeniu TLS klient może użyć certyfikatu przedstawionego przez serwer do walidacji tożsamości serwera. Przy uwierzytelnianiu klienta TLS również klient używa własnego klucza prywatnego i certyfikatu, aby serwer mógł zwalidować także tożsamość klienta.
- **Transport Layer Security (TLS)** – protokoły kryptograficzne zapewniające bezpieczeństwo komunikacji przez połączenie sieciowe.
- **Traversal Using Relays around NAT (TURN)** – rozszerzenie protokołu STUN wykorzystujące serwer TURN jako przekaźnik, gdy nie można ustanowić bezpośrednich połączeń peer-to-peer. Zdefiniowane w RFC 8656.
- **Trusted execution environment (TEE, zaufane środowisko wykonawcze)** – izolowane środowisko przetwarzania, w którym aplikacje mogą być bezpiecznie wykonywane niezależnie od reszty systemu.
- **Trusted Platform Module (TPM)** – typ HSM zwykle dołączony do większego komponentu sprzętowego, takiego jak płyta główna, i pełniący rolę „korzenia zaufania” (root of trust) dla tego systemu.
- **Trusted Service Layer** (zaufana warstwa usługowa) – dowolny zaufany punkt egzekwowania mechanizmów, taki jak mikroserwis, API serverless, strona serwerowa, zaufane API na urządzeniu klienckim z bezpiecznym rozruchem, API partnerskie lub zewnętrzne itd. „Zaufany” oznacza brak obaw, że niezaufany użytkownik będzie w stanie ominąć lub pominąć tę warstwę albo mechanizmy w niej zaimplementowane.
- **Uniform Resource Identifier (URI)** – unikalny ciąg znaków identyfikujący zasób, taki jak strona internetowa, adres e-mail czy miejsce.
- **Uniform Resource Locator (URL)** – ciąg znaków określający lokalizację zasobu w Internecie.
- **Universally Unique Identifier (UUID)** – unikalny numer referencyjny używany jako identyfikator w oprogramowaniu.
- **Verifier** (weryfikator) – osoba lub zespół dokonujący przeglądu aplikacji względem wymagań OWASP ASVS.
- **Web Real-Time Communication (WebRTC)** – stos protokołów i powiązane webowe API służące do transportu strumieni multimedialnych w aplikacjach internetowych, zwykle w kontekście telekonferencji. Oparty na SRTP, SRTCP, DTLS, SDP oraz STUN/TURN.
- **WebSocket over TLS (WSS)** – praktyka zabezpieczania komunikacji WebSocket poprzez warstwowe umieszczenie WebSocket nad protokołem TLS.
- **What You See Is What You Get (WYSIWYG)** – typ edytora treści wzbogaconej pokazujący, jak treść będzie faktycznie wyglądać po wyrenderowaniu, zamiast pokazywać kod sterujący renderowaniem.

- **X.509 Certificate** (certyfikat X.509) –certyfikat cyfrowy wykorzystujący szeroko przyjęty międzynarodowy standard infrastruktury klucza publicznego (PKI) X.509 do weryfikacji, że klucz publiczny należy do tożsamości użytkownika, komputera lub usługi zawartej w certyfikacie.
- **XML eXternal Entity (XXE)** –typ encji XML, która może uzyskiwać dostęp do treści lokalnych lub zdalnych poprzez zadeklarowany identyfikator systemowy. Może to prowadzić do różnych ataków typu wstrzyknięcia.

Załącznik B: Źródła

Następujące projekty OWASP będą najprawdopodobniej przydatne dla użytkowników i wdrażających niniejszy standard:

Główne projekty OWASP

1. OWASP Top 10 Project: <https://owasp.org/www-project-top-ten/>
2. OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
3. OWASP Proactive Controls: <https://owasp.org/www-project-proactive-controls/>
4. OWASP Software Assurance Maturity Model (SAMM): <https://owasp.org/www-project-samm/>
5. OWASP Secure Headers Project: <https://owasp.org/www-project-secure-headers/>

Projekt OWASP Cheat Sheet Series

Projekt ten zawiera wiele ściąg przydatnych przy różnych tematach ASVS.

Mapowanie do ASVS można znaleźć tutaj: <https://cheatsheetseries.owasp.org/IndexASVS.html>

Projekty związane z bezpieczeństwem mobilnym

1. OWASP Mobile Security Project: <https://owasp.org/www-project-mobile-security/>
2. OWASP Mobile Top 10 Risks: <https://owasp.org/www-project-mobile-top-10/>
3. OWASP Mobile Security Testing Guide oraz Mobile Application Security Verification Standard: <https://owasp.org/www-project-mobile-security-testing-guide/>

Projekty OWASP związane z Internetem rzeczy

1. OWASP Internet of Things Project: <https://owasp.org/www-project-internet-of-things/>

Projekty OWASP dotyczące serverless

1. OWASP Serverless Project: <https://owasp.org/www-project-serverless-top-10/>

Pozostałe

Analogicznie, następujące strony internetowe będą najprawdopodobniej przydatne dla użytkowników i wdrażających niniejszy standard:

1. SecLists Github: <https://github.com/danielmiessler/SecLists>
2. MITRE Common Weakness Enumeration: <https://cwe.mitre.org/>
3. PCI Security Standards Council: <https://www.pcisecuritystandards.org/>
4. PCI Data Security Standard (DSS) v3.2.1 Requirements and Security Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI_DSS_v3-2-1.pdf
5. PCI Software Security Framework - Secure Software Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-Software-Standard-v1_0.pdf
6. PCI Secure Software Lifecycle (Secure SLC) Requirements and Assessment Procedures: https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
7. OWASP ASVS 4.0 Testing Guide <https://github.com/BlazingWind/OWASP-ASVS-4.0-testing-guide>

Załącznik C: Standardy kryptograficzne

Rozdział „Kryptografia”wykracza poza samo definiowanie najlepszych praktyk. Jego celem jest pogłębienie zrozumienia zasad kryptografii oraz zachęcenie do przyjmowania bardziej odpornych, nowoczesnych metod bezpieczeństwa. Niniejszy załącznik dostarcza szczegółowych informacji technicznych dotyczących poszczególnych wymagań, uzupełniając nadrzędne standardy przedstawione w rozdziale „Kryptografia”.

Załącznik definiuje poziomy dopuszczenia różnych mechanizmów kryptograficznych:

- Mechanizmy zatwierdzone (A, approved) mogą być używane w aplikacjach.
- Mechanizmy przestarzałe (L, legacy) nie powinny być używane w aplikacjach, ale mogą być nadal stosowane wyłącznie dla zgodności z istniejącymi starszymi aplikacjami lub kodem. Choć użycie tych mechanizmów nie jest obecnie uznawane za podatność samo w sobie, powinny one zostać jak najszybciej zastąpione mechanizmami bezpieczniejszymi i przyszłościowymi.
- Mechanizmy niedozwolone (D, disallowed) nie mogą być używane, ponieważ są obecnie uznawane za złamane lub nie zapewniają wystarczającego bezpieczeństwa.

Lista ta może zostać nadpisana w kontekście danej aplikacji z różnych powodów, w tym:

- nowych postępów w dziedzinie kryptografii;
- zgodności z regulacjami.

Inwentarz kryptograficzny i dokumentacja

Ta sekcja dostarcza dodatkowych informacji dla V11.1 Inwentarz kryptograficzny i dokumentacja.

Ważne jest zapewnienie, że wszystkie zasoby kryptograficzne – takie jak algorytmy, klucze i certyfikaty – są regularnie wykrywane, inwentaryzowane i oceniane. Dla poziomu 3 powinno to obejmować użycie skanowania statycznego i dynamicznego do wykrywania użycia kryptografii w aplikacji. Pomocne mogą być narzędzia takie jak SAST i DAST, ale możliwe, że dla pełniejszego pokrycia potrzebne będą narzędzia dedykowane. Darmowe przykłady narzędzi obejmują:

- CryptoMon - Network Cryptography Monitor - using eBPF, written in python
- Cryptobom Forge Tool: Generating Comprehensive CBOMs from CodeQL Outputs

Równoważne siły parametrów kryptograficznych

Względne siły bezpieczeństwa różnych systemów kryptograficznych przedstawia poniższa tabela (z NIST SP 800-57 Part 1, s. 71):

Siła bezpieczeństwa	Algorytmy klucza symetrycznego	Ciało skończone	Faktoryzacja liczb całkowitych	Krzywa eliptyczna
<= 80	2TDEA	L = 1024 N = 160	k = 1024	f = 160-223
112	3TDEA	L = 2048 N = 224	k = 2048	f = 224-255
128	AES-128	L = 3072 N = 256	k = 3072	f = 256-383
192	AES-192	L = 7680 N = 384	k = 7680	f = 384-511
256	AES-256	L = 15360 N = 512	k = 15360	f = 512+

Przykłady zastosowań:

- Kryptografia ciał skończonych: DSA, FFDH, MQV
- Kryptografia faktoryzacji liczb całkowitych: RSA
- Kryptografia krzywych eliptycznych: ECDSA, EdDSA, ECDH, MQV

Uwaga: ta sekcja zakłada, że nie istnieje komputer kwantowy; gdyby taki komputer istniał, szacunki w trzech ostatnich kolumnach przestałyby być aktualne.

Wartości losowe

Ta sekcja dostarcza dodatkowych informacji dla V11.5 Wartości losowe.

Nazwa	Wersja/odniesienie	Uwagi	Status
/dev/random	Linux 4.8+ (październik 2016), obecny również w iOS, Androidzie i innych systemach POSIX opartych na Linuksie. Oparty na RFC7539	Wykorzystuje strumień ChaCha20. Obecny w iOS w SecRandomCopyBytes oraz w Androidzie w Secure Random, przy poprawnych ustawieniach każdego z nich.	A
/dev/urandom	Specjalny plik jądra Linuksa dostarczający dane losowe	Zapewnia wysokiej jakości źródła entropii z losowości sprzętowej	A
AES-CTR-DRBG	NIST SP800-90A	Stosowany w popularnych implementacjach, takich jak Windows CNG API BCryptGenRandom ustawiany przez BCRYPT_RNG_ALGORITHM.	A
HMAC-DRBG	NIST SP800-90A		A
Hash-DRBG	NIST SP800-90A		A
getentropy()	OpenBSD, dostępny w Linux glibc 2.25+ oraz macOS 10.12+	Dostarcza bezpieczne losowe bajty bezpośrednio ze źródła entropii jądra, z prostym i minimalnym API. Jest nowocześniejszy i unika pułapek związanych ze starszymi API.	A

Bazowa funkcja skrótu używana z HMAC-DRBG lub Hash-DRBG musi być zatwierdzona do tego zastosowania.

Algorytmy szyfrów

Ta sekcja dostarcza dodatkowych informacji dla V11.3 Algorytmy szyfrowania.

Zatwierdzone algorytmy szyfrów wymieniono w kolejności preferencji.

Algorytmy klucza symetrycznego	Odniesienie	Status
AES-256	FIPS 197	A
Salsa20	Salsa 20 specification	A
XChaCha20	XChaCha20 Draft	A
XSalsa20	Extending the Salsa20 nonce	A
ChaCha20	RFC 8439	A
AES-192	FIPS 197	A
AES-128	FIPS 197	L
2TDEA		D
TDEA (3DES/3DEA)		D
IDEA		D
RC4		D
Blowfish		D
ARC4		D
DES		D

Tryby szyfrowania AES

Szyfry blokowe, takie jak AES, mogą być używane w różnych trybach działania. Wiele trybów działania, takich jak Electronic codebook (ECB), jest niebezpiecznych i nie może być używanych. Tryby Galois/Counter Mode (GCM) oraz Counter with cipher block chaining message authentication code (CCM) zapewniają szyfrowanie uwierzytelnione i powinny być stosowane w nowoczesnych aplikacjach.

Zatwierdzone tryby wymieniono w kolejności preferencji.

Tryb	Uwierzytelniony	Odniesienie	Status	Ograniczenie
GCM	Tak	NIST SP 800-38D	A	
CCM	Tak	NIST SP 800-38C	A	
CBC	Nie	NIST SP 800-38A	L	
CCM-8	Tak		D	
ECB	Nie		D	
CFB	Nie		D	

Tryb	Uwierzytelniony	Odniesienie	Status	Ograniczenie
OFB	Nie		D	
CTR	Nie		D	

Uwagi:

- Wszystkie zaszyfrowane komunikaty muszą być uwierzytelnione. Dla KAŻDEGO użycia trybu CBC MUSI istnieć powiązany algorytm MAC oparty na funkcji skrótu, walidujący komunikat. Zasadniczo MUSI to być stosowane w metodzie Encrypt-Then-Hash (choć TLS 1.2 używa zamiast tego Hash-Then-Encrypt). Jeśli nie można tego zagwarantować, CBC NIE MOŻE być używany. Jedynym zastosowaniem, w którym dozwolone jest szyfrowanie bez algorytmu MAC, jest szyfrowanie dysków.
- Jeśli używany jest CBC, należy zagwarantować, że weryfikacja dopełnienia jest wykonywana w czasie stałym.
- Przy stosowaniu CCM-8 znacznik MAC ma jedynie 64 bity bezpieczeństwa. Nie spełnia to wymagania 6.2.9, które wymaga co najmniej 128 bitów bezpieczeństwa.
- Szyfrowanie dysków jest uznawane za pozostające poza zakresem ASVS. Dlatego niniejszy załącznik nie wymienia żadnej zatwierdzonej metody szyfrowania dysków. Dla tego zastosowania szyfrowanie bez uwierzytelniania jest zwykle akceptowane i typowo stosowane są tryby XTS, XEX oraz LRW.

Opakowywanie kluczy

Kryptograficzne opakowywanie klucza (key wrap, i odpowiadające mu odpakowywanie, key unwrap) to metoda ochrony istniejącego klucza poprzez jego enkapsulację (tj. opakowanie) z użyciem dodatkowego mechanizmu szyfrowania, tak aby oryginalny klucz nie był jawnie eksponowany, np. podczas transferu. Dodatkowy klucz używany do ochrony oryginalnego klucza nazywany jest kluczem opakującym (wrap key).

Operacja ta może być wykonywana, gdy pożądana jest ochrona kluczy w miejscach uznanych za niegodne zaufania albo przesyłanie wrażliwych kluczy przez niezaufane sieci lub wewnątrz aplikacji. Należy jednak poważnie rozważyć zrozumienie natury (np. tożsamości i przeznaczenia) oryginalnego klucza przed przystąpieniem do procedury opakowania/odpakowania, ponieważ może to mieć konsekwencje dla systemów/aplikacji źródłowych i docelowych w zakresie bezpieczeństwa, a zwłaszcza zgodności —co może obejmować ślady audytowe funkcji klucza (np. podpisywania) oraz właściwe przechowywanie kluczy.

W szczególności do opakowywania kluczy MUSI być używany AES-256, zgodnie z NIST SP 800-38F i z uwzględnieniem perspektywicznych zabezpieczeń przed zagrożeniem kwantowym. Tryby szyfrowania używające AES są następujące, w kolejności preferencji:

Opakowywanie kluczy	Odniesienie	Status
KW	NIST SP 800-38F	A
KWP	NIST SP 800-38F	A

AES-192 i AES-128 MOGĄ być używane, jeśli wymaga tego przypadek użycia, ale motywacja MUSI zostać udokumentowana w inwentarzu kryptograficznym podmiotu.

Szyfrowanie uwierzytelnione

Z wyjątkiem szyfrowania dysków zaszyfrowane dane muszą być chronione przed nieautoryzowaną modyfikacją z użyciem jakiejś formy schematu szyfrowania uwierzytelnionego (AE), zwykle schematu szyfrowania uwierzytelnionego z danymi powiązanymi (AEAD).

Aplikacja powinna preferencyjnie używać zatwierdzonego schematu AEAD. Alternatywnie może łączyć zatwierdzony schemat szyfru i zatwierdzony algorytm MAC w konstrukcji Encrypt-then-MAC.

MAC-then-encrypt jest nadal dozwolony dla zgodności ze starszymi aplikacjami. Jest używany w TLS v1.2 ze starymi zestawami szyfrów.

Mechanizm AEAD	Odniesienie	Status
AES-GCM	SP 800-38D	A
AES-CCM	SP 800-38C	A
ChaCha-Poly1305	RFC 7539	A
AEGIS-256	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
AEGIS-128L	AEGIS: A Fast Authenticated Encryption Algorithm (v1.1)	A
Encrypt-then-MAC		A
MAC-then-encrypt		L

Funkcje skrótu

Ta sekcja dostarcza dodatkowych informacji dla V11.4 Haszowanie i funkcje oparte na skrótach.

Funkcje skrótu do ogólnych przypadków użycia

Poniższa tabela wymienia funkcje skrótu zatwierdzone do ogólnych kryptograficznych przypadków użycia, takich jak podpisy cyfrowe:

- Zatwierdzone funkcje skrótu zapewniają silną odporność na kolizje i są odpowiednie dla aplikacji o wysokich wymaganiach bezpieczeństwa.
- Niektóre z tych algorytmów oferują silną odporność na ataki przy właściwym zarządzaniu kluczami kryptograficznymi, dlatego są dodatkowo zatwierdzone dla funkcji HMAC, KDF i RBG.
- Funkcje skrótu o długości wyjścia mniejszej niż 254 bity mają niewystarczającą odporność na kolizje i nie mogą być używane do podpisów cyfrowych ani innych zastosowań wymagających odporności na kolizje. W pozostałych zastosowaniach mogą być używane WYŁĄCZNIE dla zgodności i weryfikacji ze starszymi systemami, ale nie mogą być używane w nowych projektach.

Funkcja skrótu	Odniesienie	Status	Ograniczenia
SHA3-512	FIPS 202	A	
SHA-512	FIPS 180-4	A	
SHA3-384	FIPS 202	A	
SHA-384	FIPS 180-4	A	
SHA3-256	FIPS 202	A	
SHA-512/256	FIPS 180-4	A	
SHA-256	FIPS 180-4	A	
SHAKE256	FIPS 202	A	
BLAKE2s	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE2b	BLAKE2: simpler, smaller, fast as MD5	A	
BLAKE3	BLAKE3 one function, fast everywhere	A	
SHA-224	FIPS 180-4	L	Nieodpowiednia dla HMAC, KDF, RBG, podpisów cyfrowych
SHA-512/224	FIPS 180-4	L	Nieodpowiednia dla HMAC, KDF, RBG, podpisów cyfrowych

Funkcja skrótu	Odniesienie	Status	Ograniczenia
SHA3-224	FIPS 202	L	Nieodpowiednia dla HMAC, KDF, RBG, podpisów cyfrowych
SHA-1	RFC 3174 & RFC 6194	L	Nieodpowiednia dla HMAC, KDF, RBG, podpisów cyfrowych
CRC (dowolna długość)		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

Funkcje skrótu do przechowywania haseł

Do bezpiecznego haszowania haseł muszą być używane dedykowane funkcje skrótu. Te wolne algorytmy haszowania ograniczają ataki siłowe i słownikowe, zwiększając trudność obliczeniową łamania haseł.

KDF	Odniesienie	Wymagane parametry	Status
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$	A
		$t = 2: m \geq 19456$ (19 MiB), $p = 1$	A
		$t \geq 3: m \geq 12288$ (12 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$	A
		$p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$	A
		$p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A
bcrypt	A Future-Adaptable Password Scheme	$\text{cost} \geq 10$	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iteracje $\geq 210\,000$	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iteracje $\geq 600\,000$	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iteracje $\geq 1\,300\,000$	L

Zatwierdzone funkcje wyprowadzania klucza oparte na hasłach mogą być używane do przechowywania haseł.

Funkcje wyprowadzania klucza (KDF)

Ogólne funkcje wyprowadzania klucza

KDF	Odniesienie	Status
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
KDF oparte na MD5	RFC 1321	D
KDF oparte na SHA-1	RFC 3174 & RFC 6194	D

Funkcje wyprowadzania klucza oparte na hasłach

KDF	Odniesienie	Wymagane parametry	Status
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$	A
		$t = 2: m \geq 19456$ (19 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$	A
		$p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$	A
		$p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	iteracje $\geq 210\,000$	A
PBKDF2-HMAC-SHA-256	NIST SP 800-132, FIPS 180-4	iteracje $\geq 600\,000$	A
PBKDF2-HMAC-SHA-1	NIST SP 800-132, FIPS 180-4	iteracje $\geq 1\,300\,000$	L

Mechanizmy wymiany kluczy

Ta sekcja dostarcza dodatkowych informacji dla V11.6 Kryptografia klucza publicznego.

Schematy KEX

Dla wszystkich schematów wymiany kluczy MUSI być zapewniona siła bezpieczeństwa 112 bitów lub wyższa, a ich implementacja MUSI przestrzegać doboru parametrów z poniższej tabeli.

Schemat	Parametry dziedziny	Utajnianie z wyprzedzeniem	Status
Finite Field Diffie-Hellman (FFDH)	$L \geq 3072$ & $N \geq 256$	Tak	A
Elliptic Curve Diffie-Hellman (ECDH)	$f \geq 256-383$	Tak	A
Szyfrowany transport klucza z RSA-PKCS#1 v1.5		Nie	D

Gdzie poszczególne parametry oznaczają:

- k —rozmiar klucza dla kluczy RSA.
- L —rozmiar klucza publicznego, a N —rozmiar klucza prywatnego dla kryptografii ciał skończonych.
- f —zakres rozmiarów kluczy dla ECC.

Żadna nowa implementacja NIE MOŻE używać schematu NIEZGODNEGO z NIST SP 800-56A i B oraz NIST SP 800-77. W szczególności IKEv1 NIE MOŻE być używany w środowisku produkcyjnym.

Grupy Diffiego-Hellmana

Następujące grupy są zatwierdzone dla implementacji wymiany kluczy Diffiego-Hellmana. Siły bezpieczeństwa są udokumentowane w NIST SP 800-56A, Załącznik D, oraz NIST SP 800-57 Part 1 Rev.5.

Grupa	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A

Grupa	Status
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

Kody uwierzytelniania wiadomości (MAC)

Kody uwierzytelniania wiadomości (MAC) to konstrukcje kryptograficzne służące weryfikacji integralności i autentyczności komunikatu. MAC przyjmuje na wejściu komunikat i klucz tajny, a produkuje znacznik o stałym rozmiarze (wartość MAC). MAC są szeroko stosowane w protokołach bezpiecznej komunikacji (np. TLS/SSL), aby zapewnić, że komunikaty wymieniane między stronami są autentyczne i nienaruszone.

Algorytm MAC	Odniesienie	Status
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A
KMAC128	NIST SP 800-185	A
KMAC256	NIST SP 800-185	A
BLAKE3 (tryb keyed_hash)	BLAKE3 one function, fast everywhere	A
AES-CMAC	RFC 4493 & NIST SP 800-38B	A

Algorytm MAC	Odniesienie	Status
AES-GMAC	NIST SP 800-38D	A
Poly1305-AES	The Poly1305-AES message-authentication code	A
HMAC-SHA-1	RFC 2104 & FIPS 198-1	L
HMAC-MD5	RFC 1321	D

Podpisy cyfrowe

Schematy podpisów MUSZĄ używać zatwierdzonych rozmiarów kluczy i parametrów zgodnie z NIST SP 800-57 Part 1.

Algorytm podpisu	Odniesienie	Status
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA (dowolny rozmiar klucza)	FIPS 186-4	D

Postkwantowe standardy szyfrowania

Implementacje PQC muszą być zgodne z FIPS-203/204/205, ponieważ obecnie dostępnych jest niewiele utwardzonych przykładów kodu ani implementacji referencyjnych. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

Proponowana postkwantowa hybrydowa metoda uzgadniania klucza TLS mlkem768x25519 jest wspierana przez główne przeglądarki, takie jak Firefox w wydaniu 132 i Chrome w wydaniu 131. Może być stosowana w kryptograficznych środowiskach testowych albo gdy jest dostępna w bibliotekach zatwierdzonych branżowo lub rządowo.

Załącznik D: Zalecenia

Wprowadzenie

Podczas przygotowywania wersji 5.0 Application Security Verification Standard (ASVS) stało się jasne, że istnieje szereg pozycji –zarówno istniejących, jak i nowo zaproponowanych –które nie powinny

zostać włączone do wersji 5.0 jako wymagania. Wynikało to z faktu, że nie mieściły się w zakresie ASVS zgodnie z definicją przyjętą dla wersji 5.0, albo z przekonania, że choć stanowią dobry pomysł, nie mogą być obowiązkowe.

Nie chcąc całkowicie utracić tych pozycji, część z nich zebrano w niniejszym załączniku.

Zalecane mechanizmy w zakresie standardu

Następujące pozycje mieszczą się w zakresie ASVS. Nie powinny być obowiązkowe, ale zdecydowanie zaleca się rozważenie ich jako elementu bezpiecznej aplikacji.

- Powinien być udostępniony miernik siły hasła, pomagający użytkownikom ustawić silniejsze hasło.
- Utwórz publicznie dostępny plik `security.txt` w katalogu głównym lub `.well-known` aplikacji, jasno wskazujący link lub adres e-mail do kontaktu z właścicielami w sprawach bezpieczeństwa.
- Walidacja danych wejściowych po stronie klienta powinna być wymuszana obok walidacji w zaufanej warstwie usługowej, ponieważ stwarza to dobrą okazję do wykrycia, że ktoś ominął mechanizmy po stronie klienta, próbując zaatakować aplikację.
- Zapobiegaj pojawianiu się przypadkowo dostępnych i wrażliwych stron w wyszukiwarkach, używając pliku `robots.txt`, pola nagłówka odpowiedzi `X-Robots-Tag` lub znacznika meta `robots` w HTML.
- Przy korzystaniu z GraphQL implementuj logikę autoryzacji w warstwie logiki biznesowej zamiast w warstwie GraphQL lub resolverów, aby uniknąć konieczności obsługi autoryzacji w każdym osobnym interfejsie.

Źródła:

- Więcej informacji o `security.txt` wraz z linkiem do RFC

Zasady bezpieczeństwa oprogramowania

Następujące pozycje znajdowały się wcześniej w ASVS, ale nie są tak naprawdę wymaganiami. Są to raczej zasady, które warto brać pod uwagę przy implementacji mechanizmów bezpieczeństwa —ich przestrzeganie prowadzi do solidniejszych mechanizmów. Obejmują one:

- Mechanizmy bezpieczeństwa powinny być scentralizowane, proste (ekonomia projektowania), weryfikowalnie bezpieczne i wielokrotnego użytku. Powinno to zapobiegać mechanizmom zduplikowanym, brakującym lub nieskutecznym.
- Wszędzie tam, gdzie to możliwe, używaj wcześniej napisanych i dobrze sprawdzonych implementacji mechanizmów bezpieczeństwa, zamiast polegać na implementowaniu mechanizmów od zera.
- Najlepiej, aby do dostępu do chronionych danych i zasobów używany był jeden mechanizm kontroli dostępu. Wszystkie żądania powinny przechodzić przez ten jeden mechanizm, aby uniknąć kopiuj-wklej lub niebezpiecznych ścieżek alternatywnych.

- Kontrola dostępu oparta na atrybutach lub funkcjach to zalecany wzorzec, w którym kod sprawdza uprawnienie użytkownika do funkcji lub elementu danych, a nie jedynie jego rolę. Uprawnienia nadal powinny być przydzielane z użyciem ról.

Procesy bezpieczeństwa oprogramowania

Istnieje szereg procesów bezpieczeństwa, które zostały usunięte z ASVS 5.0, ale nadal są dobrym pomysłem. Projekt OWASP SAMM może być dobrym źródłem wiedzy o skutecznym wdrażaniu tych procesów. Pozycje wcześniej obecne w ASVS obejmują:

- Zweryfikuj stosowanie bezpiecznego cyklu wytwarzania oprogramowania, uwzględniającego bezpieczeństwo na wszystkich etapach rozwoju.
- Zweryfikuj stosowanie modelowania zagrożeń przy każdej zmianie projektowej lub planowaniu sprintu, aby identyfikować zagrożenia, planować środki zaradcze, ułatwiać właściwe reagowanie na ryzyko i ukierunkowywać testy bezpieczeństwa.
- Zweryfikuj, że wszystkie historyjki użytkownika i funkcje zawierają funkcjonalne ograniczenia bezpieczeństwa, takie jak „Jako użytkownik powinienem móc przeglądać i edytować swój profil. Nie powinienem móc przeglądać ani edytować profilu nikogo innego”.
- Zweryfikuj dostępność listy kontrolnej bezpiecznego kodowania, wymagań bezpieczeństwa, wytycznych lub polityki dla wszystkich programistów i testerów.
- Zweryfikuj, że istnieje ciągły proces zapewniający, że kod źródłowy aplikacji jest wolny od backdoorów, złośliwego kodu (np. ataki salami, bomby logiczne, bomby czasowe) oraz nieudokumentowanych lub ukrytych funkcji (np. easter egg, niebezpieczne narzędzia debugowania). Spełnienie tej sekcji nie jest możliwe bez pełnego dostępu do kodu źródłowego, w tym bibliotek stron trzecich, dlatego jest prawdopodobnie odpowiednie wyłącznie dla aplikacji wymagających najwyższych poziomów bezpieczeństwa.
- Zweryfikuj, że istnieją mechanizmy wykrywania i reagowania na dryf konfiguracji we wdrożonych środowiskach. Może to obejmować użycie infrastruktury niezmiennej (immutable), automatyczne ponowne wdrażanie z bezpiecznej linii bazowej lub narzędzia wykrywania dryfu porównujące bieżący stan z zatwierdzonymi konfiguracjami.
- Zweryfikuj, że utwardzanie konfiguracji jest wykonywane dla wszystkich produktów, bibliotek, frameworków i usług stron trzecich zgodnie z ich indywidualnymi zaleceniami.

Źródła:

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

Załącznik E - Współtwórcy

Z wdzięcznością odnotowujemy wkład następujących osób, które zgłaszały uwagi lub otwierały pull requesty od czasu wydania ASVS 4.0.0.

Jeśli zauważysz jakiegokolwiek błędy albo chcesz, aby Twoje nazwisko było wyświetlane inaczej, daj nam znać.

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii Dovydkov (oleksiidov)	IZUKA Masahiro (maizuka)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshitish9	Andrew van der Stock (vanderaj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Gargiullo (mgargiullo)	Raphael Dunant (Racater)
Cesar Kohl (cesarkohl)	inaz0	Joerg Bruenner (JoergBruenner)	David Deatherage (securitydave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	Matteo Pace (M4tteoP)	Sebastien gioria (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig-shony	Riccardo Sirigu (ricsirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael Green (RafaelGreen1)	mjang-cobalt
clallier94	Kevin W. Wall (kwwall)	Jordan Sherman (jsherm-fwdsec / deleterepo)	Ingo Rauner (ingo-rauner)
Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	Patrick Dwyer (coderpatros)	David Clarke (davidclarke-au)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoyasu Saburi (motoyasu-saburi)	leirn
wet-certitude	timhemel	RL Thornton (thornshadow99)	Thomas Bandt (aspnetde)
Roel Storms (roelstorms)	Jeroen Willemsen (commjoen)	anonymous-31	Kamran Saifullah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Herrera (hansphp)	Marx314
CarlosAllendes	Yonah Russ (yruss972)	Sander Majers (sanmai-NL)	Luboš Bretschneider (bretik)

Eva Sarafianou (esarafianou)	ataseren	Steve Thomas (Sc00bz)	Dominique RIGHETTO (righettod)
Steven van der Baan (svdb-ncc)	Michael Vacarella (Aif4thah)	Tonimir Kisasondi (tkisason)	Stefan Streichsbier (streichsbaer)
hi-unc1e	sb3k (starbuck3000)	mario-platt	Devdatta Akhawe (devd)
Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Wichers (davewichers)	Jonny Schnittger (JonnySchnittger)
Silvia Väli (silviavali)	jackgates73	1songb1rd	Timur - (timurozkul)
Gareth Heyes (hackvector)	appills	suvikaartinen	chaals (chaals)
DanielPharos (AtlasHackert)	will Farrell (willfarrell)	Alina Vasiljeva (avasiljeva)	Paul McCann (ismisepaul)
Sage (SajjadPourali)	rbsec	Benedikt Bauer (mastacheata)	James Jardine (jamesjardine)
Mark Burnett (m8urnett)	dschwarz91	Cyber-AppSec (Cyber-AppSec)	Tib3rius
BitnissWise (bitnisswise)	damienbod (damienbod)	Jared Meit (jmeit-fwdsec)	Stefan Seelmann (sseelmann)
Brendan O'Connor (ussjoin)	Andrei Titov (andrettv)	Hans-Petter Fjeld (atluxity)	markehack
Neil Madden (NeilMadden)	Michael Geramb (mgeramb)	Osama Elnaggar (ossie-git)	mackowski
Ravi Balla (raviballa)	Hazana (hazanasec)	David Means (dmeans82)	Alexander Stein (tohch4)
BaeSenseii (baesenseii)	Vincent De Schutter (VincentDS)	S Bani (sbani)	Mitsuaki Akiyama (mak1yama)
Christopher Loessl (hashier)	victorxm	Michal Rada (michalradacz)	Veeresh Devireddy (drveresh)
MaknaSEO	darkzero2022	Liam (LiamDobbelaere)	Frank Denis (jedisct1)
Otto Sulin (ottosulin)	carllaw6885	Anders Johan Holmefjord (aholmis)	Richard Fritsch (rfricz)
mesutgungor	Scott Helme (ScottHelme)	Carlo Reggiani (carloreggiani)	Suyash Srivastava (suyash5053)

Mark Potter (markonweb)	Arjan Lamers (alamers)	Gøran Breivik (gobrtg)	flo-blg
Guillaume Déflache (guillaume-d)	Toufik Airane (toufik-airane)	Keith Hoodlet (securingdev)	Sinner (SoftwareSinner)
iloving	Jeroen Beckers (TheDauntless)	Joubin Jabbari (joubin)	yu fujioka (fujiokayu)
execjosh (execjosh)	Alicja Kario (tomato42)	Sidney Ribeiro (srjsoftware)	Gabriel Marquet (Gby56)
Drew Schulz (drschulz)	bedirhan	muralito	Ronnie Flathers (ropnop)
Philippe De Ryck (philippederyck)	Malte (mal33)	MazeOfThoughts	Andreas Falk (andifalk)
Javi (javixeneize)	Daniel Hahn (averell23)	borislav-c	Robin Wood (digininja)
miro2ns	Jan Dockx (jandockx)	vipinsaini434	priyanshukumar397
Nat Sakimura (sakimura)	Benjamin Häublein (BenjaminHae)	unknown-user-from	Ali Ramazan TAŞDELEN (alitasdln)
Pedro Escalera (oEscal)	Josh (josh-hemphill)	Tim Würtele (SEctim)	AviD (avidouglen)
SheHacksPurple (shehackspurple)	fcerullo-cycubix	Hector Eryx Paredes Camacho (heryxpc)	Irene Michlin (irene221b)
Jonah Y-M (TG-Techie)	Dhiraj Bahroos (bahroos)	Jef Meijvis (jefmeijvis)	IzmaDoesItbeta
Abdessamad TEMMAR (TmmmmmR)	sectroyer	Soh Satoh (sohsatoh)	regoravalaz
james-t (james-bitherder)	Aram Hovsepyan (aramhovsepyan)	JaimeGomezGarciaSan	ValdiGit01
iwatachan (ishowta)	Vinod Anandan (VinodAnandan)	Kevin Kien (KevinKien)	paul-williamson- swoop
endergzzr	Radhwan Alshamamri (Rado0z)	Grant Ongers (rewtd)	Cure53 (cure53)
AliR2Linux	Ads Dawson (Gang- GreenTemperTatum)	William Reyor (BillReyor)	gabe (gcrow)
mascotter	luissaiz	Suren Manukyan (vx-sec)	Piotr Glizniewicz (pglizniewicz)

Tadeusz Wachowski (tadeuszwachowski)	Nasir aka Nate (andesecc)	settantasette	Lars Haulin (LarsH)
Terence Eden (edent)	JasmineScholz	Arun Sivadasan (teavanist)	Yusuf GÜR (yusuffgur)
Troy Marshall (troymarshall)	Tanner Prynn (tprynn)	Nick K. (nickific)	raoul361
Azeem Ilyas (TheAxZim)	Evo Stamatov (avioli)	Tim Potter (timpotter87)	Gavin Ray (GavinRay97)
monis (demideus)	Marcin Hoppe (MarcinHoppe)	Grambulf (ramshazar)	Jordan Pike (computersarebad)
Jason Rogers (jason-invision)	Ben Hall (benbhall)	JamesPoppyCock (jamesly123)	WhiteHackLabs (whitehacklabs)
Alex Gaynor (alex)	Filip van Laenen (filipvanlaenen)	jeurgen	GraoMelo
Andreas Kurtz (ay-kay)	Tom Tervoort (TomTervoort)	old man (deveras)	Marco Schnüriger (marcortw)
stiiin	infoseclearn (teaminfoseclearn)	hljupkij	Noe (nmarher)
Lyz (lyz-code)	Martin Riedel (mrtnrdl)	KIM Jaesuck (tcaesvk)	Barbara Schachner (bschach)
René Reuter (AresSec)	carhackpils	Tyler (tyler2cr)	Hugo (hasousa)
Wouter Bloeyaert (Someniak)	Mark de Rijk (markderijkinfosec)	Ramin (picohub)	Philip D. Turner (philipdturner)
Will Chatham (willc)			